

THE DESIGN AND IMPLEMENTATION OF HARDWARE SYSTEMS
FOR INFORMATION FLOW TRACKING

A DISSERTATION
SUBMITTED TO THE DEPARTMENT OF ELECTRICAL
ENGINEERING
AND THE COMMITTEE ON GRADUATE STUDIES
OF STANFORD UNIVERSITY
IN PARTIAL FULFILLMENT OF THE REQUIREMENTS
FOR THE DEGREE OF
DOCTOR OF PHILOSOPHY

Hari Kannan

April 2010

© 2010 by Hari S Kannan. All Rights Reserved.

Re-distributed by Stanford University under license with the author.



This work is licensed under a Creative Commons Attribution-Noncommercial 3.0 United States License.

<http://creativecommons.org/licenses/by-nc/3.0/us/>

This dissertation is online at: <http://purl.stanford.edu/hv823zb4872>

I certify that I have read this dissertation and that, in my opinion, it is fully adequate in scope and quality as a dissertation for the degree of Doctor of Philosophy.

Christoforos Kozyrakis, Primary Adviser

I certify that I have read this dissertation and that, in my opinion, it is fully adequate in scope and quality as a dissertation for the degree of Doctor of Philosophy.

Subhasish Mitra

I certify that I have read this dissertation and that, in my opinion, it is fully adequate in scope and quality as a dissertation for the degree of Doctor of Philosophy.

Oyekunle Olukotun

Approved for the Stanford University Committee on Graduate Studies.

Patricia J. Gumpert, Vice Provost Graduate Education

This signature page was generated electronically upon submission of this dissertation in electronic format. An original signed hard copy of the signature page is on file in University Archives.

Abstract

Computer security is a critical problem impacting every segment of social life. Recent research has shown that *Dynamic Information Flow Tracking (DIFT)* is a promising technique for detecting a wide range of security attacks. With hardware support, DIFT can provide comprehensive protection to unmodified application binaries against input validation attacks such as SQL injection, with minimal performance overhead. This dissertation presents Raksha, the *first* flexible hardware platform for DIFT that protects both unmodified applications, and the operating system from both low-level memory corruption exploits such as buffer overflows, and high-level semantic vulnerabilities such as SQL injections and cross-site scripting. Raksha uses tagged memory to support multiple, programmable security policies that can protect the system against concurrent attacks. It also describes the full-system prototype of Raksha constructed using a synthesizable SPARC V8 core and an FPGA board. This prototype provides comprehensive security protection with no false-positives and minimal performance, and area overheads.

Traditional DIFT architectures require significant changes to the processors and caches, and are not portable across different processor designs. This dissertation addresses this practicality issue of hardware DIFT and proposes an off-core coprocessor approach that greatly reduces the design and validation costs associated with hardware DIFT systems. Observing that DIFT operations and regular computation need only synchronize on system calls to maintain security guarantees, the coprocessor decouples all DIFT functionality from the main core. Using a full-system prototype based on a synthesizable SPARC core,

it shows that the coprocessor approach to DIFT provides the same security guarantees as Raksha, with low performance and hardware overheads. It also provides a practical and fast hardware solution to the problem of inconsistency between data and metadata in multiprocessor systems, when DIFT functionality is decoupled from the main core.

This dissertation also explores the use of tagged memory architectures for solving security problems other than DIFT. Recent work has shown that application policies can be expressed in terms of information flow restrictions and enforced in an OS kernel, providing a strong assurance of security. This thesis shows that enforcement of these policies can be pushed largely into the processor itself, by using tagged memory support, which can provide stronger security guarantees by enforcing application security even if the OS kernel is compromised. It presents the Loki architecture that uses tagged memory to directly enforce application security policies in hardware. Using a full-system prototype, it shows that such an architecture can help reduce the amount of code that must be trusted by the operating system kernel.

Acknowledgments

I am deeply indebted to many people for their contributions towards this dissertation, and the quality of my life while working on it.

It has been a privilege to work with Christos Kozyrakis, my thesis adviser. I am profoundly grateful for his persistent and patient mentoring, support, and friendship through my graduate career, starting from the day he called me to convince me to come to Stanford. I especially appreciate his honest and supportive advice, and his attention to detail while helping me polish my talks and papers. I have learned a lot from my interactions with him, which has helped me become a more competent engineer and researcher.

Over the years at Stanford, Subhasish Mitra has been a great sounding board for my ideas. His feedback on my work has been extremely useful, and his clarity of thought, inspirational. I am thankful to Kunle Olukotun for serving on my reading committee and to Krishna Saraswat for chairing the examining committee for my defense. I am also indebted to David Mazières, Monica Lam, and Dawson Engler for their help and feedback at various stages of my studies. As an undergraduate, I was fortunate to work with Sanjay Patel. I thank Sanjay for mentoring me as a researcher, and encouraging me to pursue my doctoral studies.

During the course of my research, I have had the good fortune of interacting with excellent partners in industry. I am grateful to Jiri Gaisler, Richard Pender, and the rest of the team at Gaisler Research for their numerous hours of support and help working with the

Leon processor. I would also like to thank Teresa Lynn for her untiring help with administrative matters, and Keith Gaul and Charlie Orgish for their technical support. My graduate studies have been generously funded by Cisco Systems through the Stanford Graduate Fellowships program, and by Intel through an Intel Foundation Fellowship.

This dissertation would not have been possible without my collaborators. A special thanks to my friend, philosopher, and colleague, Michael Dalton, who has worked with me on all my Raksha-related work, since my first day at Stanford. Mike's technical prowess and acerbic wit have helped enrich my graduate career immensely. I am also thankful to Nickolai Zeldovich for his guidance and help with the Loki project. JaeWoong Chung helped spice up our paper writing experience and conference trips immensely. I would also like to thank Ramesh Illikkal, Ravi Iyer, Mihai Budiu, John Davis, Sridhar Lakshmanamurthy, and Raj Yavatkar for their guidance and help during my internships. Finally, I appreciate the camaraderie and support of my current and former group-mates: Suzanne Rivoire, Chi Cao Minh, Jacob Leverich, Sewook Wee, Woongki Baek, Daniel Sanchez, Richard Yoo, Anthony Romano, and Austen McDonald. Jacob was an excellent system administrator for our group, without whose help, my RTL simulations would still be running.

On a more personal note, I've been fortunate to have had an amazing friend circle, both within and outside of Stanford, during my stay in the bay area. Angell Ct. has been a wonderfully happy abode, and I'm thankful to all the people who helped make it one. Many thanks to my extended family in the area, who took it upon themselves to feed me every so often. I've also been fortunate to have been associated with the Stanford chapter of Asha for Education. Asha's volunteers have continuously amazed me with their level of dedication and enthusiasm, and their company has made for some delightful times. And yes, Holi at Stanford rocks! A few acronyms that have helped me preserve my sanity during times of stress: ARR, MDR, SSI, LGJ, MMI, PMI, TNK, TS, IR, BCL, SRT, RSD, CM, KH, HH, PGW, YM, YPM.

Finally, I am deeply indebted to my family for the opportunities and support that they

provided me. My mother and sister have been loving and supportive presences, and learned early not to ask when the Ph.D. would be completed. My father has been an untiring source of sound guidance and advice, which has stood me in good stead. My grandmother has been a pillar of strength, and has constantly amazed me with her dedication and discipline.

My life has been enriched by innumerable people who I cannot begin to thank enough. Saint Tyagaraja's catch-all acknowledgment comes to my rescue: "endarO mahAnubhavulu antarIki vandanamu".

Contents

Abstract	iv
Acknowledgments	vi
1 Introduction	1
1.1 Contributions	3
1.2 Thesis Organization	5
2 Background and Motivation	7
2.1 Requirements of Ideal Security Solutions	8
2.2 Dynamic Information Flow Tracking	9
2.3 DIFT Implementations	11
2.3.1 Programming language platforms	11
2.3.2 Dynamic binary translation	12
2.3.3 Hardware DIFT	13
2.4 Summary	14
3 Raksha - A Flexible Hardware DIFT Architecture	16
3.1 DIFT Design Requirements	16
3.1.1 Hardware management of Tags	17
3.1.2 Multiple flexible security policies	18

3.1.3	Software analysis support	19
3.2	The Raksha Architecture	20
3.2.1	Architecture overview	21
3.2.2	Tag propagation and checks	23
3.2.3	User-level security exceptions	26
3.2.4	Discussion	28
3.3	Related Work	29
3.4	Conclusions	30
4	The Raksha Prototype System	32
4.1	The Raksha Prototype System	32
4.1.1	Hardware implementation	33
4.1.2	Software implementation	39
4.2	Security Evaluation	40
4.2.1	Security policies	40
4.2.2	Security experiments	43
4.3	Performance Evaluation	45
4.4	Summary	48
5	A Decoupled Coprocessor for DIFT	49
5.1	Design Alternatives for Hardware DIFT	49
5.2	Design of the DIFT Coprocessor	53
5.2.1	Security model	53
5.2.2	Coprocessor microarchitecture	56
5.2.3	DIFT coprocessor interface	57
5.2.4	Tag cache	60
5.2.5	Coprocessor for in-order cores	61
5.3	Prototype	61

5.3.1	System architecture	62
5.3.2	Design statistics	62
5.4	Evaluation	66
5.4.1	Security evaluation	66
5.4.2	Performance evaluation	69
5.5	Summary	76
6	Metadata Consistency in Multiprocessor Systems	77
6.1	(<i>Data, metadata</i>) Consistency	78
6.1.1	Overview of the (in)consistency problem	78
6.1.2	Requirements of a solution	79
6.1.3	Previous efforts	80
6.2	Protocol for (<i>data, metadata</i>) Consistency	81
6.2.1	Protocol overview	81
6.2.2	Protocol implementation	83
6.2.3	Example	86
6.2.4	Performance issues	87
6.3	Practicality and Applicability	89
6.3.1	Coherence protocol	89
6.3.2	Memory consistency model	90
6.3.3	Metadata length	91
6.3.4	Analysis issues	93
6.4	Experimental Results	94
6.4.1	Baseline execution	95
6.4.2	Scaling the hardware structures	98
6.4.3	Smaller tags	99
6.5	Summary	101

7	Enforcing Application Security Policies using Tags	102
7.1	Motivation	103
7.2	Requirements for Dynamic Information Flow Control Systems	105
7.2.1	Tag management	105
7.2.2	Tag manipulation	106
7.2.3	Security exceptions	106
7.3	System Architecture	107
7.3.1	Application perspective	110
7.3.2	Hardware overview	111
7.3.3	OS overview	113
7.4	Microarchitecture	114
7.4.1	Memory tagging	114
7.4.2	Granularity of tags	115
7.4.3	Permissions cache	116
7.4.4	Device access control	117
7.4.5	Tag exceptions	118
7.5	Prototype Evaluation	119
7.5.1	Loki prototype	119
7.5.2	Trusted code base	121
7.5.3	Performance	122
7.5.4	Tag usage and storage	124
7.6	Related Work	126
7.7	Summary	128
8	Generalizing Tag Architectures	129
8.1	Debugging	130
8.1.1	Tag storage and manipulation	130

8.1.2	Decoupling the hardware analysis	131
8.2	Profiling	131
8.2.1	Tag storage and manipulation	132
8.2.2	Decoupling the hardware analysis	132
8.3	Pointer bits	133
8.3.1	Tag storage and manipulation	133
8.3.2	Decoupling the hardware analysis	134
8.4	Full/empty bits	134
8.4.1	Tag storage and manipulation	134
8.4.2	Decoupling the hardware analysis	135
8.5	Fault Tolerance and Speculative Execution	135
8.5.1	Tag storage and manipulation	136
8.5.2	Decoupling the hardware analysis	136
8.6	Transactional Memory and Cache QoS	136
8.6.1	Tag storage and manipulation	137
8.6.2	Decoupling the hardware analysis	137
8.7	Generalizing Architectures for Hardware Tags	138
8.8	Related Work	141
8.9	Summary	142
9	Conclusions	144
9.1	Future Work	145
	Bibliography	147

List of Tables

4.1	The new pipeline registers added to the Leon pipeline by the Raksha architecture.	34
4.2	The new instructions added to the SPARC V8 ISA by the Raksha architecture.	35
4.3	The architectural and design parameters for the Raksha prototype.	36
4.4	The area and power overhead values for the storage elements in the Raksha prototype. Percentage overheads are shown relative to the corresponding data storage structures in the unmodified Leon design.	38
4.5	Summary of the security policies implemented by the Raksha prototype. The four tag bits are sufficient to implement six concurrently active policies to protect against both low-level memory corruption and high-level semantic attacks.	41
4.6	The DIFT propagation rules for the taint and pointer bits. r_y stands for register y . $T[x]$ and $P[x]$ refer to the taint (T) or pointer (P) tag bits respectively for memory location, register, or instruction x	42
4.7	The DIFT check rules for BOF detection. A security exception is raised if the condition in the rightmost column is true.	42
4.8	The high-level semantic attacks caught by the Raksha prototype.	43
4.9	The low-level memory corruption exploits caught by the Raksha prototype.	44

4.10	Normalized execution time after the introduction of the pointer-based buffer overflow protection policy. The execution time without the security policy is 1.0. Execution time higher than 1.0 represents performance degradation.	46
5.1	The prototype system specification.	61
5.2	Complexity of the prototype FPGA implementation of the DIFT coprocessor in terms of FPGA block RAMs and 4-input LUTs.	63
5.3	The area and power overhead values for the storage elements in the offcore prototype. Percentage overheads are shown relative to corresponding data storage structures in the unmodified Leon design.	66
5.4	The security experiments performed with the DIFT coprocessor.	67
6.1	Comparison of different schemes for maintaining (<i>data, metadata</i>) consistency.	79
6.2	Simulation infrastructure and setup.	94
7.1	The architectural and design parameters for our prototype of the Loki architecture.	120
7.2	Complexity of our prototype FPGA implementation of Loki in terms of FPGA block RAMs and 4-input LUTs.	121
7.3	Complexity of the original <i>trusted</i> HiStar kernel, the <i>untrusted</i> LoStar kernel, and the <i>trusted</i> LoStar security monitor. The size of the LoStar kernel includes the security monitor, since the kernel uses some common code shared with the security monitor. The bootstrapping code, used during boot to initialize the kernel and the security monitor, is not counted as part of the TCB because it is not part of the attack surface in our threat model.	122
7.4	Tag usage under different workloads running on LoStar.	125
8.1	Comparison of different tag analyses.	138

List of Figures

3.1	The tag abstraction exposed by the hardware to the software. At the ISA level, every register and memory location appears to be extended by four tag bits.	21
3.2	The format of the Tag Propagation Register. There are 4 TPRs, one per active security policy.	23
3.3	The format of the Tag Check Register. There are 4 TCRs, one per active security policy.	24
3.4	The logical distinction between trusted mode and traditional user/kernel privilege levels. Trusted mode is orthogonal to the user or kernel modes, allowing for security exceptions to be processed at the privilege level of the program.	26
4.1	The Raksha version of the pipeline for the Leon SPARC V8 processor. . . .	33
4.2	The GR-CPCI-XC2V board used for the prototype Raksha system.	37
4.3	The performance degradation for a microbenchmark that invokes a security handler of controlled length every certain number of instructions. All numbers are normalized to a baseline case which has no tag operations. . .	47
5.1	The three design alternatives for DIFT architectures.	50
5.2	The pipeline diagram for the DIFT coprocessor. Structures are not drawn to scale.	55

5.3	Execution time normalized to an unmodified Leon.	70
5.4	Comparison of the coprocessor approach against the hardware assisted off-loading approach.	71
5.5	The effect of scaling the capacity of the tag cache.	73
5.6	The effect of scaling the size of the decoupling queue on a worst-case tag initialization microbenchmark.	74
5.7	Performance overhead when the coprocessor is paired with higher-IPC main cores. Overheads are relative to the case when the main core and coprocessor have the same clock frequency.	75
6.1	An inconsistency scenario where updates to data and metadata are observed in different orders.	78
6.2	Overview of the system showing a single (a-core, m-core) pair. Structures are not drawn to scale.	83
6.3	The three tables added to the system.	83
6.4	Good ordering of metadata accesses.	86
6.5	Graphical representation of the protocol. AC stands for a-core, MC for m-core, and IC for Interconnect. Addr refers to the variable's memory address.	87
6.6	Deadlock scenario with the TSO consistency model.	90
6.7	Performance of Canneal when the number of processors is scaled.	95
6.8	Performance of PARSEC and SPLASH-2 benchmarks with 32 processors.	96
6.9	Scaling the PTAT/PTRT sizes with a small decoupling interval on a worst-case lock contention microbenchmark.	97
6.10	Scaling the PTAT/PTRT sizes with a large decoupling interval on a worst-case lock contention microbenchmark.	98
6.11	The overheads of using smaller tags on Ocean, and a heap traversal microbenchmark (MB).	100

7.1	A comparison between (a) traditional operating system structure, and (b) this chapter's proposed structure using a security monitor. Horizontal separation between application boxes in (a), and between stacks of applications and kernels in (b), indicates different protection domains. Dashed arrows in (a) indicate access rights of applications to pages of memory. Shading in (b) indicates tag values, with small shaded boxes underneath protection domains indicating the set of tags accessible to that protection domain. . . .	107
7.2	A comparison of the <i>discretionary access control</i> and <i>mandatory access control</i> threat models. Rectangles represent data, such as files, and rounded rectangles represent processes. Arrows indicate permitted information flow to or from a process. A dashed arrow indicates information flow permitted by the discretionary model but prohibited by the mandatory model. . . .	110
7.3	The tag abstraction exposed by the hardware to the software. At the ISA level, every register and memory location appears to be extended by 32 tag bits.	112
7.4	The Loki pipeline, based on a traditional pipelined SPARC processor. . . .	114

7.5	Relative running time (wall clock time) of benchmarks running on unmodified HiStar, on LoStar, and on a version of LoStar without page-level tag support, normalized to the running time on HiStar. The <i>primes</i> workload computes the prime numbers from 1 to 100,000. The <i>syscall</i> workload executes a system call that gets the ID of the current thread. The <i>IPC ping-pong</i> workload sends a short message back and forth between two processes over a pipe. The <i>fork/exec</i> workload spawns a new process using <code>fork</code> and <code>exec</code> . The <i>small-file</i> workload creates, reads, and deletes 1000 512-byte files. The <i>large-file</i> workload performs random 4KB reads and writes within a single 4MB file. The <i>wget</i> workload measures the time to download a large file from a web server over the local area network. Finally, the <i>gzip</i> workload compresses a 1MB binary file.	123
-----	---	-----

Chapter 1

Introduction

It is widely recognized that computer security is a critical problem with far-reaching financial and social implications [72]. Despite significant development efforts, existing security tools do not provide reliable protection against an ever-increasing set of attacks, worms, and viruses that target vulnerabilities in deployed software. Apart from memory corruption bugs such as buffer overflows, attackers are now focusing on high-level exploits such as SQL injections, command injections, cross-site scripting and directory traversals [36, 83]. Worms that target multiple vulnerabilities in an orchestrated manner are also becoming increasingly common [11, 83]. Hence, research on computer system security is timely.

The root of the computer security problem is that existing protection mechanisms do not exhibit many of the desired characteristics of an ideal security technique. They should be *safe*: provide defense against vulnerabilities with no false positives or negatives; *flexible*: adapt to cover evolving threats; *practical*: work with real-world code (including legacy binaries, dynamically generated code, or operating system code) without assumptions about compilers or libraries; and *fast*: have small impact on application performance. Additionally, they must offer clean abstractions for expressing security policies, in order to be implementable in practice.

Recent research has established *Dynamic Information Flow Tracking (DIFT)* [28, 70]

as a promising platform for detecting a wide range of security attacks. The idea behind DIFT is to tag (taint) untrusted data and track its propagation through the system. DIFT associates a tag with every word of memory in the system. Any new data derived from untrusted data is also tainted. If tainted data is used in a potentially unsafe manner, such as the execution of a tagged SQL command or the dereferencing of a tagged pointer, a security exception is raised.

The generality of the DIFT model has led to the development of several software [17, 19, 52, 66, 67, 71, 73, 93] and hardware [14, 20, 81] implementations. Nevertheless, current DIFT systems are far from ideal. Software DIFT is flexible, as it can enforce arbitrary policies and adapt to protect against different types of exploits. One technique for implementing software DIFT is to add tainting capabilities in the interpreter or runtime of languages like PHP [67, 26] to catch semantic attacks such as SQL injections. These systems, however, cannot address low-level vulnerabilities such as buffer overflows, and are *unsafe* against certain types of attacks. Furthermore, this approach is *impractical* if the user wants to protect against vulnerabilities occurring in multiple languages, as this technique is language-specific. Software DIFT can also be performed through runtime binary instrumentation, by having a dynamic binary translator insert code that performs DIFT checks. This technique, however, can lead to *slowdowns* ranging from $3\times$ to $37\times$ [66, 73]. Additionally, some software systems require access to the source code [93], while others do not work safely with multithreaded programs [73].

An alternate approach to DIFT is to perform the security checks directly in the hardware. Current proposed hardware DIFT systems address the performance and practicality issues of software DIFT systems, but suffer from other inadequacies. These systems use hardcoded security policies that are *inflexible* and cannot adapt to newer attacks, cannot protect the operating system, and suffer from *false positives and negatives* in real-world code. Additionally, they are *impractical*, since they require extensive and invasive changes

to the processor design, thereby increasing design and validation costs for processor vendors.

This dissertation explores the construction of hardware DIFT systems that can provide comprehensive and robust protection from a wide variety of low-level memory and high-level semantic attacks, are flexible enough to keep pace with the ever-evolving threat landscape, and have minimal area, performance, and power overheads.

1.1 Contributions

This dissertation explores the potential of hardware DIFT to provide comprehensive protection from a wide variety of attacks on real-world applications. It focuses on input validation vulnerabilities such as SQL injection, buffer overflows, and cross-site scripting. Input validation attacks occur because a non-malicious, but vulnerable application did not correctly validate untrusted user input. Other areas of computer security such as malware analysis, DRM, and cryptography are outside the scope of this work.

The main contributions of this dissertation are the following:

- It presents Raksha, the *first* flexible hardware DIFT platform that prevents attacks on unmodified binaries, and even the operating system. Raksha provides a framework that combines the best of both hardware and software DIFT platforms. Hardware support provides transparent, fine-grain management of security tags at low performance overhead for user code, OS code, and data that crosses multiple processes. Software provides the flexibility and robustness necessary to deal with a wide range of attacks. Raksha supports multiple active security policies and employs user-level exceptions that help apply DIFT policies to the operating system.
- It describes the implementation of a fully-featured Linux workstation prototype for Raksha using a synthesizable SPARC core and an FPGA board. Running real-world

software on the prototype, Raksha is the first DIFT architecture to detect high-level vulnerabilities such as directory traversals, command injection, SQL injection, and cross-site scripting, while providing protection against conventional memory corruption attacks both in userspace and in the kernel. All experiments were performed on unmodified binaries, with no debugging information.

- It addresses the practicality concerns of traditional DIFT hardware architectures that require significant changes to the processors and caches, and presents an off-core, decoupled coprocessor that encapsulates all the DIFT functionality in order to reduce the hardware costs associated with implementing DIFT. This approach requires no change to the design, pipeline and layout of a general-purpose core, simplifies design and verification, and enables reuse of DIFT logic with different families of processors. Using a full-system prototype based on a synthesizable SPARC core and an FPGA board, it shows that the coprocessor approach to DIFT provides the same security guarantees as traditional DIFT implementations such as Raksha, with minimal performance and hardware overheads.
- It provides a practical and fast hardware solution to the problem of inconsistency between data and metadata in multiprocessor systems, when DIFT functionality is decoupled from the main core. It leverages cache coherence to record interleaving of memory operations from application threads and replays the same order on metadata processors to maintain consistency, thereby allowing correct execution of dynamic analysis on multithreaded programs.
- It explores using tagged memory architectures to solve security problems other than those addressed by DIFT. To this end, it presents the Loki architecture that uses tagged memory to enforce an application's security policies directly in hardware. Loki simplifies security enforcement by associating security policies with data at the lowest level in the system – in physical memory. It shows how HiStar, an existing

operating system, can take advantage of such a tagged memory architecture to enforce its information flow control policies directly in hardware, and thereby reduce the amount of trusted code in its kernel by over a factor of two. Using a full-system prototype built with a synthesizable SPARC core and an FPGA board, it shows that the overheads of such an architecture are minimum.

- It also discusses various other dynamic analysis applications that make use of memory tags. It also motivates the use of a general tagged memory architecture that implements a set of features required by a whole suite of dynamic analyses, by listing requirements and implementation techniques for the same. Such an architecture would allow for design reuse, and help amortize the cost of implementing hardware support for tags, for processor vendors.

1.2 Thesis Organization

The rest of this thesis is organized as follows. Chapter 2 provides an overview of DIFT, and discusses the different proposed implementations of DIFT. In Chapter 3, we detail the characteristics of an ideal, flexible DIFT system, and introduce the Raksha DIFT architecture. Chapter 4 deals with the Raksha prototype system, and discusses the performance and area overheads of the design. It also studies the security capabilities of the architecture, and demonstrates its effectiveness at preventing security attacks.

In Chapter 5, we explain the practicality challenges of implementing a hardware DIFT solution. We then present a coprocessor architecture for DIFT that encapsulates all the DIFT functionality and obviates the need for modifying the main core. We study the implications of such a design on the performance, power, and security of the system. Chapter 6 explains the problem of inconsistency between data and metadata under decoupling in multi-threaded binaries. It then proceeds to detail a hardware solution that leverages cache coherency to record interleavings of memory operations. Finally, it studies the impact of

this solution on the performance of the system.

In Chapter 7, we present an alternative system that makes use of tagged hardware for information flow control. We introduce the Loki architecture that allows for direct enforcement of application security policies in hardware, and use a full-system prototype to study its design properties, security and performance. Chapter 8 surveys a variety of applications that make use of tagged memory, and provides a qualitative discussion on the design of a unified tag architecture framework for dynamic analysis. Finally, Chapter 9 concludes the dissertation and proposes future directions for research.

Chapter 2

Background and Motivation

Computer security has been an extremely fertile area of research over the past three decades. While computer security covers many topics including data encryption, content protection, and network trustworthiness [72], this thesis focuses on the detection of input validation attacks on deployed software. These exploits occur when a vulnerable application does not correctly validate malicious user input. Low level memory corruption exploits such as buffer overflows and format string attacks continue to remain a critical threat to modern system security, even though they have been prevalent for over 25 years. On the other end of the spectrum, with the proliferation of the internet, high-level web security attacks such as SQL injections, and cross-site scripting are rapidly becoming the preferred mode of attack for hackers. While there have been many protection mechanisms proposed for solving each of these problems individually, none of the proposed solutions provide comprehensive protection against a whole range of attacks. Additionally, most of these mechanisms suffer from various inadequacies such as insufficient coverage, or lack of compatibility with real-world code [22].

The rest of this chapter is organized as follows. Section 2.1 introduces the desired characteristics of ideal security solutions. Section 2.2 introduces dynamic information flow tracking, and provides a thorough overview of the same. In Section 2.3, we review the

different methods of implementing information flow tracking. Section 2.4 concludes the chapter.

2.1 Requirements of Ideal Security Solutions

In this section, we list the characteristics desired of security mechanisms:

- **Robustness:** They should provide defense against vulnerabilities with few false positives or false negatives. Security techniques such as the Non-executable Data page protection to prevent buffer overflows have been rendered useless by novel attacks that overwrite only data or data pointers [15]. At the same time, overly restrictive security policies could break backwards compatibility by flagging benign cases as security faults, greatly reducing the utility of the protection mechanism.
- **Flexibility:** They should adapt to provide protection against evolving threats. The landscape of security attacks is extremely dynamic and ever-changing. It is important for any protection mechanism proposed to have the ability to keep up with this evolving threat landscape. Fixing or hardcoding security policies impairs the ability of the system to do so. While the Non-executable Data page protection prevented most common forms of buffer overflow attacks prevalent at the time, it did not take long for attackers to adapt. Instead of injecting their own code, attackers began to transfer control to existing application code to gain control over the vulnerable application using a technique called return-into-libc [64].
- **End-to-end coverage:** They should be applicable to user programs, libraries, and even the operating system. Modern machines consist of applications, program libraries, operating systems, virtual machine monitors, and hardware in a precariously balanced ecosystem. A flaw in any one of these components could result in a full-system compromise. Security techniques must thus have the ability to scale beyond

individual components, and offer full-system protection.

- **Practicality:** They should work with real-world code and software models (existing binaries, dynamically generated, or extensible code) without specific assumptions about compilers or libraries. For any security mechanism to be practically viable, it is important that it be applicable to existing binaries. Many commonly used programs exist only in the raw binary format; thus, any mechanism requiring code re-compilation would not be able to support such programs. Additionally, the security mechanism must not break backwards-compatibility with legacy code. A recent exploit for Adobe Flash was able to bypass the Address Space Layout Randomization (ASLR) protection mechanism because one of Adobe's libraries was not compatible with ASLR, thus leading to ASLR being disabled [57].
- **Speed:** They should be fast and have a small impact on application performance. Large performance overheads would lead to users choosing speed over security, and disabling the protection mechanism employed.

2.2 Dynamic Information Flow Tracking

Dynamic information flow tracking (DIFT) [28, 70] is a promising platform for detecting a wide range of security attacks. DIFT tracks the runtime flow of untrusted information through the program when executing in a *runtime environment*, and prevents untrusted data from being used in an unsafe manner. This runtime environment may be implemented in software (in a virtual machine, or a dynamic runtime system), or in hardware (in a processor). DIFT associates tags with memory and resources in the system, and uses these tags to maintain information about the trustedness of the corresponding data. The flow of information through the program is tracked by use of these tags. DIFT policies are used to configure the *tag initialization*, *tag propagation*, and *tag check* rules of the system. Tags

are initialized in accordance with the source of the data. A typical tag initialization policy would be to mark data arriving from untrusted sources such as the network as tainted, while keeping files owned by the user untainted. Tag propagation refers to the combining of tags of the source operands to generate the destination operand's tag. As every instruction is processed by the program, the corresponding metadata operation must be performed by the runtime environment. For e.g, an arithmetic operation must combine the tags of the operands in accordance with the tag propagation policies, and in parallel with the data processing. Tag checks are then performed in accordance with the configured policies to check for security violations. A security exception is raised in the case of an unsafe use of untrusted information, such as the dereferencing of an untrusted pointer, or the use of a tainted SQL command.

DIFT is an extremely powerful and promising security technique that has the potential to satisfy all the requirements of an ideal security mechanism detailed earlier. DIFT is *safe* and has been shown to catch a wide range of security attacks ranging from low-level memory corruption exploits such as buffer overflows to high-level semantic vulnerabilities such as SQL injection, cross-site scripting and directory traversal [12, 14, 20, 65, 66, 73, 81, 88]. No other security technique has been shown to be applicable to such a wide spectrum of attacks. The *flexibility* of the DIFT model has allowed for a myriad of implementations at various levels of abstraction, such as preventing Java servlet vulnerabilities in the JVM, or preventing memory corruption exploits in hardware. Implementations of DIFT exist in most scripting languages (PHP [67], Java [51]), in dynamic binary translators [65], and in hardware [14]. DIFT is *practical* since it does not require any knowledge about the internals or semantics of programs. This allows DIFT to work on unmodified binaries or bytecode, without requiring any source code or debugging information. DIFT has been shown to provide *end-to-end* protection on systems by securing both operating systems and userspace programs [5] against attacks. DIFT implementations can also be *fast* as evinced by some of the high-performance DIFT systems built [14, 73, 81]. Fundamentally, DIFT

provides a clean abstraction for expressing and enforcing security policies, thereby lending itself to practical implementations.

2.3 DIFT Implementations

Owing to the popularity and versatility of the DIFT security model, researchers have explored applying DIFT to software security in a number of environments.

2.3.1 Programming language platforms

One approach to applying DIFT is via language DIFT implementations, where DIFT capabilities are added to a language interpreter or runtime. Researchers have proposed DIFT implementations for many languages, such as PHP [67] and Java [33]. Additionally, DIFT concepts are already used in limited situations by many existing interpreted languages, such as the taint mode found in Perl [70] and Ruby [84]. In such implementations, the language interpreter serves as the runtime environment. From a DIFT perspective, memory consists of language variables which are extended to accommodate taint.

Language platforms for DIFT are very flexible, and have been shown to provide good protection against high-level vulnerabilities, with low performance overheads [22, 26]. Researchers have modified the interpreters of dynamic languages such as PHP to provide protection against a wide variety of semantic, web-based input validation bugs such as SQL injection, and cross-site scripting.

The downside to language DIFT platforms is their inability to address vulnerabilities such as low-level memory corruption exploits, or operating system errors. Additionally, since this technique is language-specific, it is impractical in defending against vulnerabilities that occur in a wide variety of languages.

2.3.2 Dynamic binary translation

Another method of applying DIFT in software is using a Dynamic Binary Translator (DBT). In a DBT-based DIFT implementation, the application (or even the entire system) is run within a DBT. The binary translation framework maintains *metadata*, or state associated with the application's data. This metadata is used to maintain information about the taint-ness of the associated data. The DBT dynamically inserts instructions for DIFT when performing binary translation. Every instruction from the application has an associated metadata instruction that manipulates the associated taint values.

Dynamic binary translators have been used for performing DIFT both on individual programs [65], and the entire system [5]. Since the security analysis is performed in software, the policies employed can be arbitrarily complex and flexible. This provides the advantage of being able to use the same infrastructure for a wide range of policies. Binary translation however, requires the introduction of a whole new instruction to manipulate the taint associated with the original program's instruction. The disadvantage of this scheme is the high performance overhead. DBT-based DIFT systems have been shown to have performance overheads ranging from $3\times$ [73] to $37\times$ [66] depending upon the application and policies in question. Applying DIFT support to the entire system requires that the DBT solution virtualize all devices, the MMU, the OS, and all applications. Overheads of performing this virtualization alone using whole-system binary translation frameworks such as QEMU, are between $5\times$ to $20\times$ [5]. Adding DIFT support increases these overheads significantly. Such high performance overheads restrict the wide-spread applicability of a DBT-based DIFT solution.

Another drawback with binary translation frameworks is the lack of support for multi-threaded applications. When executing a multi-threaded workload, the DIFT platform must ensure consistency between updates to data and tags, so that all other threads in the system perceive these updates as atomic operations [18]. Failing to do so could cause race

conditions that could lead to false negatives (undetected security breaches) or false positives (spurious security exceptions), which undermine the utility of the DIFT mechanism. Software DBT schemes deal with this issue by either forgoing support for multiple threads entirely [9, 73], restricting applications to only execute a single thread at a time [65], or requiring tool developers to explicitly implement the locking mechanisms needed to access metadata [54]. Since many security critical workloads such as databases and web servers are multithreaded, this limits the practicality and applicability of the DBT DIFT solution. Recent research into hybrid DIFT systems has shown that with additional hardware support, multithreaded applications can be run within DBTs [40], but this requires significant hardware modifications to existing systems.

2.3.3 Hardware DIFT

An alternative approach to DIFT is to perform the taint tracking and checking in hardware [14, 20, 81]. The hardware is responsible for maintaining and managing the state associated with taint tracking. Hardware being the lowest layer of abstraction in a computer system is the ideal level for implementing DIFT support. All programs, binaries and executables must run on top of the hardware. Implementing DIFT mechanisms in hardware allows the DIFT security policies to be applied to scripting languages, binaries, applications, or even operating systems. This renders the protection independent of the choice of programming language, since all languages must eventually be translated to some form of assembly language understood by the hardware.

This approach has a very low performance overhead as tag propagation and checks occur in hardware, often in parallel with the execution of the original instruction. Hardware DIFT systems provides extremely low-overhead protection, even when applied to the whole operating system. Tag propagation occurs in hardware, often in parallel with the execution of the original data instruction. Additionally, hardware can apply DIFT policies to the

whole system without the performance and complexity challenges faced by whole-system dynamic binary translation.

Unlike DBT-based solutions, hardware DIFT platforms can also apply protection to multi-threaded applications. This can be done either by ensuring atomic updates to both data and tags [24, 41], or by making minor modifications to the coherence protocols to ensure that an atomic view of data and tags is always presented to other processors [40]. Since computer systems are migrating to multi-core environments, such support is key in ensuring the practical viability of the DIFT solution. Overall, hardware DIFT support has been shown to provide comprehensive support against both low-level memory corruption exploits such as buffer overflows [20, 81], and high-level web attacks such as SQL injections [66], with low performance overheads.

The downside to hardware DIFT systems, however, is their inflexibility. Hardware architectures implemented thus far use single fixed security policies to catch all classes of attacks. Worms that target multiple vulnerabilities are however, becoming exceedingly common [11]. Such worms can bypass the protection offered by current hardware DIFT architectures, since they can protect against only one kind of exploit using a solitary security policy. Casting security policies in silicon impairs the ability of the solution to adapt to future threats, and limits the utility of the solution. Modern software is extremely complex and ridden with corner cases that often require special handling. The lack of flexibility restricts the ability of a hardware DIFT system to handle such cases. We discuss this issue further in Chapter 3.

2.4 Summary

In this chapter we introduced Dynamic Information Flow Tracking (DIFT) as a powerful security mechanism capable of preventing a wide range of attacks on unmodified binaries. Current DIFT systems are however, far from ideal. Software DIFT implementations are

either limited to a single language or rely on dynamic binary translation, and have unacceptable performance overheads. Hardware DIFT implementations are fast, but are very inflexible and have high design costs. An ideal DIFT solution to DIFT would combine the speed and applicability advantages of hardware DIFT with the flexibility offered by software solutions. This would allow for practically applying DIFT to help protect against a whole suite of software attacks. We provide a detailed discussion on the features of such a solution in the next chapter.

Chapter 3

Raksha - A Flexible Hardware DIFT Architecture

This chapter describes the architecture of Raksha, a flexible DIFT platform that combines the best of both hardware and software DIFT solutions. Unlike previous DIFT systems, Raksha leverages both hardware and software to implement the DIFT analysis. Hardware is responsible for maintaining the tag state, and performing low-level operations, such as tag propagations and checks. Software is responsible for configuring the security policies that are implemented by hardware, and for performing further analysis as required.

In Section 3.1, we provide a list of desirable features that a DIFT platform must possess in order to be flexible, extensible, and adaptable. We then introduce the Raksha DIFT architecture in Section 3.2, and discuss related work in Section 3.3 before concluding the chapter.

3.1 DIFT Design Requirements

Existing research has highlighted the potential of DIFT, and the trade-offs between software and hardware DIFT implementations. Software solutions (using binary translation) offer

unlimited flexibility in terms of the policies that can be specified. These solutions however have very high performance overheads, and do not work with multi-threaded programs. Hardware solutions while providing very low performance overheads and compatibility with multi-threaded workloads, suffer from a lack of flexibility.

An ideal solution for DIFT would integrate the performance advantages of hardware DIFT with the flexibility and extensibility of software DIFT mechanisms. We argue for hardware to provide a few basic mechanisms for DIFT upon which we can layer software to configure and extend our security mechanisms, thereby allowing the solution to adapt to the ever-evolving threat landscape. Specifically, this requires that hardware be responsible for managing, propagating and checking the tags required for DIFT, and software be responsible for managing multiple, concurrently active security policies.

3.1.1 Hardware management of Tags

Hardware support for maintaining and manipulating tags is necessary for low-overhead DIFT implementations. Hardware DIFT systems associate a tag with every register, cache line, and word of memory. Support for processing the tags can be implemented either by maintaining the tag state in the main processor [81], or by maintaining shadow state in a separate coprocessor [42], or even a separate core in a multi-core system [12]. Tags can be stored either by directly extending the words of memory in the system [14], or by storing tags on different memory pages [12].

It has been shown by prior research [81] that tags tend to exhibit significant spatial locality. Thus, it is possible to maintain tags at granularities coarser than individual words of memory. Using both per-page tags and per-word tags reduces the memory storage overhead significantly, as demonstrated by Suh *et al.* [81]. Consequently, the ideal DIFT solution must have support for a multi-granular tag storage mechanism.

The hardware is also responsible for propagation and checks of these tags on every

instruction. Propagation involves performing a logical function (AND, OR, XOR, etc.) on the tags of the source operands of the instruction, and storing the result in the destination operand's tag. Tag checks are performed on every instruction to ensure that tainted data is not being used in an unsafe manner.

Security policies for tag propagation and checks are controlled by software. The hardware is responsible for performing a "security decode" of every executing instruction to determine the relevant propagation and check policies that must be applied. In order for the DIFT mechanisms to be applicable to different types of programs and binaries, it is important to have the flexibility to apply different propagation and check policies to different instructions. For this purpose, many DIFT architectures associate tag policies at the granularity of *instruction classes* [14, 81]. Instruction classes correspond to types of instructions, such as arithmetic, logical, or branch operations. The solution must also have a mechanism for specifying custom security policies for some instructions, in order to account for various corner cases that arise in real world applications.

3.1.2 Multiple flexible security policies

Current DIFT systems hard-code a single security policy, which leaves them inflexible to counter evolving threats. This restricts their applicability, since high-level attacks such as SQL injections require tag management policies very different from those required by low-level exploits such as buffer overflows. SQL injection protection, for example, requires that the system prevent tainted SQL commands from being executed. While the hardware performs taint propagation, SQL string checks are extremely complex and dependent on SQL grammar, and should be performed in software. In contrast, some memory corruption protection techniques untaint tags on validation instructions, and raise security exceptions on access of tainted pointers. The policies required for these two protection techniques are very different.

In addition, real world software is ridden with corner cases [24, 41]. These corner cases often require custom tag propagation and check rules to be applied to certain instructions. To avoid false positives or false negatives due to such corner cases, it is essential that the system be able to flexibly specify security policies.

While existing DIFT systems provide protection against single attacks, it is now common for attacks to exploit multiple vulnerabilities [11, 83]. Multiplexing all security policies on top of a single tag bit would create false positives or false negatives due to the fact that certain policies are mutually incompatible with one another (e.g. SQL injection protection vs. pointer tainting). It is essential for DIFT systems to be able to support multiple, concurrently active security policies to offer robust protection. This in turn necessitates the use of a multi-bit tag per word of memory. Every "column" of bits would then correspond to a unique security policy (e.g. bit 0 of each tag could be used for buffer overflow protection, bit 1 for SQL injection protection, etc.). While the exact number of policies is still a research topic, our experiments indicate that four policies suffice. This is discussed further in Chapter 4.

3.1.3 Software analysis support

While hardware maintains the state necessary for taint, software is responsible for configuring the security policies that dictate the propagation and check modes adopted by the hardware. Tag manipulations require the addition of instructions to the ISA that can operate upon tags. One of the main advantages of DIFT is that it can be used to catch security exploits on unmodified binaries. Support for this requires that the binary be agnostic of tags. These special tag instructions should thus be accessible only from within a supervisor operating mode.

Existing DIFT systems cannot protect the operating system since the OS runs at the highest privilege level. This is a shortcoming of these systems, since a successful attack on

the OS can compromise the entire system. In order to be able to apply DIFT to the operating system, it is necessary for the software managing the analysis (or a *software security handler*) to be outside the operating system. The security handler is responsible for configuring the propagation and check policies for the executing program, and for initializing tag values.

The security handler is also responsible for handling security exceptions. Current DIFT systems trap into the operating system on a security exception and terminate the application. Moving forward, it is more realistic to imagine that the DIFT hardware will identify potential threats for which further software analysis is required. An example is SQL injection where hardware performs taint propagation, and software is responsible for determining if the query contains tainted commands. Trapping to the operating system frequently to perform such an analysis is extremely expensive. Since OS traps cost hundreds of CPU cycles, even infrequent security exceptions can have an impact on application performance.

Thus, the method of invoking the security handler should be via *user-level tag exceptions* rather than expensive OS traps. These exceptions transfer control to the security handler in the same address space, at the same privilege level. Privilege level transitions are expensive due to events such as TLB flushes, saving and restoring registers, etc. In contrast, user-level tag exceptions incur an overhead similar to function calls. Keeping the overhead of invoking the security handler low allows for a further analysis to be performed flexibly in software, and increases the extensibility of the DIFT system greatly.

3.2 The Raksha Architecture

This section introduces *Raksha*¹, a flexible hardware DIFT architecture for software security. Raksha introduces three novel features at the architecture level. First, it provides *a flexible and programmable mechanism for specifying security policies*. The flexibility is

¹Raksha means *protection* in Sanskrit.

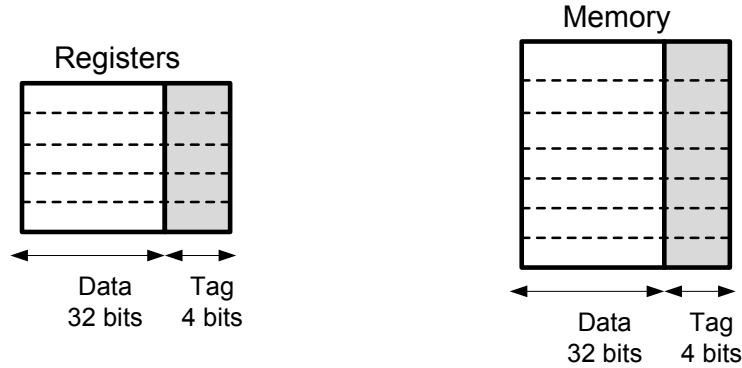


Figure 3.1: The tag abstraction exposed by the hardware to the software. At the ISA level, every register and memory location appears to be extended by four tag bits.

necessary to target high-level attacks such as cross-site scripting, and to avoid the trade-offs between false positives and false negatives due to the diversity of code patterns observed in commonly used software. Second, Raksha enables *security exceptions that run at the same privilege level and address space as the protected program*. This allows the integration of the hardware security mechanisms with additional software analyses, without incurring the performance overhead of switching to the operating system. It also makes DIFT applicable to the OS code. Finally, Raksha supports *multiple concurrently active security policies*. This allows for protection against a wide range of attacks.

3.2.1 Architecture overview

Raksha follows the general model of previous hardware DIFT systems [14, 20, 81]. All storage locations, including registers, caches, and main memory, are extended by tag bits. All ISA instructions are extended to propagate tags from input to output operands, and check tags in addition to their regular operation. Since tag operations happen transparently, Raksha can run all types of unmodified binaries without introducing runtime overheads.

Raksha, however, differs from previous work by supporting the features discussed earlier, in Section 3.1. First, it supports multiple active security policies. Specifically, each

word is associated with a 4-bit tag, where each bit supports an independent security policy with separate rules for propagation and checks. As indicated by the popularity of ECC codes, 4 extra bits per 32-bit word is an acceptable overhead for additional reliability. Figure 3.1 shows the logical view of the system at the ISA level, where every register and memory location appears to be extended with a 4-bit tag. Note that the actual implementation of the tag bits is dependent on the underlying hardware.

The tag storage overhead can be reduced significantly using multi-granular approaches that exploit the common case where all words in a cache line or in a memory page are associated with the same tag [81]. The choice of four tag bits per word was motivated by the number of security policies used to protect against a diverse set of attacks with the Raksha prototype (see Chapter 4). Even if future experiments show that a different number of active policies are needed, the basic mechanisms described in this section will apply.

The second difference is that Raksha’s security policies are highly flexible and software-programmable. Software uses a set of policy configuration registers to describe the propagation and check rules for each tag bit. The specification format allows fine-grained control over the rules. Specifically, software can independently control the tag rules for each class of instructions and configure how tags from multiple input operands are combined. Moreover, Raksha allows software to specify custom rules for a small number of individual instructions. This enables handling of corner cases within an instruction class. For example, `xor r1, r1, r1` is a commonly used idiom to reset registers, especially on *x86* machines. To avoid false positives while detecting memory corruption attacks, we must recognize this case and suppress tag propagation from the inputs to the output. Section 3.2.2 discusses how complex corner cases can be addressed using custom rules.

The third difference is that Raksha supports user-level handling of security exceptions. Hence, the exception overhead is similar to that of a function call rather than the overhead of a full OS trap. Two hardware mechanisms are necessary to support user-level exceptions handling. First, the processor has an additional *trusted mode* that is orthogonal to the

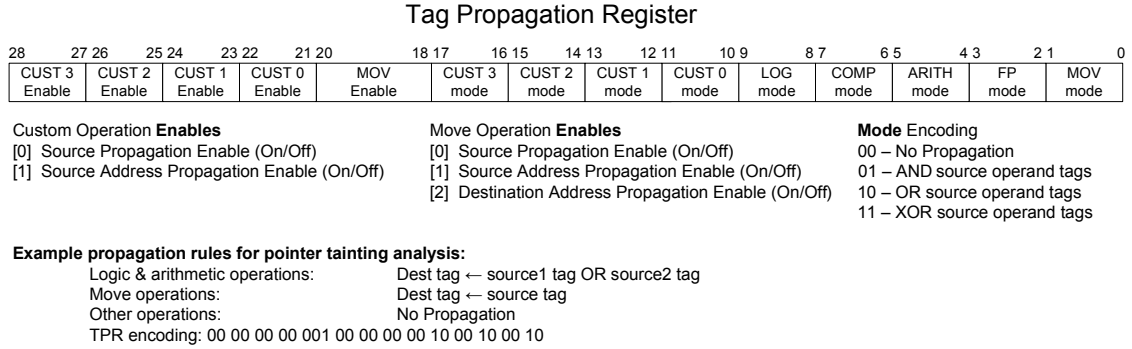


Figure 3.2: The format of the Tag Propagation Register. There are 4 TPRs, one per active security policy.

conventional user and kernel mode privilege levels. Software can directly access the tags or the policy configuration registers only when trusted mode is enabled. Tag propagation and checks are also disabled when in trusted mode. Second, a hardware register provides the address for a *predefined security handler* to be invoked on a tag exception. When a tag exception is raised, the processor automatically switches to the trusted mode but remains in the same user/kernel mode and the same address space. There is no need for an additional mechanism to protect the security handler’s code and data from malicious code. Raksha protects the handler using one of the four active security policies. Its code and data are tagged and a rule is specified that generates an exception if they are accessed outside of the trusted mode.

3.2.2 Tag propagation and checks

Hardware performs tag propagation and checks transparently for all instructions executed outside of trusted mode. The exact rules for tag propagation and checks are specified by a set of *tag propagation registers (TPR)* and *tag check registers (TCR)*. There is one TCR/TPR pair for each of the four security policies supported by hardware. Figures 3.2 and 3.3 present the formats of the two registers as well as an example configuration for a

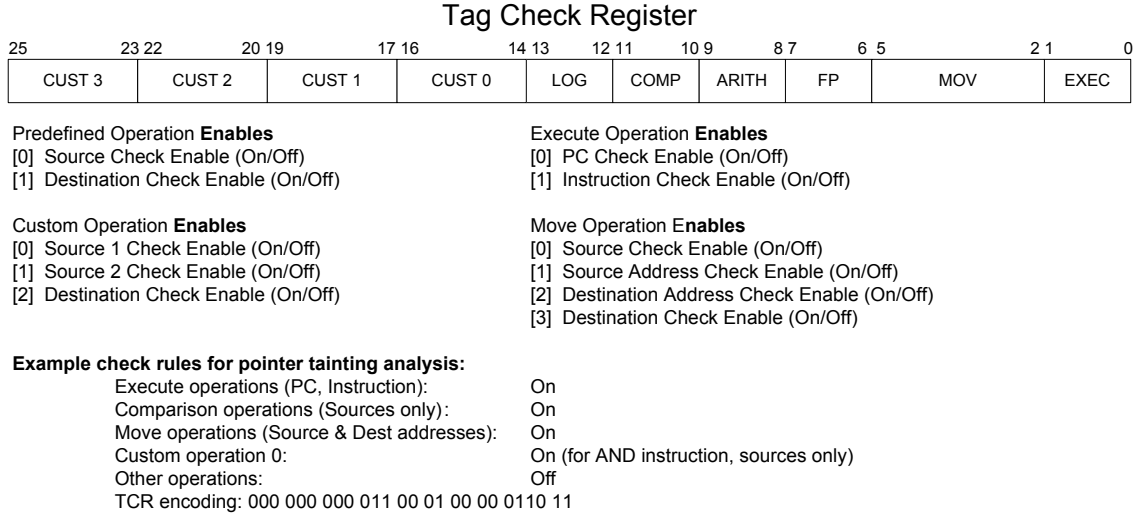


Figure 3.3: The format of the Tag Check Register. There are 4 TCRs, one per active security policy.

pointer tainting analysis.

To balance flexibility and compactness, TPRs and TCRs specify rules at the granularity of *primitive operation classes*. The classes are *floating point*, *(data) movement*, or *move*, *integer arithmetic*, *comparison*, and *logical*. The move class includes register-to-register moves, loads, stores, and jumps (move to program counter). To track information flow with high precision, we do not assign each ISA instruction to a single class. Instead, each instruction is decomposed into one or more primitive operations according to its semantics. For example, the `subcc` SPARC instruction is decomposed into two operations, a subtraction (arithmetic class) and a comparison that sets a condition code. As the instruction is executed, we apply the tag rules for both arithmetic and comparison operations. This approach is particularly important for ISAs that include CISC-style instructions, such as the `x86`. It also reflects a basic design principle of Raksha: information flow analysis tracks basic data operations, regardless of how these operations are packaged into ISA instructions. Previous DIFT systems define tag policies at the granularity of ISA instructions, which creates several opportunities for false positives and false negatives.

To handle corner cases such as register resetting with an `xor` instruction, TPRs and TCRs can also specify rules for up to four *custom operations*. As the instruction is decoded, we compare its opcode to four opcodes defined by software in the custom operation registers. If the opcode matches, we use the corresponding custom rules for propagation and checks instead of the generic rules for its primitive operation(s). An alternate way of specifying custom operation rules would be to maintain a software managed table, similar to FlexiTaint [88].

As shown in Figure 3.2, each TPR uses a series of two-bit fields to describe the propagation rule for each primitive class and custom operation (bits 0 to 17). Each field indicates if there is propagation from source to destination tags and if multiple source tags are combined using logical AND or OR. Bits 18 to 26 contain fields that provide source operand selection for tag propagation on move and custom operations. For move operations, we can propagate tags from the source, source address, and destination address operands. The load instruction `ld [r2], r1`, for example, considers register `r2` as the source address, and the memory location referenced by `r2` as the source.

As shown in Figure 3.3, each TCR uses a series of fields that specify which operands of a primitive class or custom operation should be checked for security purposes. If a check is enabled and the tag bit of the corresponding operand is set, a security exception is raised. For most operation classes, there are three operands to consider. For moves (loads and stores), we must also consider source and destination addresses. Each TCR includes an additional operation class named *execute*. This class specifies the rule for tag checks on instruction fetches. We can choose to raise a security exception if the fetched instruction is tagged or if the program counter is tagged. The former occurs when executing tainted code, while the latter can happen when a jump instruction propagates an input tag to the program counter.

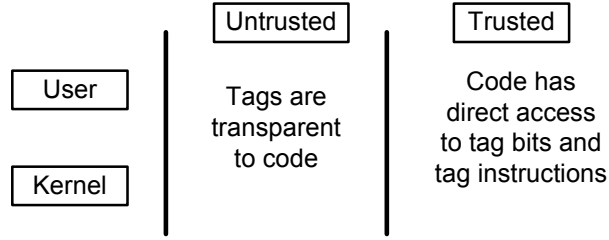


Figure 3.4: The logical distinction between trusted mode and traditional user/kernel privilege levels. Trusted mode is orthogonal to the user or kernel modes, allowing for security exceptions to be processed at the privilege level of the program.

3.2.3 User-level security exceptions

A security exception occurs when a TCR-controlled tag check fails for the current instruction. Security exceptions are *precise* in Raksha. When the exception occurs, the offending instruction is not committed. Instead, exception information is saved to a special set of registers for subsequent processing (PC, failing operand, which tag policies failed, etc.).

The distinguishing feature of security exceptions in Raksha is that they are processed at the user-level. When the exception occurs, the machine does not switch to the kernel mode and transfer control to the operating system. Instead, the machine maintains its current privilege level (user or kernel) and simply activates the trusted mode. Trusted mode, as indicated by Figure 3.4 is orthogonal to the conventional user/kernel privilege levels. Control is transferred to a predefined address for the security exception handler. In trusted mode, tag checks and propagation are disabled for all instructions. Moreover, software has access to the TCRs, TPRs and the registers that contain the information about the security exception. Finally, software running in the trusted mode can directly access the 4-bit tags associated with memory locations and regular registers². The hardware provides extra instructions to facilitate access to this additional state when in trusted mode.

The predefined address for the exception handler is available in a special register that

²Conventional code running outside the trusted mode can implicitly operate on tags but is not explicitly aware of their existence. Hence, it cannot directly read or write these tags.

can be updated only while in trusted mode. At the beginning of each program, the exception handler address is initialized before control is passed to the application. The application cannot change the exception handler address because it runs in untrusted mode.

The exception handler can include arbitrary software that processes the security exception. It may summarily terminate the compromised application or simply clean up and ignore the exception. It may also perform a complex analysis to determine whether the exception is a false positive, or try to address the security issue without terminating the code. The handler overhead depends on the complexity of the processing it performs. Since the handler executes in the same address space as the application, invoking the handler does not incur the cost of an OS trap (privilege level change, TLB flushing, etc.). The cost of invoking the security exception handler in Raksha is similar to that of a function call.

Since the exception handler and applications run at the same privilege level and in the same address space, there is a need for a mechanism that protects the handler code and data from a compromised application. Unlike the handler, user code runs only in untrusted mode and is forbidden from using the additional instructions that manipulate special registers or directly access the 4-bit tags in memory. Still, a malicious application could overwrite the code or data belonging to the handler. To prevent this, we use one of the four security policies to sandbox the handler's data and code. We set one of the four tag bits for every memory location used by the security handler for its code or data. The TCR is configured so that any instruction fetch or data load/store to locations with this tag bit set, will generate an exception. This sandboxing approach provides efficient protection without requiring different privilege levels. Hence, it can also be used to protect the trusted portion of the OS from the untrusted portion. We can also use the sandboxing mechanism (same policy) to implement the function call or system call interposition needed to detect some attacks.

3.2.4 Discussion

Raksha defines tag bits for every 32-bit word instead of every byte. We find the overhead of per-byte tags unnecessary. Considering the way compilers allocate variables, it is extremely unlikely that two variables with dramatically different security characteristics will be packed into a single word. The one exception we found to this rule so far is that some applications construct strings by concatenating untrusted and trusted information. Infrequently, this results in a word with both trusted and untrusted bytes.

To ensure that sub-word accesses do not introduce false negatives, we check the tag bit for the whole word even if a subset is read. For tag propagation on sub-word writes, we use a control register to allow software to select a method for merging the existing tag with the new one (*and*, *or*, *overwrite*, or *preserve*). As always, it is best for hardware to use a conservative policy and rely on software analysis within the exception handler to filter out the rare false positives due to sub-word accesses. We would use the same approach to implement Raksha on ISAs that support unaligned accesses that span multiple words.

Raksha can be combined with any base instruction set. For a given ISA, we decompose each instruction into its primitive operations and apply the proper check and propagate rules. This is a powerful mechanism that can cover both RISC and CISC architectures. For simple instructions, hardware can perform the decomposition during instruction decoding. For most complex CISC instructions, it is best to perform the decomposition using a micro-coding approach, as is often done for instruction decoding purposes. Raksha can handle instruction sets with condition code registers or other special registers by properly tagging these registers in the same manner as general purpose registers.

The operating system can interrupt and switch out an application that is currently in a security handler. As the OS saves/restores the process context, it also saves the trusted mode status. It must also save/store the special registers introduced by Raksha as if they were user-level registers. When the application resumes, its security handler will continue.

Like most other DIFT architectures, Raksha does not track implicit information flow since it would cause a large number of false positives. In addition, unlike information leaks, security exploits usually rely only on tainted code or data that is explicitly propagated through the system.

3.3 Related Work

Minos was one of the first systems to support DIFT in hardware [20]. Its design addresses many basic issues pertaining to integration of tags in modern processors and management of tags in the OS. Minos' security policy focuses on control data attacks that overwrite return addresses or function pointers. Minos cannot protect against non-control data attacks [15].

The architecture by Suh *et al.* [81] targets both control and non-control attacks by checking tags on both code and data pointer dereferences. Recognizing that real-world programs often validate their input through bounds checks, this design does not propagate the tag of an index if it is added to an untainted pointer with a pointer arithmetic instruction. This choice eliminates many false positive security exceptions but also allows for false negatives on common attacks such as return-into-libc [23]. A significant weakness is that most architectures do not have well-defined pointer arithmetic instructions. This restricts the applicability of the design, since RISC architectures such as the SPARC do not include such instructions. This design also introduced an efficient multi-granular mechanism for managing tag storage that reduces the memory overhead to less than 2%.

The architecture by Chen *et al.* [14] is similar to [81] but does not clear tags on pointer arithmetic, as there is no guarantee that the index has been validated. Instead, it clears the tag when tainted data is compared to untainted data, which is assumed to be a bounds check. This approach, however, results in both false positives and false negatives in commonly used code [23]. Moreover, this design does not check the tag bit while fetching

instructions, which allows for attacks when the code is writeable (JIT systems, virtual machines, etc.) [23].

DIFT can also be used to ensure the confidentiality of sensitive data [79, 87]. RIFLE [87] proposed a system solution that tracks the flow of sensitive data in order to prevent information leaks. Apart from explicit information flow, RIFLE must also track implicit flow, such as information gleaned from branch conditions. RIFLE uses software binary rewriting to turn all implicit flows into explicit flows that can be tracked using DIFT techniques. The overall system combines this software infrastructure with a hardware DIFT implementation to track the propagation of sensitive information and prevent leaks. Infoshield [79] uses a DIFT architecture to implement information usage safety. It assumes that the program was properly written and audited and uses runtime checks to ensure that sensitive information is used only in the way defined during program development.

3.4 Conclusions

In this chapter, we made the case for a flexible platform for DIFT, that combines the best of both the hardware and software worlds. We presented Raksha, a novel information flow architecture for software security. Hardware is used to maintain taint information, and perform propagation and checks of the tags used to store the taint. Software is responsible for configuring the policies used for propagation and checks, and also for performing further security analysis, if necessary, in the case of a security exception. Hardware maintains more than one tag bit per word of data, which allows the system to be able to run multiple concurrently active security policies. This flexibility, coupled with the ability to run multiple security policies is essential to be able to protect the system from the ever-evolving threat environment. Raksha also supports user-level exception handling that allows for fast security handlers that execute in the same address space as the application. Overall, Raksha supports the mechanisms that allow software to correct, complement, or extend the

hardware-based analysis.

In the next chapter, we provide more details on the implementation of the Raksha prototype. Since the tag management is done in hardware, Raksha's performance overheads are negligible. Support for multiple, simultaneously active security policies provides the ability to detect and prevent different classes of attacks. Finally, Raksha's user-level security exception mechanism ensures low-overhead exceptions, and allows us to extend our protection to the operating system.

Chapter 4

The Raksha Prototype System

This chapter describes the full-system prototype built to evaluate the Raksha architecture introduced in the previous chapter. We provide a thorough overview of the implementation issues surrounding the micro-architecture and design of Raksha, and also evaluate the security properties of the system. As this chapter illustrates, Raksha’s security features allow it to provide low-overhead protection against multiple classes of input validation attacks simultaneously.

The rest of the chapter is organized as follows. Section 4.1 provides details about the micro-architecture of the Raksha prototype. Section 4.2 evaluates Raksha’s security features, while Section 4.3 measures the performance overhead of the prototype. Section 4.4 concludes the chapter.

4.1 The Raksha Prototype System

To evaluate Raksha, we developed a prototype system based on the SPARC architecture. Previous DIFT systems used a functional model like Bochs to evaluate security issues and a separate performance model like SimpleScalar to evaluate overhead issues with user-only code [14, 20, 81]. Instead, we use a single prototype for both functional and performance

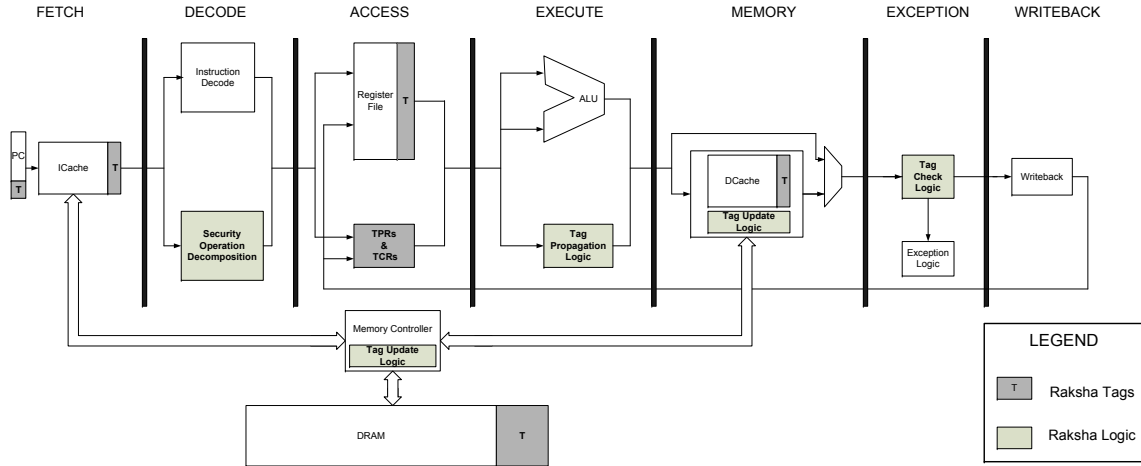


Figure 4.1: The Raksha version of the pipeline for the Leon SPARC V8 processor.

analysis. Hence, we can obtain accurate performance measurements for *any* real-world application we choose to protect. Moreover, we can use a single platform to evaluate performance and security issues related to the operating system and the interaction between multiple processes (e.g., a web server and a database).

The Raksha prototype is based on the Leon SPARC V8 processor, a 32-bit open-source synthesizable core developed by Gaisler Research [49]. We modified Leon to include the security features of Raksha and mapped the design onto an FPGA board. The resulting system is a full-featured SPARC Linux workstation.

4.1.1 Hardware implementation

Figure 4.1 shows a simplified diagram of the Raksha hardware, focusing on the processor pipeline. Leon uses a single-issue, 7-stage pipeline. Such a design is comparable to some of the simple cores currently being advocated for chip multiprocessors, such as Sun’s Niagara, and Intel’s Atom. We modified its RTL code to add 4-bit tags to all user-visible registers, and cache and memory locations; introduced the configuration and exception registers defined by Raksha; and added the instructions that manipulate special registers

Register Name	Number	Function
Tag Status Register	1	Maintain the trusted mode, individual policy enables, and merge modes
Tag Propagation Register	4	Maintain propagation policies and modes for instruction classes
Tag Check Register	4	Maintain check policies for instruction classes
Custom Operation Register	2	Maintain custom propagation and check policies for two instructions (each)
Reference Monitor Address	1	Stores the starting address of the security handler's code
Exception PC	1	Stores PC of instruction raising tag exception
Exception nPC	1	Stores nPC of instruction raising tag exception
Exception Memory Address	1	Stores the (data) memory address associated with trapping instruction
Exception Type	1	Stores information about the failed tag check (operand, operation type)

Table 4.1: The new pipeline registers added to the Leon pipeline by the Raksha architecture.

or provide direct access to tags in the trusted mode. Overall, we added 16 registers and 9 instructions to the SPARC V8 ISA. These are documented in Tables 4.1 and 4.2 respectively. These registers and instructions are only visible to code running in trusted mode, and are transparent to code running outside the trusted mode. We also added support for the low-overhead security exceptions and extended all buses to accommodate tag transfers in parallel with the associated data.

The processor operates on tags as instructions flow through its pipeline, in accordance with the policy configuration registers (TCRs and TPRs). The Fetch stage checks the program counter tag and the tag of the instruction fetched from the I-cache. The Decode stage decomposes each instruction into its primitive operations and checks if its opcode matches any of the custom operations. The Access stage reads the tags for the source operands from the register file, including the destination operand. It also reads the TCRs and TPRs. By the end of this stage, we know the exact tag propagation and check rules to apply for this instruction. Note that the security rules applied for each of the four tag bits are independent

Instruction	Example	Meaning
Read Register Tag	<code>rdt_reg r1, r2</code>	$r2 = T[r1]$
Write Register Tag	<code>wrt_reg r1, r2</code>	$T[r1] = r2$
Read Memory Tag	<code>rdt_mem r1, r2</code>	$r2 = T[M[r1]]$
Write Memory Tag	<code>wrt_mem r1, r2</code>	$T[M[r1]] = r2$
Read Memory Tag and Data	<code>rdtd_mem r1, r2</code>	$T[r2] = T[M[r1]]$ $r2 = M[r1]$
Write Memory Tag and Data	<code>wrtd_mem r1, r2</code>	$T[M[r1]] = T[r2]$ $M[r1] = r2$
Read Config Register	<code>rdtr r1, exception_pc</code>	$r1 = \text{exception_pc}$
Write Config Register	<code>wrtr r1, tpr</code>	$\text{tpr} = r1$
Return from Tag Exception	<code>tret</code>	$\text{pc} = \text{exception_pc}$

Table 4.2: The new instructions added to the SPARC V8 ISA by the Raksha architecture.

of one another. The Execute and Memory stages propagate source tags to the destination tag in accordance with the active policies. The Exception stage performs any necessary tag checks and raises a precise security exception if needed. All state updates (registers, configuration registers, etc.) are performed in the Writeback stage. Pipeline forwarding for the tag bits is implemented similar to, and in parallel with, forwarding for regular data values.

Our current implementation of the memory system simply extends all cache lines and buses by 4 tag bits per 32-bit word. We also reserved a portion of main memory for tag storage and modified the memory controller to properly access both data and tags on cached and uncached requests. This approach introduces a 12.5% space overhead in the memory system for tag storage. On a board with support for ECC DRAM, the 4 bits per 32-bit word available to the ECC code could be used to store the Raksha tags. Since tags exhibit significant spatial locality, the multi-granular tag storage approach proposed by Suh *et al.* [81] would help reduce the storage overhead for tags to less than 2% [81]. In this scheme, fine-grained tags are allocated on demand for cache lines and memory pages that actually have tagged data. The system would then maintain tags at the page granularity for memory pages that have the same tags on all data words. These tags can be cached similar to data,

Parameter	Specification
Pipeline depth	7 stages
Register windows	8
Instruction cache	8 KB, 2-way set-associative
Data cache	32 KB, 2-way set-associative
Instruction TLB	8 entries, fully-associative
Data TLB	8 entries, fully-associative
Memory bus width	64 bits
Prototype Board	GR-CPCI-XC2V board
FPGA device	XC2VP6000
Memory	512MB SDRAM DIMM
I/O	100Mb Ethernet MAC
Clock frequency	20 MHz
Block RAM utilization	22% (32 out of 144)
4-input LUT utilization	42% (28,897 out of 67,584)
Total gate count	2,405,334
Gate count increase over base Leon (with FPU)	4.85%

Table 4.3: The architectural and design parameters for the Raksha prototype.

for performance reasons, either by modifying the TLB structure to maintain page-level tags, or by maintaining a separate cache for page-level tags [96].

We synthesized Raksha on the Pender GR-CPCI-XC2V Compact PCI board which contains a Xilinx XC2VP6000 FPGA. Table 4.3 summarizes the basic board and design statistics, including the utilization of the FPGA resources. Note that gate count overhead in Table 4.3 is lower than the one in the original Raksha paper, which reports a 7.17% increase in gate count over a base Leon system with no FPU [24]. When calculating our results for an FPU-enabled design, we assume the FPU control path would require modifications of similar complexity (which we approximate as 7.17% per previous results), and that the FPU datapath would require no modifications. Most modern superscalar processors are more complex than the Leon, and contain lots of hardware units such as branch predictors, trace caches, and prefetchers etc. which do not require to be modified to accommodate tags. Thus, the overhead of implementing Raksha’s logic in a more complex superscalar

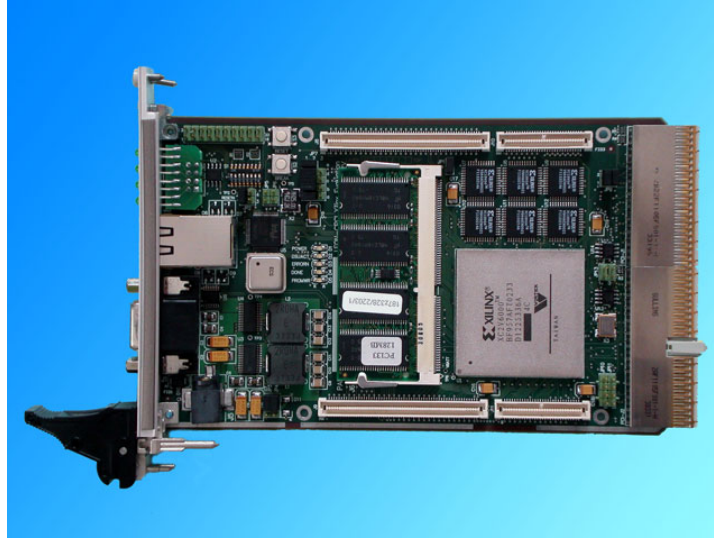


Figure 4.2: The GR-CPCI-XC2V board used for the prototype Raksha system.

design would be lower.

Since Leon uses a write-through, no-write-allocate data cache, we had to modify its design to perform a read-modify-write access on the tag bits in the case of a write miss. This change and its small impact on application performance would not have been necessary had we started with a write-back cache. There was no other impact on the processor performance since tags are processed in parallel and independently from the data in all pipeline stages. Having a write-back cache would have reduced our overhead further. We believe the same would be true for more aggressive processor designs as tags are processed in parallel and are independent from data in all pipeline stages.

Table 4.3 shows that the Raksha prototype has 4.8% more gates than the original Leon design. This roughly correlates with the overheads that a realistic Raksha chip would have. However, the gate count numbers quoted in Table 4.3 are much more than what an actual Raksha ASIC design would contain. This is because the area of an FPGA design containing both memory and logic is roughly $31 \times$ to $40 \times$ that of an equivalent ASIC design [47].

In most processor designs, the majority of the chip's area and power are consumed

Storage Element	Area Overhead (% increase)	Standby Leakage Power Overhead (% increase)	Read Dynamic Energy Overhead (% increase)
Instruction Cache	0.243mm ² (17.6%)	2.8e-08 W (10.14%)	0.172 nJ (16.08%)
Data Cache	0.329mm ² (15.05%)	9.4e-08 W (10.54%)	0.261 nJ (13.91%)
Register File	0.031mm ² (10.83%)	1.0e-08 W (4.54%)	0.003 nJ (12.17%)

Table 4.4: The area and power overhead values for the storage elements in the Raksha prototype. Percentage overheads are shown relative to the corresponding data storage structures in the unmodified Leon design.

by the storage elements such as the caches and register files. Thus, studying the area overheads and power consumption of these storage elements provides a good first-order approximation of the overheads of the entire design. Consequently, we evaluate the area and power overheads of Raksha's storage elements to obtain an estimate of the overheads of adding DIFT to a processor. We used CACTI 5.2 [85] in order to get area and power consumption data for a Raksha design fabricated at a 65nm process technology. Table 4.4 summarizes the area and power overheads of adding four bits per 32-bit word to the caches and register files, in the Raksha prototype. As is evident, the area requirements for maintaining the security bits is very low. For comparison, Leon's 32KB data cache occupies 2.185mm² at the 65nm process technology [85].

Security features are trustworthy only if they have been thoroughly validated. Similar to other ISA extensions, the Raksha security mechanisms define a relatively narrow hardware interface that can be validated using a collection of directed and randomly generated test cases that stress individual instructions and combinations of instructions, modes, and system states. We built a random test generator that creates arbitrary SPARC programs with randomly generated tag policies. Periodically, test programs enable the trusted mode and verify that any registers or memory locations modified since the last checkpoint have the

expected tag and data values. The expected values are generated by a simple functional-only model of Raksha for SPARC. If the validation fails, the test case halts with an error. The test case generator supports almost all SPARC V8 instructions. We ran tens of thousands of test cases, both on the simulated RTL using a 30-processor cluster, and on the actual FPGA prototype.

4.1.2 Software implementation

The Raksha prototype provides a full-fledged custom Linux distribution derived from Cross-Compiled Linux From Scratch [21]. The distribution is based on the Linux kernel 2.6.11, GCC 4.0.2 and GNU C Library 2.3.6. It includes 120 software packages. Our distribution can bootstrap itself from source code and run unmodified enterprise applications such as Apache, PostgreSQL, and OpenSSH.

We modified the Linux kernel to provide support for Raksha's security features. The additional registers are saved and restored properly on context switches, system calls, and interrupts. Register tags must also be saved on signal delivery and SPARC register window overflows/underflows. Tags are properly copied when inter-process communication occurs, such as through pipes or when passing program arguments or environment variables to `execve`.

Security handlers are implemented as shared libraries preloaded by the dynamic linker. The OS ensures that all memory tags are initialized to zero when pages are allocated and that all processes start in trusted mode with register tags cleared. The security handler initializes the policy configuration registers and any necessary tags before disabling the trusted mode and transferring control to the application. For best performance, the basic code for invoking and returning from a security handler have been written directly in SPARC assembly. The code for any additional software analyses invoked by the security handler can be written in any programming language. The security handlers can support checks even

on the operating system.

Most security analyses require that tags be properly initialized or set when receiving data from input channels. We have implemented tag initialization within the security handler using the system call interposition tag policy discussed in Section 4.2. For example, a SQL injection analysis may wish to tag all data from the network. The reference handler would use system call interposition on the `recv`, `recvfrom`, and `read` system calls to intercept these system calls, and taint all data returned by them.

4.2 Security Evaluation

To evaluate the capabilities of Raksha’s security features, we attempted a wide range of attacks on unmodified SPARC binaries for real-world applications. Raksha successfully detected both high-level attacks and memory corruption exploits on these programs. This section briefly highlights our security experiments and discusses the policies used.

4.2.1 Security policies

This section describes the DIFT policies used for the security experiments. We can have all the policies in Table 4.5 concurrently active using the 4 tag bits available in Raksha: one for identifying valid pointers (*pointer bit*), one for tainting (*taint bit*), one for bounds-check based tainting, and one for the protection of portions of memory, such as the software handler, using a sandboxing policy [22, 25]. This combination allows for comprehensive protection against low-level and high-level vulnerabilities.

Memory Corruption Exploits

Tables 4.6 and 4.7 present the DIFT rules for tag propagation and checks for buffer overflow prevention. The rules are intended to be as conservative as possible while still avoiding

Policy	Functionality	Pointer bit	Taint bit	Bounds-check bit	Sandbox bit
Buffer Overflows	Identify pointers and track data taint. Check for illegal tainted pointer use.	Y	Y		
Offset-based control pointer attacks	Track data taint. Bounds check to validate.			Y	
Format Strings pointer attacks	Check for tainted arguments to print commands.		Y		Y
SQL injections and Cross-site scripting (XSS)	Check for tainted SQL/XSS commands.		Y		Y
Red zone bounds checking	Protect heap data.				Y
Sandboxing policy	Protect the security handler.				Y

Table 4.5: Summary of the security policies implemented by the Raksha prototype. The four tag bits are sufficient to implement six concurrently active policies to protect against both low-level memory corruption and high-level semantic attacks.

false positives. Since our policy is based on pointer injection, we use two tag bits per word of memory. A *taint* (T) bit is set for untrusted data, and propagates on all arithmetic, logical, and data movement instructions. Any instruction with a tainted source operand propagates taint to the destination operand (register or memory). A *pointer* (P) bit is initialized for legitimate application pointers and propagates during valid pointer operations such as pointer arithmetic. A security exception is thrown if a tainted instruction is fetched, or the address used in a load, store, or jump instruction is tainted and not a valid pointer. In other words, we allow a program to combine a valid pointer with an untrusted index, but not to use an untrusted pointer directly. For a more in-depth discussion of identifying the valid pointers in the program, we refer the reader to prior work [22, 25]. As Section 4.2.2 will show, we were able to catch memory corruption exploits in both user and kernelspace.

Operation	Example	Taint Propagation	Pointer Propagation
Load	ld r2 = M[r1+imm]	$T[r2] = T[M[r1+imm]]$	$P[r2] = P[M[r1+imm]]$
Store	st M[r1+imm] = r2	$T[M[r1+imm]] = T[r2]$	$P[M[r1+imm]] = P[r2]$
Add/Sub/Or	add r3 = r1 + r2	$T[r3] = T[r1] \vee T[r2]$	$P[r3] = P[r1] \vee P[r2]$
And	and r3 = r1 $\wedge$ r2	$T[r3] = T[r1] \vee T[r2]$	$P[r3] = P[r1] \oplus P[r2]$
Other ALU	xor r3 = r1 $\oplus$ r2	$T[r3] = T[r2] \vee T[r1]$	$P[r3] = 0$
Sethi	sethi r1 = imm	$T[r1] = 0$	$P[r1] = P[insn]$
Jump	jmp r1+imm, r2	$T[r2] = 0$	$P[r2] = 1$

Table 4.6: The DIFT propagation rules for the taint and pointer bits. r_y stands for register y . $T[x]$ and $P[x]$ refer to the taint (T) or pointer (P) tag bits respectively for memory location, register, or instruction x .

Operation	Example	Security Check
Load	ld r1+imm, r2	$T[r1] \wedge \neg P[r1]$
Store	st r2, r1+imm	$T[r1] \wedge \neg P[r1]$
Jump	jmp r1+imm, r2	$T[r1] \wedge \neg P[r1]$
Instruction fetch	-	$T[insn]$

Table 4.7: The DIFT check rules for BOF detection. A security exception is raised if the condition in the rightmost column is true.

High-level Web Vulnerabilities

The tainting policy is also used to protect against high-level semantic attacks. It tracks untrusted data via tag propagation and allows software to check tainted arguments before sensitive function and system calls. For protection from Web vulnerabilities such as cross-site scripting, string tainting is applied both to Apache itself and to any associated modules such as PHP.

To protect the security handler from malicious attacks, we use a fault-isolation tag policy that implements sandboxing. The handler code and data are tagged, and a rule is specified that generates an exception if they are accessed outside of trusted mode. This policy ensures handler integrity even during a memory corruption attack on the application.

We tested for false positives by running a large number of real-world workloads such

Program	Lang.	Attack	Analysis	Detected Vulnerability
gzip	C	Directory traversal	String tainting + System call interposition	Open file with tainted absolute path
tar	C	Directory traversal	String tainting + System call interposition	Open file with tainted absolute path
Wabbit	PHP	Directory traversal	String tainting + System call interposition	Open file with tainted pathname outside web root directory
Scry	PHP	Cross-site scripting	String tainting + System call interposition	Tainted HTML output includes <i>< script ></i>
PhpSysInfo	PHP	Cross-site scripting	String tainting + System call interposition	Tainted HTML output includes <i>< script ></i>
htdig	C++	Cross-site scripting	String tainting + System call interposition	Tainted HTML output includes <i>< script ></i>
OpenSSH	C	Command injection	String tainting + System call interposition	execve tainted filename
ProFTPD	C	SQL injection	String tainting + Function call interposition	Unescaped tainted SQL query

Table 4.8: The high-level semantic attacks caught by the Raksha prototype.

as compiling applications like Apache, booting the Gentoo Linux distribution, and running Unix binaries such as perl, GCC, make, sed, awk, and ntp. Despite our conservative tainting policy [25], no false positives were encountered.

4.2.2 Security experiments

Tables 4.8 and 4.9 summarize the security experiments we performed. They include attacks in both user and kernelspace on basic utilities, network utilities, servers, Web applications,

Program	Lang.	Attack	Analysis	Detected Vulnerability
polymorph	C	Stack overflow	Pointer tainting	Tainted frame pointer dereference
atphttpd	C	Stack overflow	Pointer tainting	Tainted frame pointer dereference
sendmail	C	BSS overflow	Pointer tainting	Application data pointer overwrite
traceroute	C	Double free	Pointer tainting	Heap metadata pointer overwrite
nullhttpd	C	Double free	Pointer tainting	Heap metadata pointer overwrite
quotactl syscall	C	User/kernel pointer	Pointer tainting	Tainted pointer to kernelspace
i20 driver	C	User/kernel pointer	Pointer tainting	Tainted pointer to kernelspace
sendmsg syscall	C	Heap overflow	Pointer tainting	Kernelspace heap pointer overwrite
moxa driver	C	BSS overflow	Pointer tainting	Kernelspace BSS pointer overwrite
cm4040 driver	C	Heap overflow	Pointer tainting	Kernelspace heap pointer overwrite
SUS	C	Format string bug	String tainting + Function call interposition	Tainted format string specifier in syslog
WU-FTPD	C	Format string bug	String tainting + Function call interposition	Tainted format string specifier in <code>vfprintf</code>

Table 4.9: The low-level memory corruption exploits caught by the Raksha prototype.

drivers, system calls and search engine software. For each experiment, we list the programming language of the application, the type of attack, the DIFT analyses used for the detection, and the actual vulnerability detected by Raksha [22, 24, 25].

Unlike previous DIFT architectures, Raksha does not have a fixed security policy. The four supported policies can be set to detect a wide range of attacks. Hence, Raksha can be programmed to detect high-level attacks like SQL injection, command injection, cross-site scripting, and directory traversals, as well as conventional memory corruption and format string attacks. The correct mix of policies can be determined on a per-application basis by the system operator. For example, a Web server might select SQL injection and cross-site scripting protection, while an SSH server would probably select pointer tainting and format string protection.

To the best of our knowledge, Raksha is the *first* DIFT architecture to demonstrate detection of high-level attacks on unmodified application binaries. This is a significant result because high-level attacks now account for the majority of software exploits [83]. All prior work on high-level attack detection required access to the application source code or Java bytecode [52, 67, 71, 93]. High-level attacks are particularly challenging because they are language and OS independent. Enforcing type safety cannot protect against these semantic attacks, which makes Java and PHP code as vulnerable as C and C++.

An additional observation from Tables 4.8 and 4.9 is that by tracking information flow at the level of primitive operations, Raksha provides attack detection in a language-independent manner. The same policies can be used regardless of the application's source language. For example, `htdig` (C++) and `PhpSysInfo` (PHP) use the same cross-site scripting policy, even though one is written in a low-level, compiled language and the other in a high-level, interpreted language. Raksha can also apply its security policies across multiple collaborating programs that have been written in different programming languages.

4.3 Performance Evaluation

Hardware DIFT systems, including Raksha, perform fine-grained tag propagation and checks transparently as the application executes. Hence, they incur minimal runtime overhead compared to program execution with security checks disabled [14, 20, 81]. The small overhead is due to tag management during program initialization, paging, and I/O events. Nevertheless, such events are rare and involve significantly higher sources of overhead compared to tag manipulation. For reference, consider Table 4.10, which shows the overall runtime overhead introduced by our security scheme on a suite of SPEC2000 benchmarks. The runtime overhead is negligible ($<0.1\%$) and is due to the initialization of the pointer bit (assuming no caching of the pointer bit).

We focus our performance evaluation on a feature unique to Raksha - the low-overhead

Program	Normalized overhead
164.zip	1.002x
175.vpr	1.001x
176.gcc	1.000x
181.mcf	1.000x
186.crafty	1.000x
197.parser	1.000x
254.gap	1.000x
255.vortex	1.000x
256.bzip2	1.000x
300.twolf	1.000x

Table 4.10: Normalized execution time after the introduction of the pointer-based buffer overflow protection policy. The execution time without the security policy is 1.0. Execution time higher than 1.0 represents performance degradation.

handlers for security exceptions. Raksha supports user-level exception handlers as a mechanism to extend and correct the hardware security analysis. This exception overhead is not particularly important in protecting against semantic vulnerabilities. High-level attacks require software intervention only at the boundaries of certain system calls, which are infrequent and expensive events that transition to the operating system by default. The overhead of the security exception is negligible in comparison. On the other hand, fast software handlers can sometimes be useful in the protection against memory corruption attacks, by helping identify potential bounds-check operations, or performing custom propagation operations to reduce hardware costs and manage the tradeoff between false positives and false negatives.

To better understand the tradeoffs between the invocation frequency of software handlers and runtime overhead, we developed a simple microbenchmark. The microbenchmark invokes a security handler every 100 to 100,000 instructions. The duration of the handler is also controlled to be 0, 200, 500, or 1000 arithmetic instructions. This is in addition to

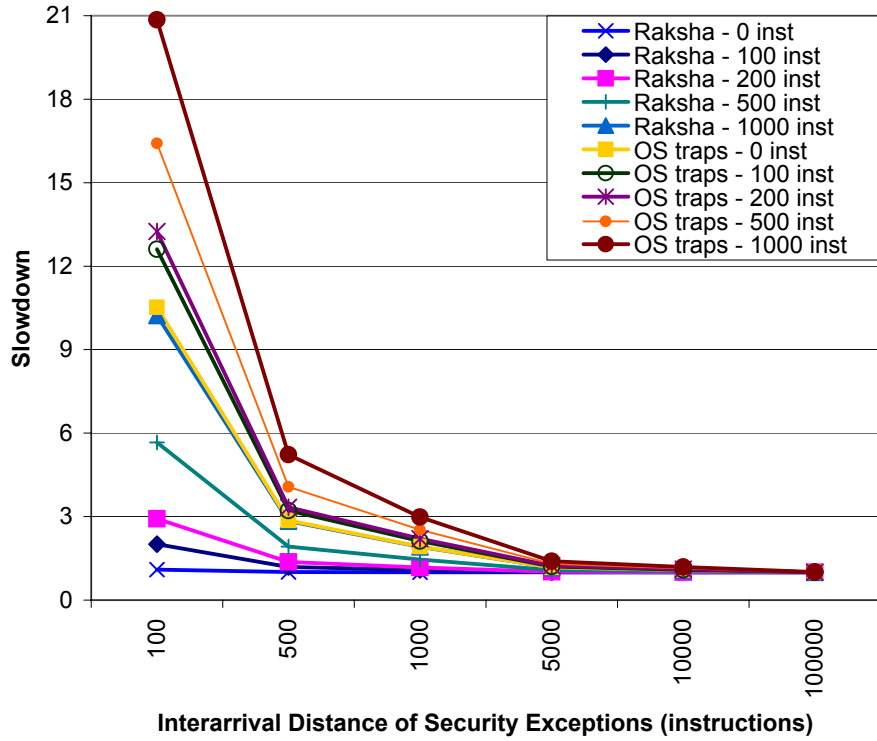


Figure 4.3: The performance degradation for a microbenchmark that invokes a security handler of controlled length every certain number of instructions. All numbers are normalized to a baseline case which has no tag operations.

the instructions necessary to invoke and terminate the handler. Figure 4.3 shows that if security exceptions are invoked less frequently than every 5,000 instructions, both user-level and OS-level exception handling are acceptable as their cost is easily amortized. On the other hand, if software is involved as often as every 1,000 or 100 instructions, user-level handlers are critical in maintaining acceptable performance levels. Low-overhead security exceptions allow software to intervene more frequently or perform more work per invocation. For reference, the software monitors we typically used required approximately 100 instructions per invocation.

For the microbenchmark, we built a customized version of Raksha which throws a full operating system trap for every tag exception, and modified the Linux kernel to handle this new trap. Other than minor changes required to run in an operating system, the tag handler

code is the same for Raksha's low-cost exception mechanism and full operating system trap.

4.4 Summary

We implemented a fully-featured Linux workstation as a prototype for Raksha using a synthesizable SPARC core and an FPGA board. Running real-world software on the prototype, we demonstrated that Raksha is the first DIFT architecture to detect high-level vulnerabilities such as directory traversals, command injection, SQL injection, and cross-site scripting, while providing protection against conventional memory corruption attacks in both userspace and in the kernel, without false positives. We also demonstrated that Raksha's performance overheads are negligible, and that the area overhead of the hardware structures introduced by Raksha is low. Overall, Raksha provides a security framework that is flexible, robust, end-to-end, practical, and fast.

Like previous hardware DIFT architectures, Raksha also requires invasive modifications to the core's pipeline to accommodate tags, which increases the design and validation costs for processor vendors. In the next chapter, we discuss how DIFT processing can be decoupled from the main core and thus be made practical to processor designers.

Chapter 5

A Decoupled Coprocessor for DIFT

DIFT architectures such as Raksha that provide DIFT support within the main pipeline require *significant modifications* to the processor design. These changes make it difficult for processor vendors to adopt hardware support for DIFT. This chapter observes that it is possible to decouple the hardware logic for DIFT from the main processor, to a dedicated coprocessor. Synchronizing the main core and the coprocessor on system calls is sufficient to maintain the same security model as Raksha. A full-system FPGA prototype of a DIFT coprocessor proves that this scheme has minimal performance and area overheads.

This chapter is organized as follows. Section 5.1 surveys the different methods of implementing hardware DIFT. Section 5.2 discusses the security model, and the design of the DIFT coprocessor. Section 5.3 describes the full-system prototype, while Section 5.4 provides an evaluation of the security features, performance and cost of the system. Section 5.5 concludes the chapter.

5.1 Design Alternatives for Hardware DIFT

Figure 5.1 presents the three design alternatives for hardware support for DIFT: (a) the *integrated*, in-core design; (b) the multi-core based, *offloading* design; and (c) an off-core,

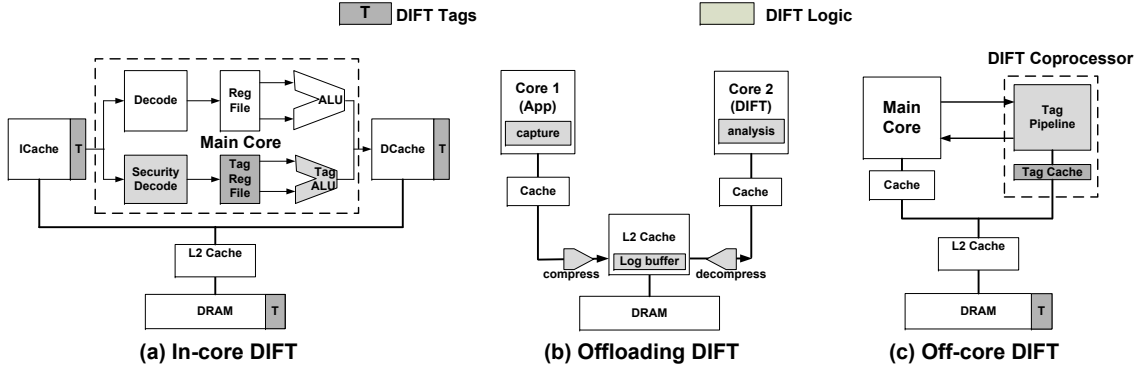


Figure 5.1: The three design alternatives for DIFT architectures.

coprocessor approach.

Most of the proposed DIFT systems follow the integrated approach, which performs tag propagation and checks in the processor pipeline in parallel with regular instruction execution [14, 20, 24, 81]. This approach does not require an additional core for DIFT functionality and introduces no overhead for inter-core coordination. Overall, its performance impact in terms of clock cycles over native execution is minimal. On the other hand, the integrated approach requires significant modifications to the processor core. All pipeline stages must be modified to buffer the tags associated with pending instructions. The register file and first-level caches must be extended to store the tags for data and instructions. Alternatively, a specialized register file or cache that only stores tags and is accessed in parallel with the regular blocks must be introduced in the processor core. Overall, the changes to the processor core are significant and can have a negative impact on design and verification time. Depending on the constraints, the introduction of DIFT may also affect the clock frequency. The high upfront cost and inability to amortize the design complexity over multiple processor designs can deter hardware vendors from adopting this approach. Feedback from processor vendors has impressed upon us that the extra effort required to change the design and layout of a complex superscalar processor to accommodate DIFT, and re-validate are enough to prevent design teams from adopting DIFT [80].

FlexiTaint [88] uses the approach introduced by the DIVA architecture [3] to push changes for DIFT to the back end of the pipeline. It adds two pipeline stages prior to the final commit stage, which access a separate register file and a separate cache for tags. FlexiTaint simplifies DIFT hardware by requiring few changes to the design of the out-of-order portion of the processor. Nevertheless, the pipeline structure and the processor layout must be modified. To avoid any additional stalls due to accesses to the DIFT tags, FlexiTaint modifies the core to generate prefetch requests for tags early in the pipeline. While it separates regular computation from DIFT processing, it does not fully decouple them. FlexiTaint synchronizes the two on every instruction, as the DIFT operations for each instruction must complete before the instruction commits. Due to the fine-grained synchronization, FlexiTaint requires an OOO core to hide the latency of two extra pipeline stages.

An alternative approach is to offload DIFT functionality to another core in a multi-core chip [12, 13, 62]. The application runs on one core, while a second general-purpose core runs the DIFT analysis on the application trace. The advantage of the offloading approach is that hardware does not need explicit knowledge of DIFT tags or policies. It can also support other types of analyses such as memory profiling and locksets [13]. The core that runs the regular application and the core that runs the DIFT analysis synchronize only on system calls. Nevertheless, the cores must be modified to implement this scheme. The application core is modified to create and compress a trace of the executed instructions. The core must select the events that trigger tracing, pack the proper information (PC, register operands, and memory operands), and compress in hardware. The trace is exchanged using the shared caches (L2 or L3). The security core must decompress the trace using hardware and expose it to software.

The most significant drawback of the multi-core approach is that it requires a full general-purpose core for DIFT analysis. Hence, it halves the number of available cores

for other programs and doubles the energy consumption due to the application under analysis. The cost of the modifications to each core is also non-trivial, especially for multi-core chips with simple cores. For instance, the hardware for trace (de)compression uses a 32-Kbyte table for value prediction. The analysis core requires an additional 16-Kbyte SRAM for static information [12]. These systems also require other modifications to the cores, such as additional TLB-like structures to maintain metadata addresses, for efficiency [13]. While the multi-core DIFT approach can also support memory profiling and lockset analyses, the hardware DIFT architectures [24, 25, 88] are capable of performing all the security analyses supported by offloading systems, at a lower cost.

The approach we propose is an intermediate between FlexiTaint and the multi-core one. Given the simplicity of DIFT propagation and checks (logical operations on short tags), using a separate general-purpose core is overkill. Instead, we propose using a small attached coprocessor that implements DIFT functionality for the main processor core and synchronizes with it only on system calls. The coprocessor includes all the hardware necessary for storing DIFT state (register tags and tag caches), and performing tag propagation and checks.

Compared to the multi-core DIFT approach, the coprocessor eliminates the need for a second core for DIFT and does not require changes to the processor and cache hierarchy for trace exchange. As we show in Section 5.3.2, the coprocessor is actually smaller than the hardware necessary to compress and decompress the log in the offloading approach. Compared to FlexiTaint, the coprocessor eliminates the need for any changes to the design, pipeline, or layout of the main core. Hence, there is no impact on design, verification or clock frequency of the main core. Coarse-grained synchronization enables full decoupling between the main core and the coprocessor. As we show in the following sections, the coprocessor approach provides the same security guarantees and the same performance as FlexiTaint and other integrated DIFT architectures. Unlike FlexiTaint, the coprocessor can also be used with in-order cores, such as Atom and Larrabee in Intel chips, or Niagara in

Sun chips.

5.2 Design of the DIFT Coprocessor

The goal of our design is to minimize the cost and complexity of DIFT support by migrating its functionality to a dedicated coprocessor. The main core operates only on data, and has no idea that tags exist. The main core passes information about control flow to the coprocessor. The coprocessor in turn, performs all tag operations and maintains all tag state (configuration registers, register and memory tags). This section describes the design of the DIFT coprocessor and its interface with the main core.

5.2.1 Security model

The full decoupling of DIFT functionality from the processor is possible by synchronizing the regular computation and DIFT operations at the granularity of system calls [62, 74, 75]. Synchronization at the system call granularity operates as follows. The main core can commit all instructions other than system calls and traps before it passes them to the coprocessor for DIFT propagation and checks through a coprocessor interface. At a system call or trap, the main core waits for the coprocessor to complete the DIFT operations for the system call and all preceding instructions, before the main core can commit the system call. External interrupts (e.g., time interrupts) are treated similarly by associating them with a pending instruction which becomes equivalent to a trap. When the coprocessor discovers that a DIFT check has failed, it notifies the core about the security attack using an asynchronous exception.

The advantage of this approach is that the main core does not stall for the DIFT coprocessor even if the latter is temporarily stalled due to accessing tags from main memory. It essentially eliminates most performance overheads of DIFT processing without requiring

OOO execution capabilities in the main core. While there is a small overhead for synchronization at system calls, system calls are not frequent and their overheads are typically in the hundreds or thousands of cycles. Thus, the few tens of cycles needed in the worst case to synchronize the main core and the DIFT coprocessor are not a significant issue.

Synchronizing at system calls implies that a number of additional instructions will be able to commit in the processor behind an instruction that causes a DIFT check to fail in the coprocessor. This, however, is acceptable and does not change the strength of the DIFT security model [62, 74, 75]. While the additional instructions can further corrupt the address space of the application, an attacker cannot affect the rest of the system (other applications, files, or the OS) without a system call or trap to invoke the OS. The state of the affected application will be discarded on a security exception that terminates the application prior to taking a system call trap. Other applications that share read-only data or read-only code are not affected by the termination of the application under attack. Only applications (or threads) that share read-write data or code with the affected application (or thread), and access the corrupted state need to be terminated, as is the case with integrated DIFT architectures. Thus, DIFT systems that synchronize on system calls provide the same security guarantees as DIFT systems that synchronize on every instruction [75].

For the program under attack or any other programs that share read-write data with it, DIFT-based techniques do not provide recovery guarantees to begin with. DIFT detects an attack at the time the vulnerability is exploited via an illegal operation, such as dereferencing a tainted pointer. Even with a precise security exception at that point, it is difficult to recover as there is no way to know when the tainted information entered the system, how many pointers, code segments, or data-structures have been affected, or what code must be executed to revert the system back to a safe state. Thus, DIFT does not provide reliable recovery. Consequently, delaying the security exception by a further number of instructions does not weaken the robustness of the system. If DIFT is combined with a checkpointing scheme that allows the system to roll back in time for recovery purposes, we

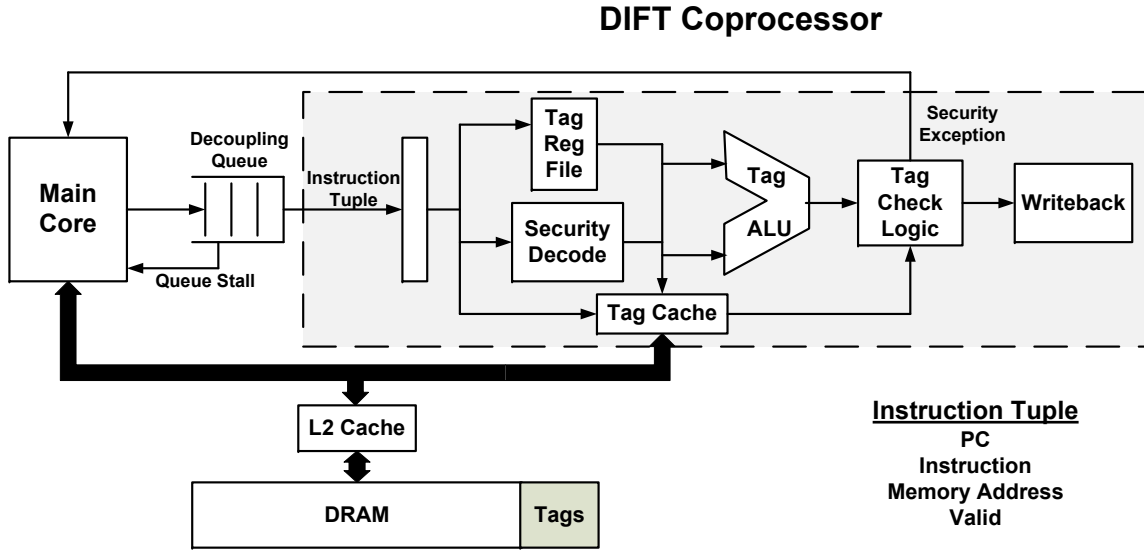


Figure 5.2: The pipeline diagram for the DIFT coprocessor. Structures are not drawn to scale.

can synchronize the main processor and the DIFT coprocessor every time a checkpoint is initiated.

While system call synchronization works for user-level code, it cannot be used to protect the operating system. We address this issue by synchronizing the main core and the DIFT coprocessor on device driver accesses within the operating system. This effectively prevents the application from performing any I/O and effecting any state change, before passing all the required security checks. This allows us to use the DIFT coprocessor for protecting the operating system as well. Critical sections of memory, such as the security handler, are protected by mapping them to read-only memory pages. This prevents the attacker from being able to override the security guarantees of the system.

5.2.2 Coprocessor microarchitecture

Figure 5.2 presents the pipeline of the DIFT coprocessor. Its microarchitecture is quite simple, as it only needs to handle tag propagation and checks. All other instruction execution capabilities are retained by the main core. Similar to Raksha [24], our coprocessor supports up to four concurrent security policies using 4-bit tags per word.

The coprocessor's state includes three components. First, there is a set of configuration registers that specify the propagation and check rules for the four security policies. We discuss these registers further in Section 5.2.3. Second, there is a register file that maintains the tags for the associated architectural registers in the main processor. Third, the coprocessor uses a cache to buffer the tags for frequently accessed memory addresses (data and instructions).

The coprocessor uses a four-stage pipeline. Given an executed instruction by the main core, the first stage decodes it into primitive operations and determines the propagation and check rules that should be applied based on the active security policies. In parallel, the 4-bit tags for input registers are read from the tag register file. This stage also accesses the tag cache to obtain the 4-bit tag for the instruction word. The second stage implements tag propagation using a tag ALU. This 4-bit ALU is simple and small in area. It supports logical OR, AND, and XOR operations to combine source tags. The second stage will also access the tag cache to retrieve the tag for the memory address specified by load instructions, or to update the tag on store instructions (if the tag of the instruction is zero). The third stage performs tag checks in accordance with the configured security policies. If the check fails (non-zero tag value), a security exception is raised. The final stage does a write-back of the destination register's tag to the tag register file.

The coprocessor's pipeline supports forwarding between dependent instructions to minimize stalls. The main source of stalls are misses in the tag cache. If frequent, such misses will eventually stall the main core and lead to performance degradation, as we discuss in Section 5.2.3. We should point out, however, that even a small tag cache can provide high

coverage. Since we maintain a 4-bit tag per 32-bit word, a tag cache size of T provides the same coverage as an ordinary cache of size $8 \times T$.

5.2.3 DIFT coprocessor interface

The interface between the main core and the DIFT coprocessor is a critical aspect of the architecture. There are four issues to consider: *coprocessor setup*, *instruction flow information*, *decoupling*, and *security exceptions*.

DIFT Coprocessor Setup: To allow software to control the security policies, the coprocessor includes four pairs of registers that control the propagation and check rules for the four tag bits. These policy registers specify the propagation and check modes for each class of *primitive operations*. Their operation and encoding are modeled on the corresponding registers in Raksha [24]. The configuration registers can be manipulated by the main core either as *memory-mapped registers* or as registers accessible through *coprocessor instructions*. In either case, the registers should be accessible only from within a trusted *security monitor*. Our prototype system uses the coprocessor instructions approach. The coprocessor instructions are treated as `nops` in the main processor pipeline. These instructions are used to manipulate tag values, and read and write the coprocessor's tag register file. This functionality is necessary for context switches. Note that coprocessor setup typically happens once per application or context switch.

Instruction Flow Information: The coprocessor needs information from the main core about the committed instructions in order to apply the corresponding DIFT propagation and checks. This information is communicated through a coprocessor interface.

The simplest option is to pass a stream of committed program counters (PCs) and load/store memory addresses from the main core to the coprocessor. The PCs are necessary to identify instruction flow, while the memory addresses are needed because the coprocessor only tracks tags and does not know the data values of the registers in the main core. In this scenario, the coprocessor must obtain the instruction encoding prior to performing

DIFT operations, either by accessing the main core’s I-cache or by accessing the L2 cache and potentially caching instructions locally as well. Both options have disadvantages. The former would require the DIFT engine to have a port into the I-cache, creating complexity and clock frequency challenges. The latter increases the power and area overhead of the coprocessor and may also constrain the bandwidth available at the L2 cache. There is also a security problem with this simple interface. In the presence of self-modifying or dynamically generated code, the code in the main core’s I-cache could differ from the code in the DIFT engine’s I-cache (or the L2 cache) depending on eviction and coherence policies. This inconsistency can compromise the security guarantees of DIFT by allowing an attacker to inject instructions that are not tracked on the DIFT coprocessor.

To address these challenges, we propose a coprocessor interface that includes the instruction encoding in addition to the PC and memory address. As instructions become ready to commit in the main core, the interface passes a tuple with the necessary information for DIFT processing (PC, instruction encoding, and memory address). Instruction tuples are passed to the coprocessor in program order. Note that the information in the tuple is available in the re-order buffer of OOO cores or the last pipeline register of in-order cores to facilitate exception reporting. The processor modifications are thus restricted to the interface required to communicate this information to the coprocessor. This interface is similar to the lightweight profiling and monitoring extensions recently proposed by processor vendors for performance tracking purposes [2]. The instruction encoding passed to the coprocessor may be the original one used at the ISA level or a predecoded form available in the main processor. For x86 processors, one can also design an interface that communicates information between the processor and the coprocessor at the granularity of micro-ops. This approach eliminates the need for x86 decoding logic in the coprocessor.

Decoupling: The physical implementation of the interface also includes a *stall* signal that indicates the coprocessor’s inability to accept any further instructions. This is likely to happen if the coprocessor is experiencing a large number of misses in the tag cache. Since

the locality of tag accesses is usually greater than the locality of data accesses (see Section 5.2.4), the main core will likely be experiencing misses in its data accesses at the same time. Hence, the coprocessor will rarely be a major performance bottleneck for the main core. Since the processor and the coprocessor must only synchronize on system calls, an extra queue can be used between the two in order to buffer instruction tuples. The queue can be sized to account for temporary mismatches in instruction processing rates between the processor and the coprocessor. The processor stalls only when the decoupling queue is full or when a system call instruction is executed.

To avoid frequent stalls due to a full queue, the coprocessor must achieve an instruction processing rate equal to, or greater than, that of the main core. Since the coprocessor has a very shallow pipeline, handles only committed instructions from the main core, and does not have to deal with mispredicted instructions, a single-issue coprocessor is sufficient for most superscalar processors that achieve IPCs close to one. For wide-issue superscalar processors that routinely achieve IPCs higher than one, a wide-issue coprocessor pipeline would be necessary. Since the coprocessor contains 4-bit registers and 4-bit ALUs and does not include branch prediction logic, a wide-issue coprocessor pipeline would not be particularly expensive. In Section 5.4.2, we provide an estimate of the IPC attainable by a single-issue coprocessor, by showing the performance of the coprocessor when paired with higher IPC main cores.

Security Exceptions: As the coprocessor applies tag checks using the instruction tuples, certain checks may fail, indicating potential security threats. On a tag check failure, the coprocessor interrupts the main core in an asynchronous manner. To make DIFT checks applicable to the operating system code as well, the interrupt should switch the core to the trusted security monitor which runs in either a special trusted mode [24, 25], or in the hypervisor mode in systems with hardware support for virtualization [39]. This allows us to catch bugs in both userspace and in the kernel [25]. The security monitor uses the protection mechanisms available in these modes to protect its code and data from a compromised

operating system. Once invoked, the monitor can initiate the termination of the application or guest OS under attack. We protect the security monitor itself using a sandboxing policy on one of the tag bits. For an in-depth discussion of exception handling and security monitors, we refer the reader to related work [24]. Note, however, that the proposed system differs from integrated DIFT architectures only in the synchronization between the main core and the coprocessor. Security checks and the consequent exception processing (if necessary) have the same semantics and operation in the coprocessor-based and the integrated designs.

5.2.4 Tag cache

The main core passes the memory addresses for load/store instructions to the coprocessor. Since instructions are communicated to the coprocessor after being committed by the main core, the address passed can be a physical one. Hence, the coprocessor does not need a separate TLB. Consequently, the tag cache is physically indexed and tagged, and does not need to be flushed on page table updates and context switches.

To detect code injection attacks, the DIFT coprocessor must also check the tag associated with the instruction's memory location. As a result, tag checks for load and store instructions require two accesses to the tag cache. This problem can be eliminated by providing separate instruction and data tag caches, similar to the separate instruction and data caches in the main core. A cheaper alternative that performs equally well is using a unified tag cache with an L0 buffer for instruction tag accesses. The L0 buffer can store a cache line. Since tags are narrow (4 bits), a 32-byte tag cache line can pack tags for 64 memory words providing good spatial locality. We access the L0 buffer and the tag cache in parallel. For non memory instructions, we access both components with the same address (the instruction's PC). For loads and stores, we access the L0 buffer with the PC and the unified tag cache with the address for the memory tags. This design causes a pipeline stall only when the L0 buffer misses on an instruction tag access, and the instruction is a load or a

Parameter	Specification
Leon pipeline depth	7 stages
Leon instruction cache	8 KB, 2-way set-associative
Leon data cache	16 KB, 2-way set-associative
Leon instruction TLB	8 entries, fully associative
Leon data TLB	8 entries, fully associative
Coprocessor pipeline depth	4 stages
Coprocessor tag cache	512 Bytes, 2-way set-associative
Decoupling queue size	6 entries

Table 5.1: The prototype system specification.

store that occupies the port of the tag cache. This combination of events is rare.

5.2.5 Coprocessor for in-order cores

There is no particular change in terms of functionality in the design of the coprocessor or the coprocessor interface if the main core is in-order or out-of-order. Since the two synchronize on system calls, the only requirement for the main processor is that it must stall if the decoupling queue is full, or if a system call is encountered. Coupling the DIFT coprocessor with different main cores could highlight different performance issues. For example, we may need to re-size the decoupling queue to hide temporary performance mismatches between the two. Our full-system prototype (see Section 5.3) makes use of an in-order main core.

5.3 Prototype

To evaluate the coprocessor-based approach for DIFT, we developed a full-system FPGA prototype based on the SPARC architecture and the Linux operating system. Our prototype is based on the framework provided by the Raksha integrated DIFT architecture [24]. This allows us to make direct performance and complexity comparisons between the integrated and coprocessor-based approaches for DIFT hardware.

5.3.1 System architecture

The main core in our prototype is the Leon SPARC V8 processor, a 32-bit synthesizable core [49]. Leon uses a single-issue, in-order, 7-stage pipeline that does not perform speculative execution. Leon supports SPARC coprocessor instructions, which we use to configure the DIFT coprocessor and provide security exception information. We introduced a decoupling queue that buffers information passed from the main core to the DIFT coprocessor. If the queue fills up, the main core is stalled until the coprocessor makes forward progress. Since the main core commits instructions before the DIFT coprocessor, security exceptions are imprecise.

The DIFT coprocessor follows the description in Section 5.2. It uses a single-issue, 4-stage pipeline for tag propagation and checks. Similar to Raksha, we support four security policies, each controlling one of the four tag bits. The tag cache is a 512-byte, 2-way set-associative cache with 32-byte cache lines. Since we use 4-bit tags per word, the cache can effectively store the tags for 4 Kbytes of data.

Our prototype provides a full-fledged Linux workstation environment. We use Gentoo Linux 2.6.20 as our kernel and run unmodified SPARC binaries for enterprise applications such as Apache, PostgreSQL, and OpenSSH. We have modified a small portion of the Linux kernel to provide support for our DIFT hardware [24, 25]. The security monitor is implemented as a shared library preloaded by the dynamic linker with each application.

5.3.2 Design statistics

We synthesized our hardware (main core, DIFT coprocessor, and memory system) onto a Xilinx XUP board with an XC2VP30 FPGA. Table 5.1 presents the default parameters for the prototype. Table 5.2 provides the basic design statistics for our coprocessor-based design. We quantify the additional resources necessary in terms of 4-input LUTs (lookup tables for logic) and block RAMs, for the changes to the core for the coprocessor interface, DIFT coprocessor (including the tag cache), and the decoupling queue. For comparison

Component	BRAMs	4-input LUTs
Base Leon core (integer)	46	13,858
FPU control & datapath Leon	4	14,000
Core changes for Raksha	4	1,352
% Raksha increase over Leon	8%	4.85%
Core changes for coprocessor IF	0	22
Decoupling queue	3	26
DIFT coprocessor	5	2,105
Total DIFT coprocessor	8	2,131
% coprocessor increase over Leon	16%	7.64%

Table 5.2: Complexity of the prototype FPGA implementation of the DIFT coprocessor in terms of FPGA block RAMs and 4-input LUTs.

purposes, we also provide the additional hardware resources necessary for the Raksha integrated DIFT architecture. Note that the same coprocessor can be used with a range of other main processors: processors with larger caches, speculative execution, etc. In these cases, the overhead of the coprocessor as a percentage of the main processor would be even lower in terms of both logic and memory resources.

The coprocessor design represents a 7% increase in LUTs and a 16% increase in BRAMs over the base Leon design. Most of the complexity is isolated in the coprocessor. The increase in the logic of the main core for the core-coprocessor interface is less than 0.1%. A significant portion of the coprocessor overhead is due to the decoupling queue. Note that the same coprocessor can be used with a range of other main processors with sustained IPC of 1: a processor with larger caches, speculative and out of order execution, SIMD extensions, etc. In these cases, the overhead of the coprocessor as a percentage of the main processor would be even lower in terms of both logic and memory resources.

For example, we can consider the synthesizable Intel Pentium design presented by Lu et al [53]. This is a 32-bit, in-order, dual-issue, 5-stage pipeline for the x86 ISA that includes floating-point hardware [69]. It uses 8-KByte, 2-way set-associative first-level caches for data and instructions. Since the IPC of the dual-issue Pentium is typically below 1, the single-issue DIFT coprocessor would be sufficient for servicing this main core as well.

On a Xilinx Virtex-4 LX200 FPGA, the design uses 65,615 4-input LUTs and 118 block RAMs, roughly 2.3 times the size of Leon. Hence, the area overhead of adding the DIFT coprocessor to the Pentium would be roughly 3% (first-order approximation). Modern superscalar designs are significantly more complicated than the Leon and Pentium. They include far deeper pipelines, more physical registers, and more functional units (integer, FPUs, SIMD, etc.). Even if the coprocessor pipeline is upgraded to be dual or quad issue, the area overhead of the coprocessor is likely to be below 1%. This is primarily because the coprocessor processes only non-speculative instructions and performs simple 4-bit logical operations. We evaluate the issue of performance (mis)match between the main core and the coprocessor in Section 5.4.2.

We can also compare the cost of the coprocessor to that of alternative approaches for DIFT hardware. The overhead of the Raksha integrated DIFT system over the base Leon design is 8% in terms of BRAMs and 4% in terms of logic. This is roughly half the overhead of the coprocessor. Raksha benefits from sharing logic and buffering resources between the data and DIFT functionalities within the core. For the specific FPGA mapping, it also benefits from the fact that Xilinx BRAMs provide 36-bit words; hence extending registers and cache lines by 4 bits per word in Raksha is essentially free. Nevertheless, there are two important issues to note. First, the overhead of the integrated approach is proportional to the complexity of the core. Since all registers (physical and architectural) and all pipeline buffers must be extended, the absolute cost of the integrated approach would be higher for a more complicated processor with a deeper pipeline or a bigger data cache. In contrast, the complexity of the DIFT coprocessor is only proportional to the sustained IPC of the main core. Second, modifications required by an integrated DIFT approach such as Raksha must be in-lined with the processor logic. In contrast, the coprocessor approach separates all functionality for DIFT, and thus its complexity does not affect the processor design or verification time.

We can also compare the coprocessor's complexity to that of the offloading DIFT approach. Offloading would lead to an area overhead of 100% in order to provide the second core for the DIFT analysis. The absolute overhead would be even higher if we consider more advanced processor cores as the complexity of the superscalar processor core typically grows superlinearly with IPC (due to speculation), while the complexity of the coprocessor only grows roughly linearly. It is also interesting to consider the changes to the processor core that are required to support the trace exchange between the application and the DIFT core in the offloading approach. Each core requires a 32-Kbyte table for compression, while an additional 16-Kbyte table is required for the analysis core [12, 13]. The 32-Kbyte table is significantly larger than the tag cache (512 bytes) and decoupling queue (6 entries) in our DIFT coprocessor. A 32-Kbyte SRAM is larger than the whole coprocessor and probably as large as the Leon core (integer and floating point hardware) in most implementation technologies. Reducing the size of compression tables will lead to additional traffic and performance overheads. The offloading systems also require other significant modifications to the cores for inheritance tracking [13]. Overall, the area, cost, and power advantages of the coprocessor approach over the offloading approach are significant.

At its core, the coprocessor is comprised mainly of a cache and a register file for tags, with basic combinatorial logic for manipulating 4-bit tags. Table 5.3 provides area and power overhead numbers for the memory elements of the coprocessor. Similar to the evaluation in Chapter 4, we use CACTI 5.2 [85] to get area and power utilization numbers for a coprocessor design fabricated at a 65nm process technology. Compared to the equivalent overheads of the Raksha design (discussed in Chapter 4), these numbers are extremely low. This is because of the extremely small cache used for tags. Note that this varies from the FPGA utilization numbers quoted in Table 5.2, which seem to indicate that the caches in the coprocessor design occupy more space than in the Raksha design. This disparity in FPGA BRAM usage can be attributed to the fact that the Virtex-II FPGAs have 36-bit wide

Storage Element	Area Overhead (% increase)	Standby Leakage Power Overhead (% increase)
Unified Cache	0.423mm ² (12.86%)	4.75e-07 W (14.09%)
Register File	0.031mm ² (10.91%)	0.162e-08 W (7.62%)

Table 5.3: The area and power overhead values for the storage elements in the offcore prototype. Percentage overheads are shown relative to corresponding data storage structures in the unmodified Leon design.

BRAMs. Since the Raksha design makes modifications to the Leon’s caches, the FPGA place and route utilities store the security tags in the BRAMs already used to implement the caches. The coprocessor being a separate entity requires its own set of BRAMs.

5.4 Evaluation

This section evaluates the security capabilities and performance overheads of the DIFT coprocessor.

5.4.1 Security evaluation

To evaluate the security capabilities of our design, we attempted a wide range of attacks on real-world applications in userspace and kernelspace, using unmodified SPARC binaries. We configured the coprocessor to implement the same DIFT policies (check and propagate rules) used for evaluating the security of the Raksha design [24, 25]. For the low-level memory corruption attacks such as buffer overflows, hardware performs taint propagation and checks for the use of tainted values as instruction pointers, data pointers, or instructions. Synchronization between the main core and the coprocessor occurs on system calls and device-driver accesses to ensure that any pending security exceptions are taken. For

Program (Lang)	Attack	Analysis	Detected Vulnerability
gzip (C)	Directory traversal	String tainting + System call interposition	Open file with tainted absolute path
tar (C)	Directory traversal	String tainting + System call interposition	Open file with tainted absolute path
Scry (PHP)	Cross-site scripting	String tainting + System call interposition	Tainted HTML output includes < script >
htdig (C++)	Cross-site scripting	String tainting + System call interposition	Tainted HTML output includes < script >
polymorph (C)	Buffer (stack) overflow	Pointer injection	Tainted code pointer dereference (return address)
sendmail (C)	Buffer (BSS) overflow	Pointer injection	Tainted data pointer dereference (application data)
quotactl syscall (C)	User/kernel pointer dereference	Pointer injection	Tainted pointer to kernel space
SUS (C)	Format string bug	String tainting + Function call interposition	Tainted format string specifier in syslog
WU-FTPD (C)	Format string bug	String tainting + Function call interposition	Tainted format string specifier in <code>vfprintf</code>

Table 5.4: The security experiments performed with the DIFT coprocessor.

high-level semantic attacks such as directory traversals, the hardware performs taint propagation, while the software monitor performs security checks for tainted commands on sensitive function and system call boundaries similar to Raksha [24]. We protect against Web vulnerabilities like cross-site scripting by applying this tainting policy to Apache, and any associated modules like PHP.

Table 5.4 summarizes our security experiments. The applications were written in multiple programming languages and represent workloads ranging from common utilities (gzip, tar, polymorph, sendmail, sus), to server and web systems (scry, htdig, wu-ftpd), to kernel code (quotactl). All experiments were performed on unmodified SPARC binaries with

no debugging or relocation information. The coprocessor successfully detected both high-level attacks (directory traversals and cross-site scripting) and low-level memory corruptions (buffer overflows and format string bugs), even in the OS (user/kernel pointer). We can concurrently run all the analyses in Table 5.4 using 4 tag bits: one for tainting untrusted data, one for identifying legitimate pointers, one for function/system call interposition, and one for protecting the security handler. The security handler is protected by sandboxing its code and data.

We used the pointer injection policy described in [25] for catching low-level attacks. This policy uses two tag bits, one for identifying all the legitimate pointers in the system, and another for identifying tainted data. The invariant enforced is that tainted data cannot be dereferenced, unless it has been deemed to be a legitimate pointer. This analysis is very powerful, and has been shown to reliably catch low-level attacks such as buffer overflows, and user/kernel pointer dereferences, in both userspace and kernelspace, without any false positives [25].

Our offcore DIFT implementation of these security policies gave us results consistent with prior state-of-the-art integrated DIFT designs [24, 25], proving that our delayed synchronization model does not compromise on security. Note that the security policies used to evaluate our coprocessor are stronger than those used to evaluate other DIFT architectures, including FlexiTaint [14, 20, 81, 88]. For instance, FlexiTaint does not detect code injection attacks and suffers from false positives and negatives on memory corruption attacks. Overall, the coprocessor provides software with exactly the same security features and guarantees as the Raksha design [24, 25], proving that our delayed synchronization model does not compromise on security.

5.4.2 Performance evaluation

Performance Analysis

We measured the performance overhead due to the DIFT coprocessor using the SPECint2000 benchmarks. We ran each program twice, once with the coprocessor disabled and once with the coprocessor performing DIFT analysis (checks and propagates using taint bits). Since we do not launch a security attack on these benchmarks, we never transition to the security monitor (no security exceptions). The overhead of any additional analysis performed by the monitor is not affected when we switch from an integrated DIFT approach to the coprocessor-based one.

Figure 5.3 presents the performance overhead of the coprocessor configured with a 512-byte tag cache and a 6-entry queue (the default configuration), over an unmodified Leon. The integrated DIFT approach of Raksha has the same performance as the base design since there are no additional stalls [24]. The average performance overhead due to the DIFT coprocessor for the SPEC benchmarks is 0.79%. The negligible overheads are almost exclusively due to memory contention between cache misses from the tag cache and memory traffic from the main processor.

Performance Comparison

It is difficult to provide a direct performance comparison between the coprocessor-based approach and the offloading approach for DIFT hardware. Apart from creating a multi-core prototype following the description in [12], we would also need access to the dynamic binary translation environment described in [13]. For reference, the reported average slowdowns for applications using the offloading approach are 36% [13]. We performed an indirect comparison by evaluating the impact of communicating the trace between the application and analysis core, on application performance. After compression, the trace is exchanged between the two cores using bulk accesses to shared caches. Even though the

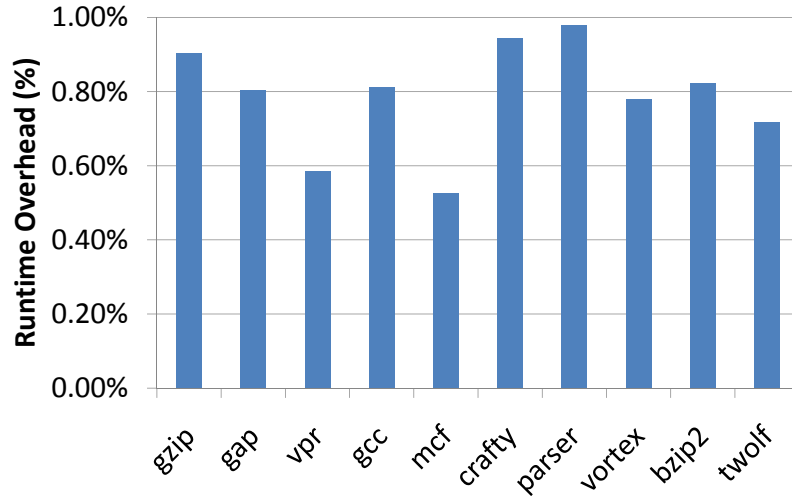


Figure 5.3: Execution time normalized to an unmodified Leon.

L1 cache of the application core is bypassed, the application core may still slow down due to contention at the shared caches between trace traffic and its own instruction and cache misses. To minimize contention, the offloading architecture described in [12] uses a 32-Kbyte table for value prediction that achieves a compression rate of 0.8 bytes of trace per executed instruction. The uncompressed trace is roughly 16 bytes per executed instruction. The application processor accumulates 64 bytes of compressed traces before it sends them to the application core. We found the performance overhead of exchanging these compressed traces between cores in bulk 64-byte transfers to be 5%. The actual multi-core system may have additional runtime overheads due to the synchronization of the application and analysis cores. In contrast, as Figure 5.3 shows, even a small tag cache and queue suffice for the DIFT coprocessor to keep up with the main core with minimal runtime overheads.

Figure 5.4 presents the performance impact on the main core while running three benchmarks (perl, gzip and gap) if we create and communicate an instruction trace. The trace is collected, compressed in hardware, and is sent to the memory system in bulk, 64-byte

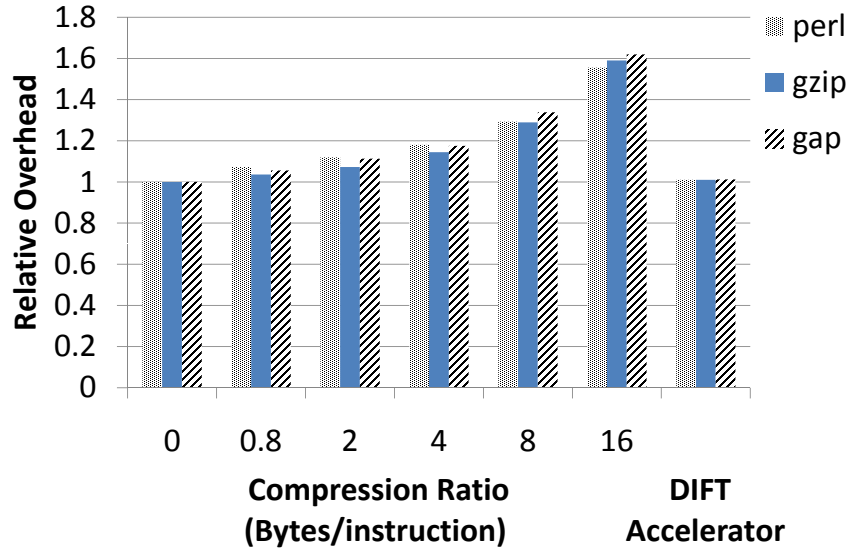


Figure 5.4: Comparison of the coprocessor approach against the hardware assisted off-loading approach.

transfers. The trace is immediately picked up by an additional device on the on-chip memory bus without causing actual DRAM accesses. Hence, the only performance bottleneck due to the trace is the contention for bus bandwidth. The trace does not go through the first level caches. Figure 5.4 shows execution time overhead as a function of the compression ratio achieved for the trace. If the trace is sent uncompressed (16 bytes per instruction), the applications slow down by around 60%. Increasing the compression rate by using a bigger table for value prediction reduces memory contention and the performance overhead. With a 32-Kbyte table, the compression rate is 0.8 bytes per instructions [13] and the overhead for the three applications is less than 5%. The actual offloading system may have additional overheads due to the synchronization of the application and analysis core. In contrast, our proposal (the last set of bars in Figure 5.4) leads to overheads of less than 1% using the significantly smaller and simpler coprocessor for DIFT processing.

Sensitivity Analysis

Since we synchronize the processor and the coprocessor at system calls, and the coprocessor achieves good locality with its tag cache, we did not observe a significant number of memory contention or queue related stalls for the SPECint2000 benchmarks. To evaluate the worst-case performance scenario, we wrote a microbenchmark that put pressure on the tag cache. The microbenchmark performed continuous memory operations designed to miss in the tag cache, without any intervening operations. This was aimed at increasing contention for the memory bus, thus causing the main processor to stall. Frequent misses in the tag cache could also cause the decoupling queue to fill up and stall the processor. Figure 5.5 presents the performance overhead due to the DIFT coprocessor as we run the microbenchmark and vary the capacity of the tag cache between 16 bytes and 1 Kbyte. This implies that the tag cache can store tags for an equivalent data memory of 128 bytes to 8 Kbytes. All our experiments use a two-way set-associative cache and a six entry decoupling queue. We break down execution time overhead into two components: the time that the processor is stalled because the decoupling queue of the coprocessor is full, and the time the processor is stalled because the memory system serves tag cache misses and cannot serve instruction or data misses. We observe that for tag cache sizes below 128 bytes, tag cache misses are frequent causing runtime overheads of 10% to 20%. With a tag cache of 512 bytes or more, tag cache misses are rare and the overhead drops to 2% even for this worst case scenario. The overhead is primarily due to compulsory and conflict misses in the tag cache that occur when the processor core is not stalled on its own due to pipeline dependencies, or data and instruction misses.

Since we synchronize the processor and the coprocessor at system calls, and the coprocessor has good locality with a small tag cache, we did not observe a significant number of memory contention or queue related stalls for the SPECint2000 benchmarks. We evaluated the worst-case scenario for the tag cache, by performing a series of continuous memory

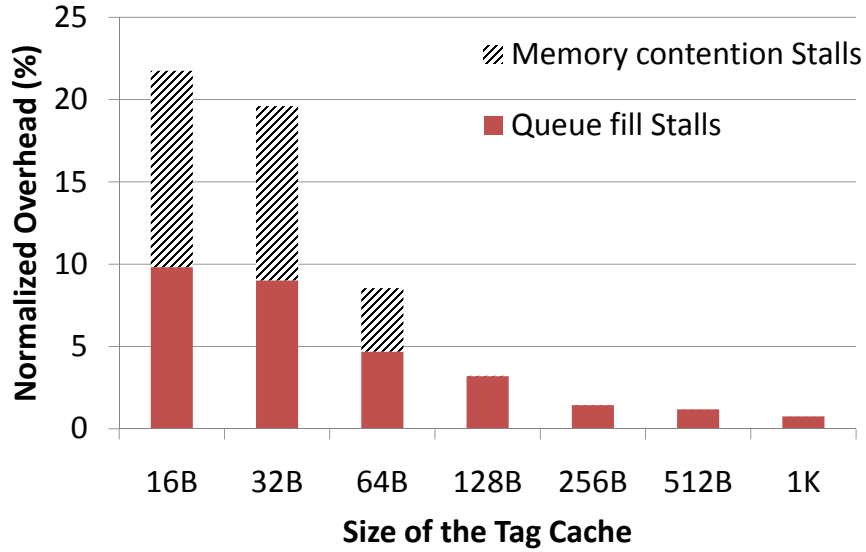


Figure 5.5: The effect of scaling the capacity of the tag cache.

operations designed to miss in the tag cache, without any intervening operations. This was aimed at increasing contention for the shared memory bus, causing the main processor to stall. We found that tag cache misses were rare with a cache of 512 bytes or more, and the overhead dropped to 2% even for this worst-case scenario. We also wrote a microbenchmark to stress test the performance of the decoupling queue. This worst-case scenario microbenchmark performed continuous operations that set and retrieved memory tags to simulate tag initialization. Since the coprocessor instructions that manipulate memory tags are treated as `nops` by the main core, they impact the performance of only the coprocessor, causing the queue to stall. Figure 5.6 shows the performance overhead of our coprocessor prototype as we run this microbenchmark and vary the size of the decoupling queue from 0 to 6 entries. For these runs we use a 16-byte tag cache in order to increase the number of tag misses and put pressure on the decoupling queue. Without decoupling, the coprocessor introduces a 10% performance overhead. A 6-entry queue is sufficient to drop the performance overhead to 3%. Note that the overhead of a 0-entry queue is equivalent to the overhead of a DIVA-like design which performs DIFT computations within the core, in

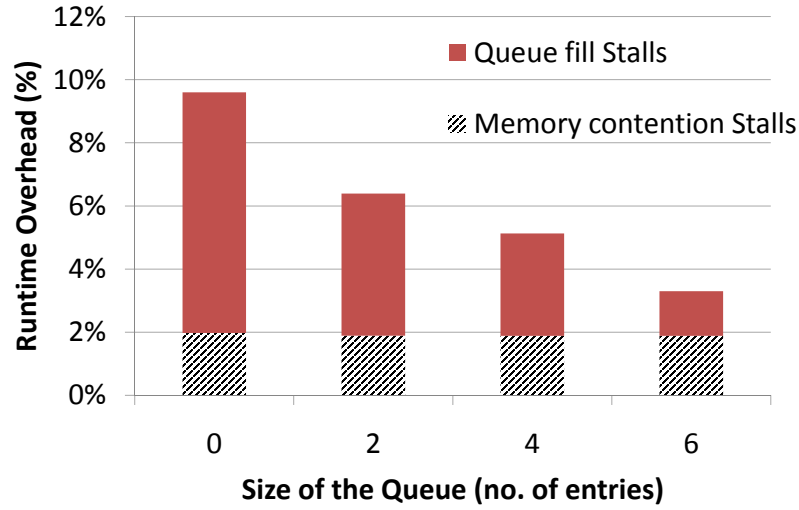


Figure 5.6: The effect of scaling the size of the decoupling queue on a worst-case tag initialization microbenchmark.

additional pipeline stages prior to instruction commit.

This result also provides an indirect evaluation of the pressure on the ROB of an out-of-order processor with precise security exceptions in a design like DIVA or FlexiTaint. At any point in time, there could be up to 10 instructions in the ROB that are ready to commit but are waiting for the coprocessor to complete the DIFT processing (6 in the decoupling queue and 4 in the coprocessor’s pipeline in this experiment). The FlexiTaint prototype reports lower performance overheads thanks to the prefetching hints for tags issued by the processor core prior to the DIFT pipeline stages. This, however, has the disadvantage of requiring additional changes in the out-of-order core (see discussion in Section 5.1). Our coprocessor-based design does not use prefetching hints from the main core. The decoupling queue and the coarse-grained synchronization at system calls provide sufficient time to deal with cache misses for tags without slowing down the main core.

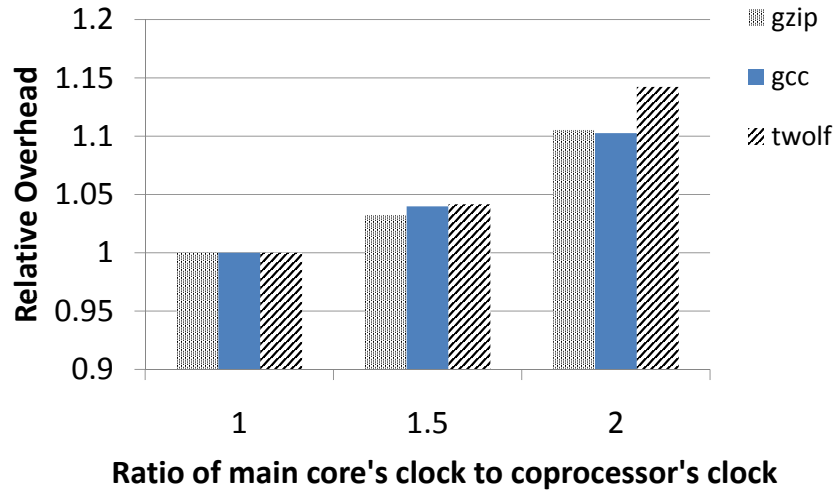


Figure 5.7: Performance overhead when the coprocessor is paired with higher-IPC main cores. Overheads are relative to the case when the main core and coprocessor have the same clock frequency.

Processor/Coprocessor Performance Ratio

The decoupling queue and the coarse-grained synchronization scheme allow the coprocessor to fall temporarily behind the main core. The coprocessor should however, be able to match the long-term IPC of the main core. While we use a single-issue core and coprocessor in our prototype, it is reasonable to expect that a significantly more capable main core will also require the design of a wider-issue coprocessor. Nevertheless, it is instructive to explore the right ratio of performance capabilities of the two. While the main core may be dual or quad issue, it is unlikely to frequently achieve its peak IPC due to mispredicted instructions, and pipeline dependencies. On the other hand, the coprocessor is mainly limited by the rate at which it receives instructions from the main core. The nature of its simple operations allows it to operate at high clock frequencies without requiring a deeper pipeline that would suffer from data dependency stalls. Moreover, the coprocessor only handles committed instructions. Hence, we may be able to serve a main core with peak IPC higher than 1 with the simple coprocessor pipeline presented.

To explore this further, we constructed an experiment where we clocked the coprocessor at a lower frequency than the main core. Hence, we can evaluate coupling the coprocessor with a main core that has a peak instruction processing rate $1.5\times$, or $2\times$ that of the coprocessor. As Figure 5.7 shows, the coprocessor introduces a modest performance overhead of 3.8% at the $1.5\times$ ratio and 11.7% at the $2\times$ ratio, with a 16-entry decoupling queue. These overheads are likely to be even lower on memory or I/O bound applications. This indicates that the same DIFT coprocessor design can be (re)used with a wide variety of main cores, even if their peak IPC characteristics vary significantly.

5.5 Summary

This chapter presented an architecture that provides hardware support for dynamic information flow tracking using an off-core, decoupled coprocessor. The coprocessor encapsulates all state and functionality needed for DIFT operations and synchronizes with the main core only on system calls. This design approach drastically reduces the cost of implementing DIFT: it requires no changes to the design, pipeline and layout of a general-purpose core, it simplifies design and verification, it enables use with in-order cores, and it avoids taking over an entire general-purpose CPU for DIFT checks. Moreover, it provides the same guarantees as traditional hardware DIFT implementations. Using a full-system prototype, we showed that the coprocessor introduces a 7% resource overhead over a simple RISC core. The performance overhead of the coprocessor is less than 1% even with a 512-byte cache for DIFT tags. We also demonstrated in practice that the coprocessor can protect unmodified software binaries from a wide range of security attacks.

Decoupling tags from the main core, however, has the effect of breaking the atomicity between tags and data. In the next chapter, we discuss the problems that could arise due to this lack of atomicity in multi-threaded workloads, and provide a low-cost solution to the same.

Chapter 6

Metadata Consistency in Multiprocessor Systems

Decoupling metadata processing as explained in the previous chapter helps render hardware DIFT analyses practical. This decoupling, however, breaks the atomicity between data and metadata updates and leads to consistency issues in multiprocessor systems [42, 88]. This can lead to incorrect metadata causing false positives (spurious attacks detected) or false negatives (real attacks missed). An attacker can actually exploit this inconsistency to subvert the security analysis [18].

This chapter introduces a comprehensive solution to the problem of consistency between application data and dynamic analysis metadata in multiprocessor systems. We use hardware that tracks coherence requests to dirty data made by processors running the application to ensure that analogous requests are made in the same order by processors used for metadata processing (analysis), hence eliminating incorrect orderings. This solution is also applicable to different models of memory consistency, including the relaxed consistency models used by commercial architectures such as *x86* and *SPARC* [40].

The rest of this chapter is organized as follows. Section 6.1 provides more insight into the consistency issue, and discusses related work. Section 6.2 presents our solution to the

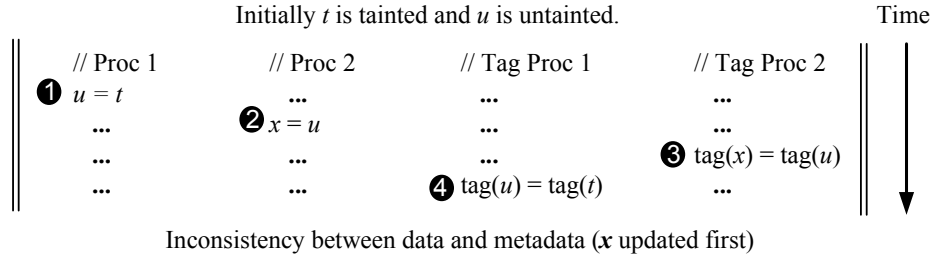


Figure 6.1: An inconsistency scenario where updates to data and metadata are observed in different orders.

consistency problem, and Section 6.3 discusses the related implementation and applicability issues. Section 6.4 presents the experimental evaluation, and Section 6.5 concludes the chapter.

6.1 (*Data, metadata*) Consistency

6.1.1 Overview of the (in)consistency problem

Figure 6.1 provides an example of a (*data, metadata*) consistency problem. Consider a multithreaded program running on a multi-core chip that operates on variables t and u . We use two additional cores that run parallel DIFT analyses to detect security attacks. These could either be the DIFT coprocessors introduced in Chapter 5, or the general-purpose analysis cores used by the log-based architecture [12]. Each word is associated with a tag that taints data arriving from untrusted sources (e.g., the network). Initially, t is tainted (untrusted), while u is untainted (trusted). Processor 1 first copies t to u which is subsequently read by processor 2. The associated tag (metadata) processors now perform analogous operations on the tags. Given the lack of any synchronization mechanism, tag processor 2 can perform a metadata load of $\text{tag}(u)$ prior to tag processor 1 storing to $\text{tag}(u)$.

This sequence of events would result in tag processor 2 getting a stale value of the

Requirement	SW [18, 61]	HW [88]	Work in this Chapter
Fast (speed)	N	Y	Y
Allows for full decoupling	Y	N	Y
Applicability to generic processors	N (TM)	N (OOO)	Y
Limited changes to processor/cache	Y	N	Y
Works with unmodified binaries	Y	Y	Y
Works with relaxed consistency	Y	Y	Y
Tag-data address variable mapping	Y	N	Y

Table 6.1: Comparison of different schemes for maintaining (*data*, *metadata*) consistency.

tag. Even though tag processor 2 uses the untrusted value obtained from processor 1, the associated tag indicates the data to be safe. If $\times$ is subsequently used as code or as a code pointer, an undetected security breach will occur (false negative) that may allow an attacker to take over the system [18]. Similarly, it is possible to construct scenarios where a stale tag could indicate that safe information is untrusted, causing erroneous security breaches (false positives) to be reported [18]. In general, one can construct numerous scenarios with races in updates to (*data*, *metadata*) pairs. Depending on the exact use of the metadata, the races can lead to incorrect results, program termination, undetected malicious actions, etc.

6.1.2 Requirements of a solution

Table 6.1 lists the desired characteristics of a solution to the (*data*, *metadata*) consistency problem. Of course, any solution must have a minimal *performance overhead*. Prior work [12, 42] has demonstrated the feasibility and practicality of the hardware decoupling of data and metadata for single processor workloads. Our goal in this chapter is to extend these architectures to work correctly in multiprocessor systems.

Degree of Decoupling: The solution must work well with both approaches for decoupling metadata processing: dedicated programmable coprocessors [42] and use of additional cores in a multi-core system [12]. Both approaches handle metadata operations many cycles after the corresponding application instructions have committed. These approaches

differ in the degree of decoupling. If a conventional core is used, the metadata processing may happen hundreds of cycles later as the application and analysis cores communicate using compressed traces over the coherence interconnect and through shared caches [13].

Applicability: The solution must work equally well for in-order and out-of-order (OOO) cores. Processor vendors are introducing multi-core chips using both types of cores. Upcoming heterogeneous designs will further stress this requirement. It is also our goal to limit hardware changes to outside the core's pipeline and primary caches, since any modification to either of these components significantly increases design and validation costs. Moreover, dynamic analysis should be transparent to the application binary without the need for recompilation or other changes to solve the consistency problem. Finally, the solution should work for any memory consistency model, sequential or relaxed.

Metadata flexibility: To accommodate different dynamic analyses, the solution should work with metadata of different lengths (short or long). Moreover, it should impose no restrictions in the mapping scheme from data to metadata addresses. The solution should be able to use any mapping in order to minimize storage overheads for metadata [81].

6.1.3 Previous efforts

Software approaches: Chung *et al.* [18] proposed a software solution for (*data, metadata*) consistency using transactional memory (TM). A dynamic binary translator (DBT) instruments the application by inserting metadata operations after the corresponding data accesses. Atomicity of (*data, metadata*) updates is maintained by encapsulating both the data and metadata operations within a transaction.

The main drawback of this solution is its runtime overhead. In addition to the overhead of running the analysis in the same core as the application ($3\times$ to $40\times$ [65, 73]), this approach introduces a 40% slowdown to solve consistency issues. The overhead can be

reduced if the processor has hardware support for TM. A recent proposal [61] uses translation to encapsulate the data and metadata references within an atomic block similar to a transaction, and uses *coupled coherence* where the coherence actions for metadata are triggered by those on the application data. This proposal suffers from performance issues similar to the TM approach.

Hardware approaches: FlexiTaint [88] implements DIFT in hardware at the back end of the processor. It adds two pipeline stages prior to the final commit stage, which operate on metadata from a separate register file and cache. Application instructions are not committed until the corresponding metadata operations are performed. By looking up coherence requests in queues of pending instructions, FlexiTaint can detect when a consistency problem occurs. In this case, a replay trap (pipeline flush) is used to restore ordering. FlexiTaint also modifies the store logic to store to the tag and data caches only when both writes are hits. The disadvantage of this approach is that it requires an OOO processor with support for replay traps. The processor and primary caches must be modified significantly to accommodate the DIFT hardware. This approach cannot be used with in-order processors or when the analysis hardware is decoupled to a coprocessor or another core. Moreover, it does not work with a variable mapping between data and metadata addresses.

6.2 Protocol for (*data, metadata*) Consistency

6.2.1 Protocol overview

Our solution maintains (*data, metadata*) consistency by keeping track of coherence requests to dirty application data and requests for exclusive access over data cache blocks (as part of a write on the requesting core), and requiring that there be corresponding metadata requests. For each address, we force metadata requests to match data requests. That is to say, if core A requests a data word written by core B, we require that tag core A request the

corresponding metadata word from tag core B. Any intervening access to the same metadata from a different core will be delayed to ensure consistency. Keeping track of coherence requests to dirty data, and requests for exclusive access over cache blocks, essentially provides us with a log of the *memory races* between threads. This information allows us to faithfully recreate the application’s execution ordering on the metadata. Consequently, incorrect executions such as the one in Figure 6.1 are avoided. Using coherence events to recreate the access order has been shown to be deadlock-free under sequentially consistent memory models [92]. We discuss relaxed consistency memory models in Section 6.3.2.

Our protocol assumes the presence of an application core (*a-core*) and a separate analysis core (*m-core* for metadata processing) as shown in Figure 6.2. This is the model adopted by previous work that focuses on decoupling metadata processing from processor cores [12, 42]¹. Multiple such pairs exist in a multi-core chip. The a-core provides the m-core with a stream of committed instructions to analyze. Each instruction in the stream is associated with a unique ID for tracking purposes. We introduce two new tables that are shared by the two cores and keep track of the a-core’s coherence requests (PTRT) and responses (PTAT) for dirty data or exclusive access. The table entries track both the a-core instruction IDs that generate or service the request², as well as the addresses involved. Software prefetching requests (such as PrefetchW instructions) are also tracked, since they modify the state of the cache line.

The m-core checks these tables prior to issuing coherence requests on cache misses for metadata. The PTRT provides the m-core with information on the proper destination for the metadata request. The PTAT is consulted when the m-core receives coherence requests for metadata from other analysis cores. For each address, the m-core services the metadata requests in the same order in which the a-core serviced the data requests. If metadata

¹It is possible for one m-core to serve multiple a-cores [42]. In such cases, we associate a virtual instance of each m-core with every physical a-core.

²We define the instruction that generates the memory value used to service a coherence request, as the instruction *servicing the request*.

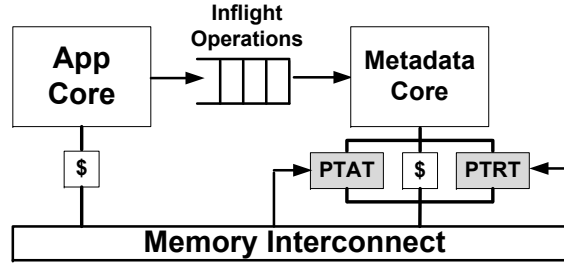


Figure 6.2: Overview of the system showing a single (a-core, m-core) pair. Structures are not drawn to scale.

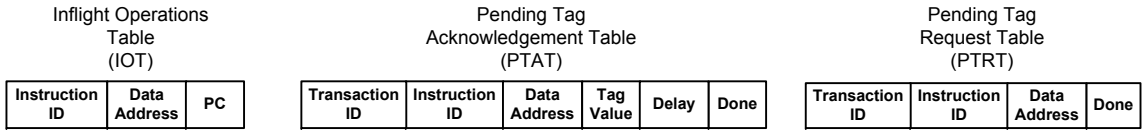


Figure 6.3: The three tables added to the system.

requests do not find matching entries in the two tables, they are allowed to proceed as normal (benign case). The advantage of this scheme is that it does not pessimistically enforce atomicity between application data and metadata accesses, while ensuring that no inconsistent ordering is observable.

6.2.2 Protocol implementation

The tracking scheme for consistency enforcement is fully distributed. The m-core in Figure 6.2 could either be a general purpose core [13] or a dedicated coprocessor [42]. Decoupling metadata processing requires a buffer to keep track of instructions committed by the a-core until they are processed by the m-core. Figure 6.2 uses an *Inflight Operations Table* (IOT) which is similar to the decoupling queue used in the coprocessor design [42]. The instruction stream can also be exchanged through the memory interconnect and shared caches (log buffering and compression [13]). To enforce (*data, metadata*) consistency, we

need three fields per entry in this table: an *Instruction ID* field, a *Memory address* field, and a *PC* field that stores the program counter. Additional fields per instruction are necessary to support various types of analyses (see [13, 42]). The ID can be a simple counter that is incremented for each committed instruction. We assign the instruction ID outside of the processor (after the instruction has committed) to avoid any changes to its pipeline. Table entries are deallocated when they are processed by the m-core.

We introduce two new tracking tables called the *Pending Tag Acknowledgment Table* (PTAT), and the *Pending Tag Request Table* (PTRT). The PTRT keeps track of coherence requests made by the a-core when it experiences cache misses. The PTAT keeps track of responses provided by the a-core when it receives coherence requests due to misses at other a-cores in the system. The format of these tables is shown in Figure 6.3. These tables merely monitor the a-core's coherence requests and responses, but do not need to be part of the a-core. Aside from providing a simple interface to communicate with the m-core via the IOT (as per decoupled processing architectures [13, 42]), the a-core requires no modifications.

The PTRT provides the m-core with information on the destination for its coherence requests on metadata misses. PTRT entries are allocated whenever (a) the a-core issues a request for exclusive control over a cache block as part of a store, or (b) the a-core receives a response to a coherence request it issued to a dirty cache block. The *Transaction ID* of the request is noted, along with the *Instruction ID* of the a-core instruction making the request. The Instruction ID is obtained by searching the IOT for the ID associated with the memory address and PC of the requesting instruction. The Transaction ID is the ID of the coherence request on the interconnect, and is assumed to contain information about the a-core responding to the request. This might not be true in some directory based systems, in which case, an extra field must be added to coherence messages. The m-core analyzes instructions after the a-core commits them. The corresponding metadata request must lookup the PTRT using the instruction ID. If there is a matching entry, the metadata

request is sent to the m-core associated with the a-core that serviced the data request. If the destination m-core evicted the block in question from its cache in the meantime, the request is redirected to the lower levels of the memory hierarchy. The PTRT entry is deallocated when the response for the metadata request is received.

The PTAT allows the m-core to delay servicing any incoming coherence requests for metadata in order to avoid consistency issues. PTAT entries are allocated when the a-core responds to a coherence request from another a-core. The *Transaction ID* of the coherence request is noted in the table, along with the *Instruction ID* of the last instruction in this a-core to have used that memory address. One way of obtaining this information would be to add an Instruction ID field to every data cache block in the a-core and update it when the block is touched. To avoid invasive changes to the a-core, we use the following approach: whenever a coherence response is issued by the a-core, we perform an associative search in the IOT for the last instruction to have accessed that address. If found, the corresponding ID is inserted in the PTAT and the *Delay* bit is set. When the m-core completes the metadata processing for this instruction, it resets the Delay bit for the PTAT entry that matches the Instruction ID. If no instruction is found in the IOT, we conclude that the metadata processing for the last accessing instruction has already completed and there can be no problem due to interleaving memory accesses. We use a special Instruction ID value (-1) to indicate this. The m-core looks up its PTAT on external metadata requests. If there is a PTAT entry for this metadata address with the Delay bit set, the reply is delayed or NACKed, depending on the coherence protocol. Once the Delay field is reset, any metadata request to that memory address can be serviced. When a memory coherence response for a PTAT entry is finally issued, the *Done* field is set and the entry is deallocated.

The PTAT and PTRT only note the application's memory addresses. Translation between application and metadata addresses is done by the m-core. This solution is agnostic of mapping between application data and metadata allowing for fixed [88], or variable address mapping schemes [13].

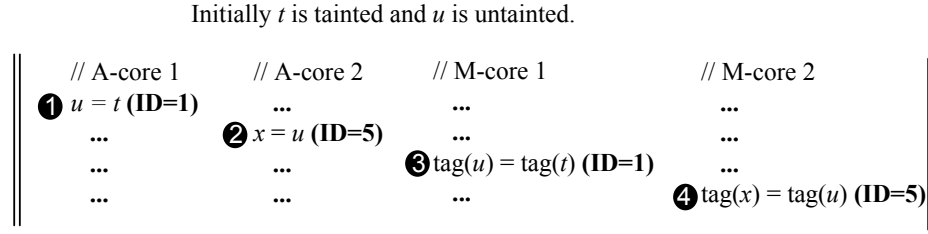


Figure 6.4: Good ordering of metadata accesses.

6.2.3 Example

We now consider how consistency is maintained for the code fragment in Figure 6.4. Figure 6.5 shows the state of the system at different times. For clarity, we only show the PTAT of the responder, and the PTRT of the requestor.

After steps ❶ and ❷ in Figure 6.4, the PTRT of m-core 2 and PTAT of m-core 1 are populated with the information for the data request and response for u as shown in Figure 6.5(a). The two IOTs are also populated with the first two instructions. Note that the pending operation in m-core 1 corresponds to the instruction that updates u .

At step ❸ in Figure 6.4, m-core 1 finishes the metadata processing for ID=1 and resets the Delay bit in the corresponding PTAT entry as shown in Figure 6.5(b). While executing step ❹ in Figure 6.4, m-core 2 experiences a miss on u 's metadata as it analyzes instruction ID=5. Before it issues its request, it finds a PTRT entry for this ID. Hence, the metadata request is sent to m-core 1, since it was a-core 1 that replied to the data request for u by a-core 2. The metadata request uses the Transaction ID associated with the PTRT entry. M-core 1 receives the metadata request and looks up its PTAT. It finds the entry with the proper Transaction ID and finds the corresponding Delay field to be reset. Hence, m-core 1 can reply with the metadata in its cache and deallocate the PTAT entry as shown in Figure 6.5(c).

Now, assume m-core 2 were to issue the metadata request for u for ID=5 before m-core

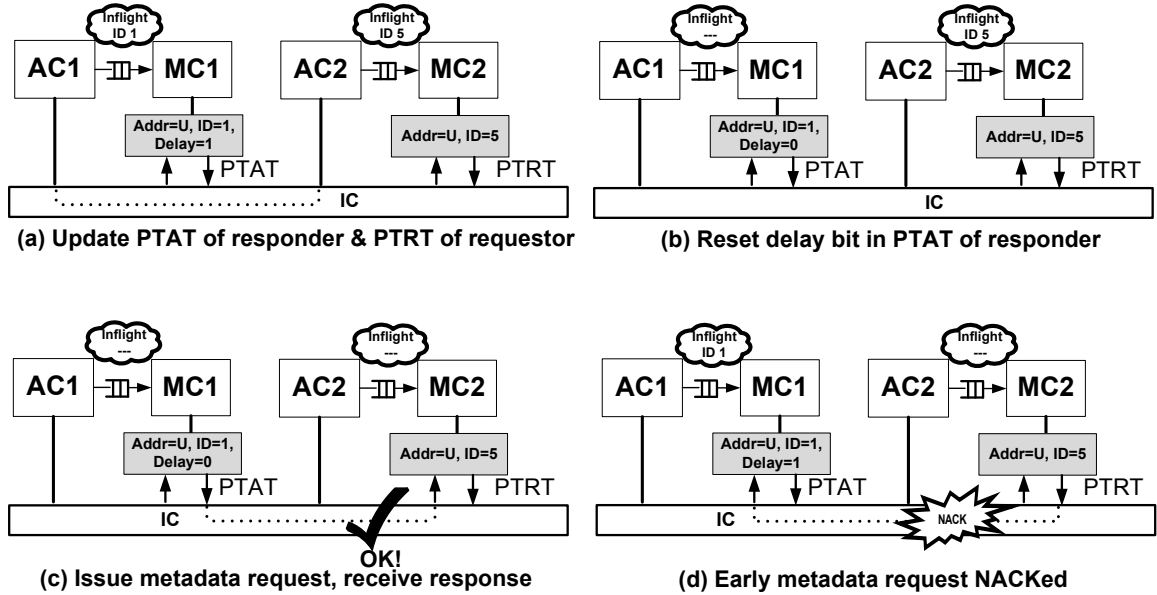


Figure 6.5: Graphical representation of the protocol. AC stands for a-core, MC for m-core, and IC for Interconnect. Addr refers to the variable's memory address.

1 had completed processing ID=1 (as shown in Figure 6.1). M-core 2 would still forward the request to m-core 1 after the PTRT lookup. M-core 1 would find the Delay bit set in the corresponding PTAT entry. The metadata request from m-core 2 would be stalled or NACKed as shown in Figure 6.5(d).

6.2.4 Performance issues

PTAT options: The simplest way to ensure consistency is by having each m-core respond to metadata requests in the same order in which data requests appear in the PTAT. Treating the PTAT as a FIFO could impact performance since coherence requests are occasionally stalled in the interconnect waiting for earlier, unrelated requests to be serviced. While the FIFO scheme works well for most cases, its pathologies warrant a discussion of further approaches.

Treat PTAT as set of FIFOs: We can allow each m-core to respond to metadata requests out of order if they refer to cache blocks different from those referred to by older entries in the PTAT. Thus, the PTAT is conceptually treated as a set of FIFOs, one for each cache block address. This implies a monolithic PTAT structure should be able to support an associative lookup on the address field.

Serve PTAT requests out of order: We can also serve metadata requests completely out-of-order (i.e., as soon as the corresponding PTAT entry has the Delay bit reset). For this purpose, we will need an additional field in each PTAT entry (*Tag Value*) to implement version management on the metadata. This field keeps a copy of the metadata produced through the analysis of the instruction with the corresponding Instruction ID until the matching metadata request is received. This allows metadata requests to be serviced out-of-order, and not stall until all previous requests are received. This approach is practical if the metadata field is short so that versioning is not particularly expensive.

While this method provides the requesting m-core with the correct metadata value, the metadata block in the corresponding m-core could be stale, i.e. not have the right cache coherence bits set. Consider the example of two successive metadata stores, and an intervening load request from another m-core. While the load still gets the right value of metadata, the cache block itself now has a new value, rendering the first version of the metadata block stale. The m-core requesting the metadata would thus not be able to cache the block.

There are two solutions to this issue. One is to shift the onus to software. The hardware would guarantee the metadata to be correct on the first access. The analysis would then be responsible for copying it or caching it if subsequent accesses are possible. An alternate solution is to leverage the fact that the problem of invalid cache blocks is true only for inflight instructions. Thus, it is possible to add a field to IOT entries that stores the invalid cache block obtained from the PTAT. This block can then be used to service any inflight requests to the tag, without causing cache pollution.

Sizing of the hardware tables: The sizes of the hardware tables directly impacts performance. The IOT provides decoupling between the a-core and the m-core, leading to a-core stalls when it is full. The issue of analysis decoupling is studied in [13, 42]. The two new tables needed for consistency enforcement, PTRT and PTAT, also stall the a-core when they are full. However, since the tables track coherence requests and replies, their size is proportional to the number of pending misses which is rather small for most core designs. In Section 6.4.2 we show that even as few as five entries are sufficient to minimize performance overheads, both when the m-core is an attached coprocessor (10s of cycles of decoupling from the a-core) or a separate core (100s of cycles of decoupling).

6.3 Practicality and Applicability

6.3.1 Coherence protocol

The proposed solution is agnostic of the protocol for cache-coherence. The PTRT and PTAT entries are updated when there is a response to a coherence request for data in the requesting and responding cores respectively. As long as we can monitor the coherence requests and responses issued by an a-core, the scheme is equally applicable to snooping and directory-based coherence. If the m-core is an attached coprocessor, the information for the PTRT and PTAT updates can be sent over a coprocessor interface. If the m-core is a general-purpose core, the update information can either be sent to the m-core through special messages on a general interconnect, or by having the m-core snoop the a-core requests on a snooping network. The protocol is also agnostic of the choice of cores: in-order or out-of-order, as it only relies on tracking coherence traffic between cores.

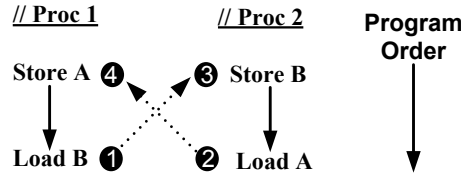


Figure 6.6: Deadlock scenario with the TSO consistency model.

6.3.2 Memory consistency model

Similar to deterministic replay schemes [92], our protocol tracks coherence traffic to determine orderings for accesses to data and replays the same order on the metadata. Hence, it works well with sequential consistency. However, it is known that these schemes can be susceptible to deadlocks under weaker consistency models used in many commercial architectures (e.g., x86 and SPARC) [92]. For instance, the SPARC Total Store Order (TSO) model allows loads to bypass unrelated stores and get their values from either memory, or a write-buffer. For the code in Figure 6.6, it is possible for both loads to be ordered at memory prior to their preceding stores. Note that instructions still commit in program order, but can be ordered at memory out of order. Thus, from the point of view of the memory model, we have $\textcircled{1} \rightarrow \textcircled{3}$ and $\textcircled{2} \rightarrow \textcircled{4}$, where $\rightarrow$ denotes a happens-before relation. For deterministic replay systems, this code can cause a deadlock during replay, due to the cycle of dependences [92].

This is because schemes such as RTR that are based on deterministic replay, merely log the coherence actions, and try and replay them in the same order [92]. If the replayer follows the sequentially consistent memory ordering, then it would try and issue $\textcircled{4}$ before $\textcircled{1}$, and $\textcircled{3}$ before $\textcircled{2}$. This would cause a deadlock due to a cycle of dependencies. There have been mechanisms proposed to convert these dependencies into artificial write-dependencies to circumvent this problem. The hardware and software support required for this, however, is significant [92].

In our solution, this is not an issue with loads that are ordered before stores and get their values from memory. The Tag Value field in the PTAT provides version management of tag values, allowing for PTAT entries to be processed out of order (as in Section 6.2.4). Thus, the m-core servicing requests can process ③ and ④ even if they are ordered first at memory during replay. The subsequent loads (① and ②) get their correct tag values from the source m-core’s PTAT. Thus, a ①→③ ordering is not imposed on the metadata.

Loads that return values from the a-core’s write buffers pose a more subtle problem. These loads are not observed by the interconnect, and do not have entries in the PTAT. Thus, the previous scheme does not work. Since the a-core commits and orders ① at memory before ④, there is already an entry for ① behind ④ in the IOT by the time ④ is ordered at memory. At this time, while allocating ④’s PTRT entry, we add a field with the ID of the youngest instruction in the IOT behind it (note that the IOT is populated when the instruction commits, in program order). This gives a list of loads that have committed behind ④, but have been ordered at memory before it. A TSO-compliant m-core can use this to order its metadata memory operations correctly. This argument can be extended to other consistency models that relax the write→read ordering, such as processor consistency on the *x86*.

6.3.3 Metadata length

Different dynamic analysis scenarios require different metadata lengths. The consistency protocol must be portable and able to accommodate the various lengths used.

Short metadata: The metadata is often much shorter than the actual data. Raksha, for example, associates a 4-bit tag with every 32-bit word of data [24]. Thus, the access to a single 4-byte word of metadata might stem from 8 different 4-byte words of the application. Since we track coherence events to enforce consistency, we enforce orderings at cache block granularity. Accesses to different data cache blocks result in accesses to different

metadata words, and thus short tags do not cause correctness problems for our protocol.

On the other hand, short tags can cause a performance problem. Since the metadata that correspond to multiple data cache blocks are packed in a single block, the m-cores can experience higher miss rates than the a-cores due to false sharing. This issue is explored further in Section 6.4.3.

Long metadata: Some analyses require metadata that are longer than the actual data. For instance, the Lockset analysis used by LBA maintains a sorted list of lock addresses for each lock [13]. Thus, each data update corresponds to an update of multiple words of metadata. This creates the following problem: metadata may span multiple cache blocks (or even pages) leading to non-atomic transfers of metadata between m-core caches as the coherence system handles each block separately.

In the analysis architectures proposed thus far, long metadata are always handled in software using short routines with a few instructions [13]. This makes it expensive to handle the atomicity problem for long metadata using software locks. The analysis programmer can potentially avoid using a lock unless the metadata actually spans across multiple cache blocks. Nevertheless, this makes the analysis code architecture-dependent and difficult to write. A better solution is to use Read-Copy-Update (RCU) for metadata. Anytime an analysis routine needs to update long metadata, it creates a copy of the current value and updates the new version. The old metadata is then garbage-collected once its users relinquish hold over it. RCU eliminates the need for software locks in analysis code and the related issues (overhead, deadlocks, etc.). The only change needed in our hardware protocol to work with the RCU approach is the following. Instead of versioning the actual metadata values in the Tag Value field of PTAT entries, we pass a pointer to the active metadata copy. The hardware protocol itself has no other correctness issues.

If RCU is used, garbage collection of the old metadata can be performed by maintaining reference counts in software [59]. Reference counts for each version of metadata are incremented when processors enter the analysis routine, and are decremented when they

exit. When no processor is actively using a version of metadata (its reference count reaches zero), it can be garbage collected by software.

6.3.4 Analysis issues

In some cases, the analysis routine performs different operations on the metadata than those performed on the corresponding data. For example, an analysis might maintain a counter in the metadata that gets incremented every time a variable is accessed. This implies that a-core data reads may trigger m-core writes to the corresponding metadata. Our protocol for *(data, metadata)* consistency, however, relies on coherence activity. Thus, if an a-core read on shared data gets translated into a metadata write, it is not always clear as to which m-core should be able to perform the write first. This could cause consistency issues due to metadata writes being performed out of order. In reality, this is not a major issue because the proposed analyses that convert a-core reads to m-core writes, perform commutative operations on the metadata. Counter increments and lockset updates [13] are commutative operations, and thus the order in which the updates happen does not affect the final value.

To support analyses where data reads lead to non-commutative metadata updates, our protocol must track read accesses to shared data in the PTAT and PTRT structures so that the order can be replayed for metadata operations. Hence, reads to shared data must now be visible on the coherence protocol which is not the case for MESI or MOESI systems (multiple cores can have a copy of the same data in S state and thus, no coherence traffic occurs on reads). A solution would be similar to the scheme by Suh *et al.* [82], where the authors explain how to implement a MEI coherence scheme on top of MESI or MOESI coherence in order to gain visibility into reads for shared data. Note that the overhead of a MEI protocol would only be paid when such an analysis is actually performed.

Feature	Description
Processors	2 to 32 x86 cores, in-order, single issue
Simulator	TCC x86 simulator [34] + Wisconsin GEMS [58]
Coherence protocol	MESI Directory
Private split L1	64 KB, 4-way set assoc., 3-cycle acc. latency
Shared L2	32 MB, 4-way set assoc., 6-cycle acc. latency
Main Memory	160-cycle acc. latency
Default table sizes	20 (IOT), 10 (PTAT), 10 (PTRT) entries

Table 6.2: Simulation infrastructure and setup.

It is important to note that the evaluation presented in Section 6.4 assumes the worst-case scenario where all instructions (including those in the operating system) must be analyzed by the m-core. Developers might however choose to concentrate the analysis on a single application, in which case the hardware structures track only the instructions analyzed by the m-core. Similar to the decoupled DIFT architectures [42], system events such as context switches or interrupts do not require any special handling of the hardware structures.

6.4 Experimental Results

Table 6.2 presents the main parameters of our simulated multi-core system. We couple every application processor with a metadata processor. After the application core commits an instruction, it is passed on to the metadata core. We also modified GEMS [58] to include the previously described hardware tables (IOT, PTAT and PTRT). We simulate a two-level cache hierarchy with private, split L1 caches, and a shared, unified L2. We use a large L2 cache in all our experiments in order to decrease the number of accesses to main memory. Our goal is to study the overheads of our mechanism for maintaining (*data*, *metadata*) consistency, which is affected only by requests between processors for exclusive access or dirty data. A smaller L2 cache would cause more accesses to main memory, which would

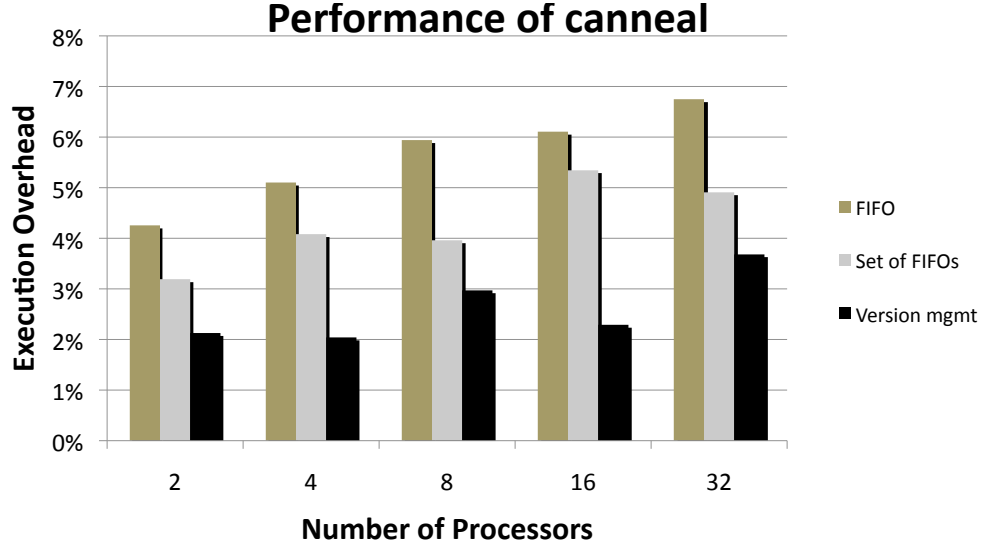


Figure 6.7: Performance of Canneal when the number of processors is scaled.

end up masking the overhead of these cache-to-cache requests and subsequent stalls. Thus, the relative overhead of the consistency mechanism would have decreased with a smaller cache size. The choice of L2 access latency was motivated by a similar desire to sensitize the experimental evaluation primarily towards the consistency mechanism.

6.4.1 Baseline execution

In order to evaluate the performance of our system, we ran a spread of unmodified benchmarks from the PARSEC [8], and SPLASH-2 [91] suites. These benchmarks were chosen to study the performance overheads of our solution over programs with differing levels of data sharing, and data exchange. These benchmarks use parallel, dependent threads. Sharing between the threads stresses the performance of our metadata consistency mechanism. We chose to not evaluate our solution with multiprogramming workloads due to the lack of races in such workloads.

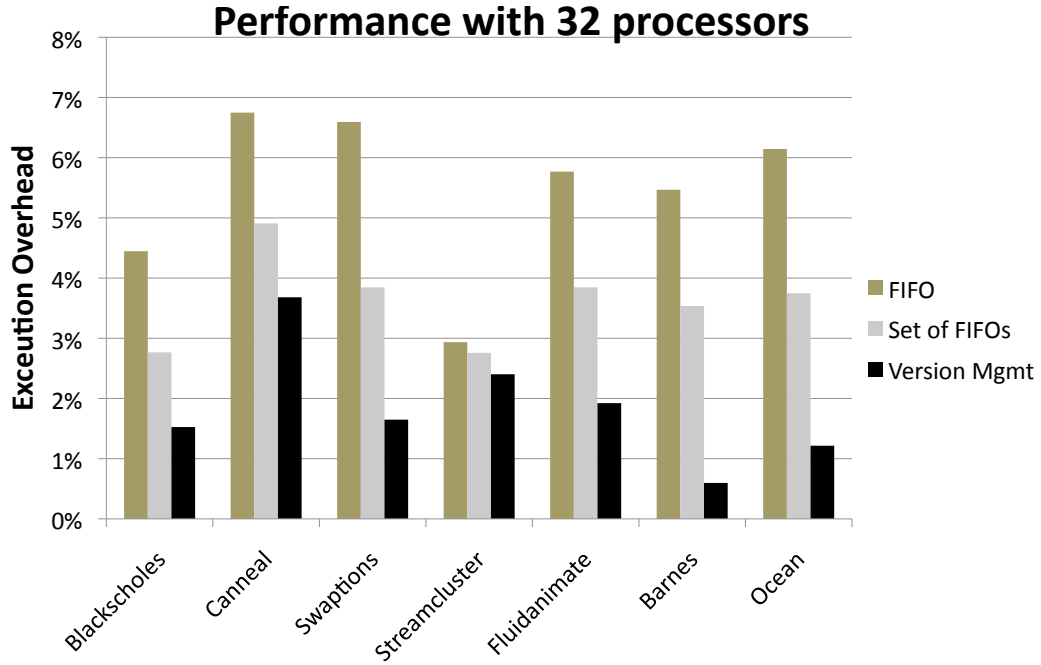


Figure 6.8: Performance of PARSEC and SPLASH-2 benchmarks with 32 processors.

We associate 32-bit tags with 32-bit application data words and perform an information flow analysis. As mentioned in Section 6.2.4, there are different PTAT designs possible, each offering different performance and price tradeoffs. In both Figures 6.7 and 6.8, we show three different configurations. We consider a configuration with no consistency mechanism between data and metadata to be our base case, and show execution overheads relative to it. The first bar represents the case when the PTAT is treated as a FIFO. Metadata requests are processed strictly in the order in which data requests were processed. The second bar represents the case when the PTAT is treated as a set of FIFOs, one for each cache block address. Thus, requests that do not map to the same address, can be reordered at the PTAT. The third bar represents the case when all PTAT requests can be processed out of the order in which the original data requests were processed.

Figure 6.7 shows the performance of the Canneal benchmark from the PARSEC suite over a different number of processors. We use Canneal in Figure 6.7 since it requires

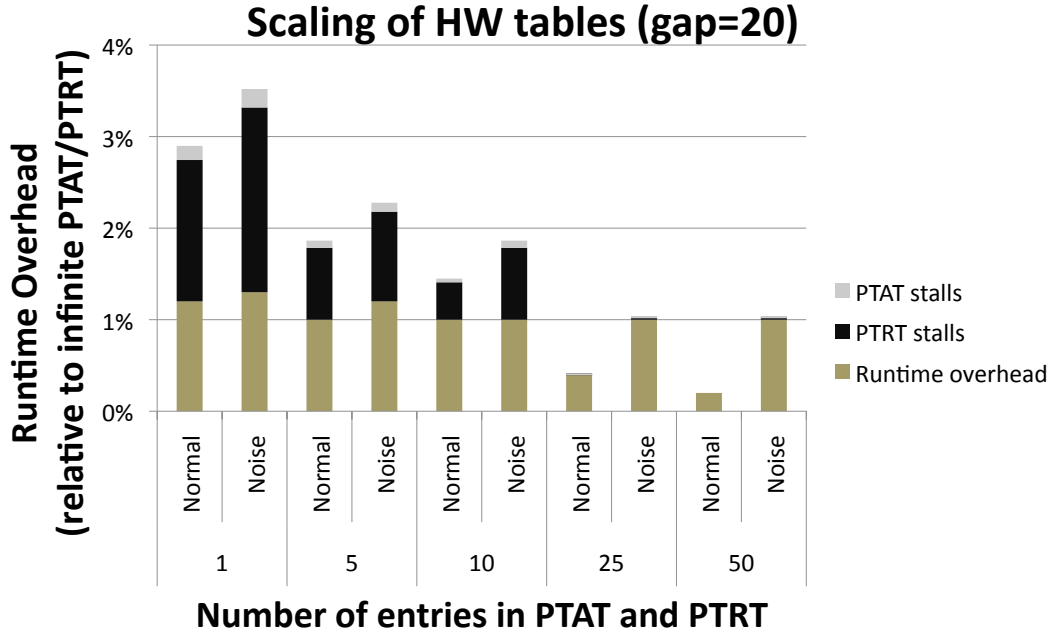


Figure 6.9: Scaling the PTAT/PTRT sizes with a small decoupling interval on a worst-case lock contention microbenchmark.

extensive fine-grained sharing and data exchange between processors [8]. As is evident from Figure 6.7, the performance overhead of the consistency scheme is low. Even with 32 processors, treating the PTAT as a FIFO still only has an overhead of 6.5%. This overhead decreases as we add sophisticated hardware support to the PTAT, increasing its cost.

In order to evaluate the worst case performance of the system, we ran our benchmark suite on 32 processors. Figure 6.8 shows the results of running the different configurations explained earlier, over this selection of benchmarks. As is evident from both Figures 6.7 and 6.8, the overheads of the synchronization scheme are low: less than 7% even when the PTAT is treated as a FIFO. This implies that even the simple FIFO design provides good performance.

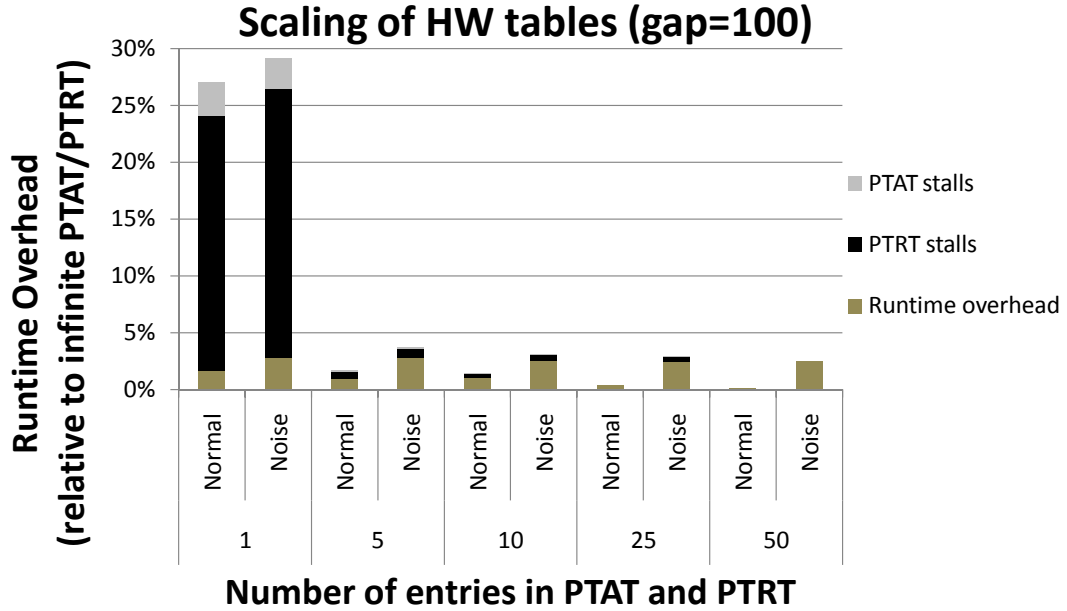


Figure 6.10: Scaling the PTAT/PTRT sizes with a large decoupling interval on a worst-case lock contention microbenchmark.

6.4.2 Scaling the hardware structures

While our solution is equally applicable to both the coprocessor [42], and LBA models [12], these architectures differ in the degree of decoupling between metadata and data processing. This requires that the hardware structures introduced by our protocol be sized accordingly.

Due to the low overheads exhibited by our benchmark suite, we wrote a microbenchmark to stress test the worst case performance due to scaling the hardware structures. This microbenchmark evaluated the performance of multiple threads competing for a shared lock and synchronizing on a barrier, over hundreds of iterations. Figures 6.9 and 6.10 plot the results of varying the sizes of the PTAT and the PTRT, for these different degrees of decoupling, mimicking the coprocessor and log-based models respectively. Figure 6.9 has a short decoupling interval of 20 cycles between metadata and data instructions. Figure 6.10 uses a larger decoupling interval of 100 cycles. In order to account for uncertainties in

the interconnection network, we also randomly introduced some noise: an extra delay of 10 cycles between data and metadata processing. Results are plotted relative to a system with an infinitely sized PTAT and PTRT, and no additional noise. We use a system with 32 processors in this experiment, and used the FIFO configuration for the PTAT. We show the overheads due to stalls in the PTAT and PTRT, and also the runtime overhead due to m-core requests being NACKed. This last bar represents the cases where we have to restore correct ordering of requests.

As can be seen from Figure 6.9, even a single entry PTAT/ PTRT combination is enough for good performance even in the presence of noise, since the overhead is less than 4%. The low degree of decoupling, however ensures that there are only a few outstanding requests at any given time. Thus, even PTATs and PTRTs with five entries are sufficient to provide good performance. A larger degree of decoupling introduces additional outstanding requests as evinced by Figure 6.10. The overhead of the single entry PTAT/PTRT combination increases to as much as 29% (with the addition of noise). Larger structures however reduce the overheads to around 5%. The size of the PTAT and PTRT structures directly relates to the hardware cost of the system. These results show that small structures (few tens of entries) suffice to both provide good performance, and reduce the hardware cost.

6.4.3 Smaller tags

As explained in Section 6.3.3, metadata is often of a smaller size than the data itself. Most DIFT architectures such as Raksha, MINOS, etc., associate a 4-bit tag with every 32-bit word of data. Thus, if metadata is stored contiguously, a single cache-block of metadata could have accesses stemming from different cache-blocks of application data. While this reduces the storage overhead of metadata, it could introduce additional traffic in the system due to false sharing. One possible way of addressing this problem is to map each metadata word to a separate cache block, or use smaller cache-blocks on the metadata processor.

Overhead of 4-bit tags

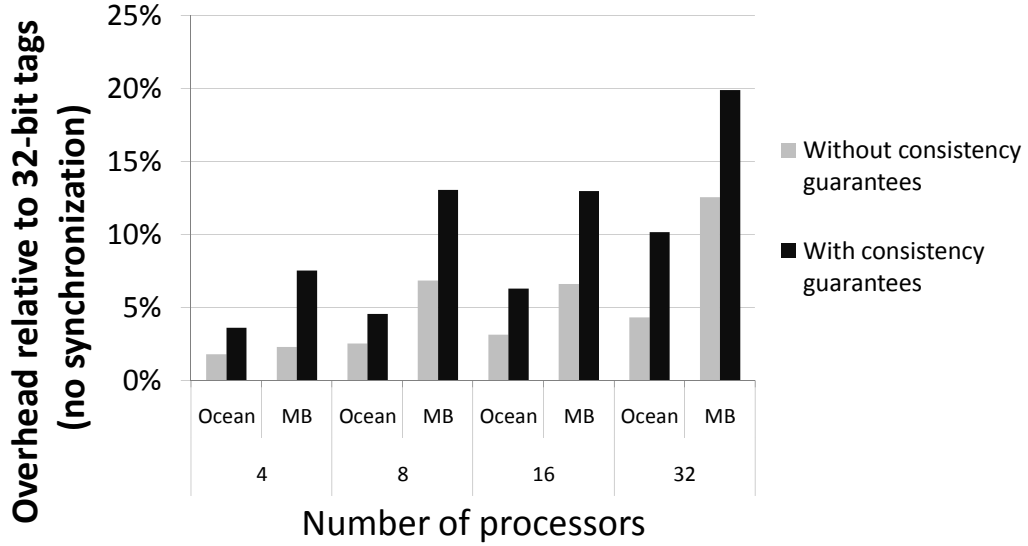


Figure 6.11: The overheads of using smaller tags on Ocean, and a heap traversal microbenchmark (MB).

While this would solve the problem of false sharing, it would also negate the positive effects of larger cache blocks, such as added spatial locality.

We studied the impact of false sharing on the Ocean benchmark from the SPLASH-2 suite, when the FIFO configuration for the PTAT is used. Ocean has the highest percentage of shared writes among our benchmarks [7] and is thus the most sensitive to false sharing. We also wrote a microbenchmark to stress test the worst possible scenario. The microbenchmark implemented a multi-threaded binary heap traversal, with the heap stored as a contiguous array. Each access of the array, required the thread to contend for the lock on the root of the array, and move outwards acquiring locks on children nodes. We used a 4-bit tag for every 32-bit word, and 64-byte cache blocks.

Figure 6.11 shows the overheads due to small tags on Ocean and our microbenchmark. All numbers are normalized to the base case of running the workload with 32-bit tags for

every 32-bit word, without providing any (*data*, *metadata*) consistency guarantees. The first set of numbers indicates the overhead of merely using smaller tags (without any consistency guarantees), and quantifies the impact of false sharing. The second set of numbers shows the overhead of using smaller tags, and providing (*data*, *metadata*) consistency guarantees using our hardware solution. As can be seen from the figure, the overhead of using smaller tags is 10% for Ocean, and less than 20% for the worst case microbenchmark, when 32 processors are used.

6.5 Summary

This chapter presented a practical, fast hardware solution for correct execution of dynamic analysis on multithreaded programs. We leverage cache coherence to record the interleaving of memory operations from application threads, and replay the same order on metadata processors, thereby maintaining consistency between data and metadata. We add hardware tables accessible by the analysis cores and coherence fabric that record the application's coherence messages, and enforce the same ordering on the metadata threads. This mechanism does not require any changes to the main cores and caches, and is applicable to both sequentially consistent, and relaxed memory consistency models. Our experiments showed that the overhead of this approach was less than 7% with 32 processors, over a suite of PARSEC and SPLASH-2 benchmarks.

In effect, this scheme provides the last piece of the DIFT puzzle. We have discussed how to provide low-overhead, flexible, and expressive hardware support for DIFT in Chapters 3 and 4, how to lower the cost of providing DIFT support in Chapter 5, and how to extend the DIFT solution to be compliant with multi-threaded programs. In the following chapter, we discuss another security analysis that makes use of hardware tags.

Chapter 7

Enforcing Application Security Policies using Tags

Thus far, we have studied the development of hardware architectures for DIFT. The underlying tagged memory abstraction used by DIFT architectures is very powerful, and can be used to solve other security problems. In this chapter we look at one such technique, known as *Dynamic Information Flow Control* (or DIFC) that can benefit from another flavor of tagged memory. DIFC is a security technique that prevents potentially malicious applications from disclosing or modifying sensitive data, without correct authorization. This security mechanism associates a tag or a label at the granularity of operating system processes. This label is indicative of the data that the process has access to, and regulates the flow of information in the system, i.e. a process labeled untrusted will be prevented from accessing data belonging to a process labeled sensitive. Unlike DIFT, DIFC does not assume that applications are non-malicious. While DIFT is concerned with validating untrusted input to non-malicious applications, DIFC helps maintain security guarantees and protects the system even in the face of compromised, or malicious applications.

In this chapter, we show how hardware mechanisms similar to those introduced in the previous chapters can be used by DIFC systems. The use of hardware tags allows for DIFC

policy enforcement to be done at the lowest level of the system, the hardware, thereby ensuring the security of the system even in the face of a compromised operating system. The rest of the chapter is structured as follows. Section 7.1 motivates the use of information flow control for direct enforcement of application security policies. Section 7.2 describes the hardware requirements for an information flow control system in more detail, and Section 7.3 describes our overall system architecture and its security goals, as well as our experimental prototype. Section 7.4 describes the tagged memory processor we developed as part of this work. Section 7.5 presents an evaluation of the security and performance of our prototype, Section 7.6 discusses related work, and Section 7.7 concludes.

7.1 Motivation

A significant part of the computer security problem stems from the fact that the security of large-scale applications usually depends on millions of lines of code behaving correctly, rendering security guarantees all but impossible. One way to improve security is to separate the enforcement of security policies into a small, trusted component, typically called the trusted computing base [48], which can then ensure security even if the other components are compromised. This usually means enforcing security policies at a lower level in the system, such as in the operating system or in hardware. Unfortunately, enforcing application security policies at a lower level is made difficult by the *semantic gap* between different layers of abstraction in a system. Since the interface traditionally provided by the OS kernel or by hardware is not expressive enough to capture the high-level semantics of application security policies, applications resort to building their own ad-hoc security mechanisms. Such mechanisms are often poorly designed and implemented, leading to an endless stream of compromises [72].

As an example, consider a web application such as Facebook or MySpace, where the web server stores personal profile information for millions of users. The application's

security policy requires that one user's profile can be sent only to web browsers belonging to the friends of that user. Traditional low-level protection mechanisms, such as Unix's user accounts or hardware's page tables, are of little help in enforcing this policy, since they were designed with other policies in mind. In particular, Unix accounts can be used by a system administrator to manage different users on a single machine; Unix processes can be used to provide isolation; and page tables can help in protecting the kernel from application code. However, enforcing or even expressing our example website's high-level application security policy using these mechanisms is at best difficult and error-prone [45]. Instead, such policies are usually enforced throughout the application code, effectively making the entire application part of the trusted computing base.

A promising technique for bridging this semantic gap between security mechanisms at different abstraction layers is to think of security in terms of what can happen to data, instead of specifying the individual operations that can be invoked at any particular layer (such as system calls). For instance, recent work on operating systems [30, 46, 94, 95] has shown that many application security policies can be expressed as restrictions on the movement of data in a system, and that these security policies can then be enforced using an information flow control mechanism in the OS kernel.

This chapter shows that hardware support for tagged memory allows enforcing data security policies at an even lower level—directly in the processor—thereby providing application security guarantees even if the kernel is compromised. To support this claim, we designed Loki, a hardware architecture that provides a word-level memory tagging mechanism, and ported the HiStar operating system [94] (which was designed to enforce application security policies in a small trusted kernel) to run on Loki. Loki's tagged memory simplifies security enforcement by associating security policies with data at the lowest level in the system—in physical memory. The resulting simplicity is evidenced by the fact that the port of HiStar to Loki has less than half the amount of trusted code than HiStar

running on traditional CPUs. Finally, we show that tagged memory can achieve strong security guarantees at a minimal performance cost, by building and evaluating a full system prototype of Loki running HiStar.

7.2 Requirements for Dynamic Information Flow Control Systems

Dynamic Information Flow Control, similar to DIFT, can be implemented wholly in hardware or software. The tradeoffs between the two approaches too, are similar to those discussed earlier in the context of DIFT in Section 2.2. Implementing DIFC wholly in software in a binary translator incurs extremely high performance overheads. Since DIFC is applied on operating system processes as well, the overheads would be far worse than those observed by systems performing DIFT on user-level applications. Leveraging hardware support for maintaining metadata, and checking access control violations reduces this overhead drastically, and helps make this technique practically viable. Similar to DIFT, DIFC systems require the ability to specify and manage security policies in software, in order to be flexible, and easily adapt and extend the protection mechanisms. Thus, we make the case for DIFC systems to use hardware to maintain metadata that serves to encode information flow control restrictions, and software to manage these security policies.

7.2.1 Tag management

Metadata, or information about the DIFC analysis is maintained in hardware in *tags*. Tags in DIFC convey a very different meaning from those used in DIFT solutions. In DIFT, a tag bit is used to implement a unique security policy. A tag value of one usually indicates that the associated data is tainted (for a taint analysis, say), and a tag check of that bit would potentially raise a security trap. In contrast, tag values in DIFC map to access-permissions

on the associated data. Every process has an associated *label* that places restrictions on the other processes it can communicate with. These labels are maintained in software and can be arbitrarily complex. Labels are mapped to a fixed-width tag that is stored with every memory word. This tag in turn must be used to index a lookup-table, or a *permissions table* to obtain the relevant memory access permissions (read/write/execute).

Both DIFC and DIFT systems associate tags with every word of memory. Similar to DIFT, DIFC systems also exhibit significant spatial locality in tags, and can thus use a multi-granular tag storage scheme. In this approach, tags can be maintained at the granularity of every page of memory, and in the case finer grain tags are needed, at the granularity of every word of memory.

7.2.2 Tag manipulation

Dynamic Information Flow Control is concerned with restricting, rather than tracking the flow of information. Thus, DIFC does not require tag propagation. Tags are initialized by a software routine, and remain immutable until explicitly modified by software. DIFC does, however, require tag checks on every instruction. Tag checks in DIFC require an instruction to index the permissions table with its tag, and check if the associated access permissions are valid. Similar to DIFC systems, both instructions and data have tags. Thus, every instruction must access the permissions table once at the minimum. Instructions that access memory must access the permissions table a second time, with the data-memory tag.

7.2.3 Security exceptions

When a tag check fails, the system generates a security exception. This transitions control to a *security monitor* that is responsible for performing any associated analysis. Similar to DIFT systems, the monitor is also responsible for configuring the security policies. Specifically, the monitor is responsible for managing the mapping between software labels and

hardware tags, and maintaining correct access permissions. The monitor runs in a separate operating mode, outside of the operating system. Thus, the monitor’s security policies cannot be subverted in the face of a compromised operating system.

7.3 System Architecture

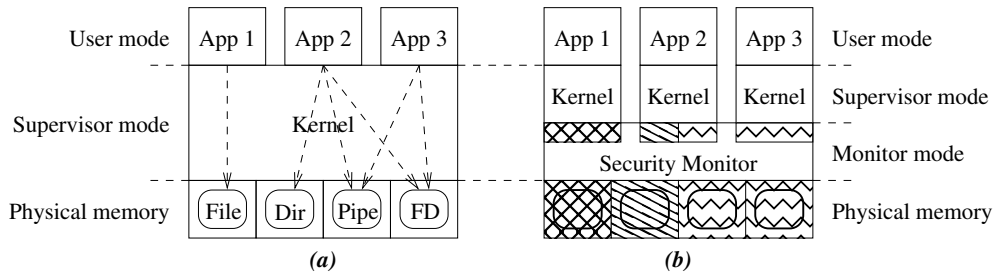


Figure 7.1: A comparison between (a) traditional operating system structure, and (b) this chapter’s proposed structure using a security monitor. Horizontal separation between application boxes in (a), and between stacks of applications and kernels in (b), indicates different protection domains. Dashed arrows in (a) indicate access rights of applications to pages of memory. Shading in (b) indicates tag values, with small shaded boxes underneath protection domains indicating the set of tags accessible to that protection domain.

This section describes a combination of a new hardware architecture, called Loki, that enforces security policies in hardware by using tagged memory, together with a modified version of the HiStar operating system [94], called LoStar, that enforces discretionary access components of its information flow policies using Loki [96]. The overall structure of this system is shown in Figure 7.1.

Traditional OS kernels, shown in Figure 7.1 (a), are tasked with both implementing abstractions seen by user-level code as well as controlling access to data stored in these abstractions. LoStar, shown in Figure 7.1 (b), separates these two functions by using hardware to control data access. In particular, the Loki hardware architecture associates *tags* with words of memory, and allows specifying protection domains in terms of the tags that can be accessed. LoStar manages these tags and protection domains from a small software

component, called the *security monitor*, which runs underneath the kernel in a special processor privilege mode called *monitor mode*. The security monitor translates application security policies on data, specified in terms of *labels* on kernel objects in the HiStar operating system, into tags on the corresponding physical memory, which the hardware then enforces.

Most systems enforce security policies in hardware through a translation mechanism, such as paging or segmentation. However, enforcing security in a translation mechanism means that security policies are bound to virtual resources, and not to the actual physical memory storing the data being protected. As a result, the policy for a particular piece of data in memory is not well-defined in hardware, and instead depends on various invariants being implemented correctly in software, such as the absence of aliasing. Tagging physical memory helps bridge the semantic gap between the data and its security policy, and makes the security policy unambiguous even at a low level, while requiring a much smaller trusted code base.

As mentioned previously, tagged memory alone is not sufficient for enforcing strict information flow control, because dynamic allocation of resources with fixed names, such as physical memory, contains inherent covert channels. For example, a malicious process with access to a secret bit of data could signal that bit to a colluding non-secret process on the same machine by allocating many physical memory pages and freeing only the odd- or even-numbered pages depending on the bit value. Operating systems like HiStar solve such problems by virtualizing resource names (e.g. using kernel object IDs) and making sure that these virtual names are never reused. However, the additional kernel complexity can lead to bugs far worse than the covert channels the added code was trying to fix. Moreover, implementing equivalent functionality in hardware would not be inherently any simpler than the OS kernel code it would be replacing, and would not necessarily improve security.

What hardware support for tagged memory can address, however, is the the tension between stronger security and increased complexity seen in an OS kernel. In particular,

hardware can provide a new, intermediate level of security, which can enforce a subset of the kernel's security guarantees, as illustrated by our hybrid threat model in Figure 7.2 [96]. In the simplest case, we are concerned with two security levels, *high* and *low*, and the goal is ensuring that data from the high level cannot influence data in the low level. There are multiple interpretations of high and low. For instance, high might represent secret user data, in which case low would be world-readable, as in [4]. Alternatively, low could represent high-integrity system configuration files, which should not be affected by high user inputs, as in [6].

The hybrid model provides a different enforcement of our security goal under different assumptions. In particular, the weaker *discretionary access control* model, enforced by the tagging hardware and the security monitor, disallows both high processes from modifying low data and low processes from reading high data. However, if a malicious pair of high and low processes collude, they can exploit covert channels to subvert our security goal, as shown by the dashed arrow in Figure 7.2. The stronger *mandatory access control* model aims to prevent such covert communication, by providing a carefully designed kernel interface, like the one in HiStar, in a more complex OS kernel. The resulting hybrid model can enforce security largely in hardware in the case of only one malicious or compromised process, and relies on the more complex OS kernel when there are multiple malicious processes that are colluding.

The rest of this section will first describe LoStar from the point of view of different applications, illustrating the security guarantees provided by different parts of the operating system. We will then provide an overview of the Loki hardware architecture, and discuss how the LoStar operating system interacts with Loki's hardware mechanisms.

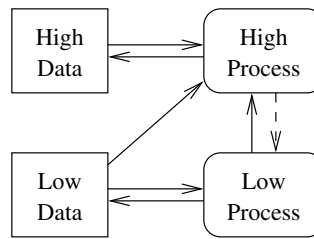


Figure 7.2: A comparison of the *discretionary access control* and *mandatory access control* threat models. Rectangles represent data, such as files, and rounded rectangles represent processes. Arrows indicate permitted information flow to or from a process. A dashed arrow indicates information flow permitted by the discretionary model but prohibited by the mandatory model.

7.3.1 Application perspective

One example of an application in LoStar is the Unix environment itself. HiStar implements Unix in a user-space library, which in turn uses HiStar’s kernel labels to implement its protection, such as the isolation of a process’s address space, file descriptor sharing, and file system access control. As a result, unmodified Unix applications running on LoStar do not need to explicitly specify labels for any of their objects. The Unix library automatically specifies labels that mimic the security policies an application would expect on a traditional Unix system. However, even the Unix library is not aware of the translation between labels and tags being done by the kernel and the security monitor. Instead, the kernel automatically passes the label for each kernel object to the underlying security monitor.

LoStar’s security monitor, in turn, translates these labels into tags on the physical memory containing the respective data. As a result, Loki’s tagged memory mechanism can directly enforce Unix’s discretionary security policies without trusting the kernel. For example, a page of memory representing a file descriptor is tagged in a way that makes it accessible only to the processes that have been granted access to that file descriptor. Similarly, the private memory of a process’s address space can be tagged to ensure that only threads within that particular process can access that memory. Finally, Unix user IDs are also mapped to labels, which are then translated into tags and enforced using the same

hardware mechanism.

An example of an application that relies on both discretionary and mandatory access control is the HiStar web server [95]. Unlike other Unix applications, which rely on the Unix library to automatically specify all labels for them, the web server explicitly specifies a different label for each user's data, to ensure that user data remains private even when handled by malicious web applications. In this case, if an attacker cannot compromise the kernel, user data privacy is enforced even when users invoke malicious web applications on their data. On the other hand, if an attacker can compromise the kernel, malicious web applications can leak private data from one user to another, but only for users that invoke the malicious code. Users that don't invoke the malicious code will still be secure, as the security monitor will not allow malicious kernel code to access arbitrary user data.

7.3.2 Hardware overview

The design of the Loki hardware architecture was driven by three main requirements. First, hardware should provide a large number of non-hierarchical protection domains, to be able to express application security policies that involve a large number of disjoint principals. Second, the hardware protection mechanism should protect low-level physical resources, such as physical memory or peripheral devices, in order to push enforcement of security policies to the lowest possible level. Finally, practical considerations require a fine-grained protection mechanism that can specify different permissions for different words of memory, in order to accommodate programming techniques like the use of contiguous data structures in C where different data structure members could have different security properties.

To address these requirements, Loki logically associates an opaque 32-bit *tag* with every 32-bit word of physical memory. Figure 7.3 shows the logical view of the system at the ISA level, where every register and memory location appears to be extended with a 32-bit

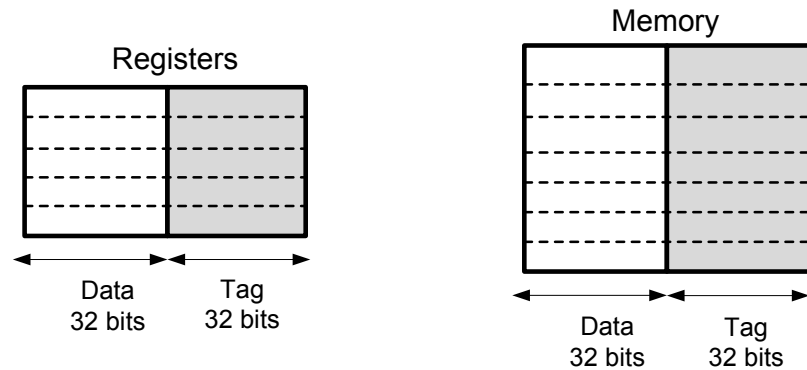


Figure 7.3: The tag abstraction exposed by the hardware to the software. At the ISA level, every register and memory location appears to be extended by 32 tag bits.

tag. Tag values correspond to a security policy on the data stored in locations with that particular tag. Protection domains in Loki are specified in terms of tags, and can be thought of as a mapping between tags and permission bits (read, write, and execute). Loki provides a software-filled *permissions cache* in the processor, holding permission bits for some set of tags accessed by the current protection domain, which is checked by the processor on every instruction fetch, load, and store.

A naive implementation of word-level tags could result in a 100% memory overhead for tag storage. To avoid this problem, Loki implements a multi-granular tagging scheme, which allows tagging an entire page of memory with a single 32-bit tag value.

Tag values and permission cache entries can only be updated in Loki while in a special processor privilege mode called *monitor mode*, which can be logically thought of as more privileged than the traditional supervisor processor mode. Hardware invokes tag handling code running in monitor mode on any tag permission check failure or permission cache miss by raising a *tag exception*. To avoid including page table handling code in the trusted computing base, the processor's MMU is disabled while executing in monitor mode.

7.3.3 OS overview

Kernel code in Loki continues to execute at the supervisor privilege level, with access to all existing privileged supervisor instructions. This includes access to traditionally privileged state, such as control registers, the MMU, page tables, and so on. However, kernel code does not have direct access to instructions that modify tags or permission cache entries. Instead, it invokes the security monitor to manage the tags and the permission cache, subject to security checks that we will describe later. By disabling the MMU on entry into monitor mode, hardware ensures that even malicious kernel code cannot compromise security policies specified by the monitor.

The kernel requires word-level tags for two main reasons. First, existing C data structures often combine data with different security requirements in contiguous memory. For example, the security label field in a kernel object should not be writable by kernel code, but the rest of the object's data can be made writable, subject to the policy specified by the security label. Word-level tagging avoids the need to split up such data structures into multiple parts according to security requirements. Second, word-level tags reduce the overhead of placing a small amount of data, such as a 32-bit pointer or a 64-bit object ID, in a unique protection domain.

Although Loki enforces memory access control, it does not guarantee liveness. All of the kernel protection domains in LoStar participate in a cooperative scheduling protocol, explicitly yielding the CPU to the next protection domain when appropriate. Buggy or malicious kernel code can perform a denial of service attack by refusing to yield, yielding only to other colluding malicious kernels, halting the processor, misconfiguring interrupts, or entering an infinite loop. Liveness guarantees can be enforced at the cost of a larger trusted monitor, which would need to manage timer interrupts, perform preemptive scheduling, and prevent processor state corruption. A more in-depth discussion of the security monitor can be found in [96].

7.4 Microarchitecture

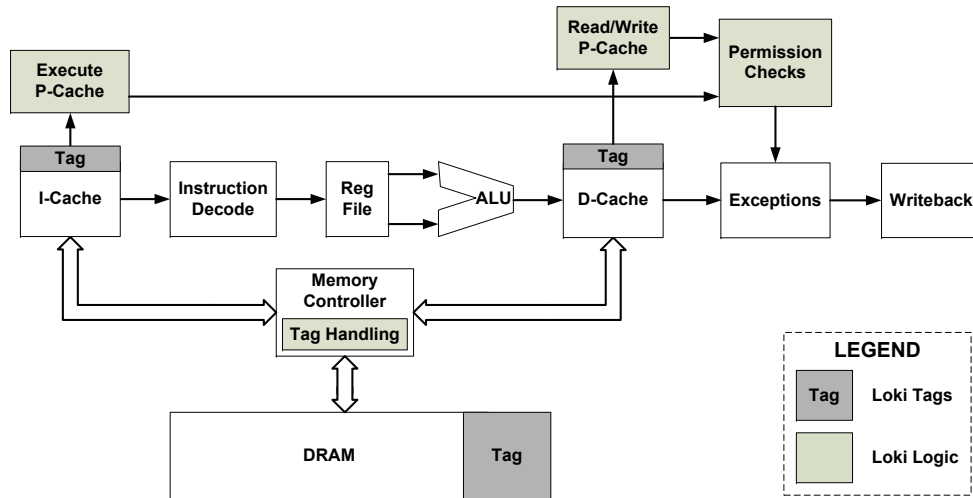


Figure 7.4: The Loki pipeline, based on a traditional pipelined SPARC processor.

Loki enables building secure systems by providing fine-grained, software-controlled permission checks and tag exceptions. This section discusses several key aspects of the Loki design and microarchitecture. Figure 7.4 shows the overall structure of the Loki pipeline.

7.4.1 Memory tagging

Loki provides memory tagging support by logically associating an opaque 32-bit tag with every 32-bit word of physical memory. Associating tags with physical memory, as opposed to virtual addresses, avoids potential aliasing and translation issues in the security monitor. Tags are used to specify security policies for different variables, objects, or data structures, as mandated by the monitor. The monitor then specifies access permissions in terms of these tag values. These tags are cacheable, similar to data, and have identical locality. Special instructions are provided to read and write these memory tags, and only trusted code executing in the monitor mode may execute these instructions.

When a context switch to a process occurs, the monitor populates the permission cache with the access rights of the new protection domain. Only trusted code executing in the monitor mode may execute the special instructions that initialize permissions. The monitor protects itself from the kernel and applications by tagging all monitor memory with a special tag value which no one else can access.

7.4.2 Granularity of tags

System designers must balance the number of concurrently active security policies and tag granularity with the storage overhead of tags and the permission cache. Naively associating a 32-bit tag value with each 32-bit physical memory location would not only double the amount of physical memory, but also impact runtime performance. Setting tag values for large ranges of memory would be prohibitively expensive if it required manually updating a separate tag for each word of memory. Since tags tend to exhibit high spatial locality [81], our design adopts a *multi-granular* tag storage approach in which page-level tags are stored in a linear array in physical memory, called the *page-tag array*, allocated by the monitor code. This array is indexed by the physical page number to obtain the 32-bit tag for that page. These tags are cached in a structure similar to a TLB for performance. Note that this is different from previous work where page-level tags are stored in the TLBs and page tables [81]. Since we do not make any assumptions about the correctness of the MMU code, we must maintain our tags in a separate structure. The monitor can specify fine-grained tags for a page of memory on demand, by allocating a shadow memory page to hold a 32-bit tag for every 32-bit word of data in the original page, and putting the physical address of the shadow page in the appropriate entry in the linear array, along with a bit to indicate an indirect entry. The benefit of this approach is that DRAM need not be modified to store tags, and the tag storage overhead is proportional to the use of fine-grained tags.

7.4.3 Permissions cache

Fine-grained permission checks are enforced in hardware using a *permission cache*, or P-cache. The P-cache stores a set of tag values, along with a 3-bit vector of permissions (read, write, and execute) for each of those tag values, which represent the privileges of the currently executing code. Each memory access (load, store, or instruction fetch) checks that the accessed memory location's tag value is present in the P-cache and that the appropriate permission bit is set.

The P-cache is indexed by the least significant bits of the tag. A P-cache entry stores the upper bits of the tag and its 3-bit permission vector. The monitor handles P-cache misses by filling it in as required, similar in spirit to a software-managed TLB. All known TLB optimization techniques apply to the P-cache design as well, such as multi-level caches, separate caches for instruction and data accesses, hardware assisted fills, and so on.

The size of the P-cache, and the width of the tags used, are two important hardware parameters in the Loki architecture that impact the design and performance of software. The size of the P-cache affects system performance, and effectively limits the working set size of application and kernel code in terms of how many different tags are being accessed at the same time. Applications that access more tags than the P-cache can hold will incur frequent exceptions invoking the monitor code to refill the P-cache. However, the total number of security policies specified in hardware is not limited by the size of the P-cache, but by the width of the tag. In our experience, 32-bit tags provide both a sufficient number of tag values, and sufficient flexibility in the design of the tag value representation scheme. Finally, as we will show later in the evaluation of our prototype, even a small number of P-cache entries is sufficient to achieve good performance for a wide variety of workloads.

7.4.4 Device access control

Device drivers present a significant security challenge in modern operating systems. Often written by third-party developers rather than operating system experts, device drivers have been shown to be of much lower quality than other operating system code. 85% of reported Windows XP crashes have been traced to faulty device drivers [68], while static analysis tools have found error rates in Linux device drivers to be up to 7 times higher than other kernel code [16]. Even a high-security operating system such as HiStar would have to trust millions of lines of code to support the same breadth of devices as Linux or Windows.

Existing hardware makes it difficult to remove device drivers from the TCB. Many hardware devices support DMA, which can read or write physical memory without involving the CPU or MMU. As a result, DMA bypasses all the protection and security mechanisms in the CPU and MMU. Thus, a device driver with access to a DMA-capable device can use the device to initiate DMA transfers and arbitrarily read or write any location in physical memory, including those that are part of the TCB.

To prevent device drivers from compromising the TCB, Loki provides additional hardware support, a *DMA permission table* stored in the memory controller. For each device, the table specifies the device's access rights for different memory tag values that can be accessed via DMA. The memory controller then ensures that DMA transactions can only access memory whose tags are marked accessible in the DMA permission table. This table is managed by the security monitor. As a consequence, untrusted code must make a call to the monitor to add a region of memory as a DMA source or destination. While this adds some overhead, this operation is infrequent. This design protects trusted code from device drivers, allowing device drivers to be removed from the TCB.

Loki also prevents rogue device drivers from corrupting other devices, by providing fine-grained device access control. Loki does this by associating tags with all memory-mapped registers. Permission table entries are then set by the monitor to ensure that each

device driver can only access memory that has the data tag of its associated device, and any memory accesses to other hardware devices are forbidden. Loki also forbids DMA transactions between devices, in order to prevent a rogue device driver from using DMA to bypass the protection mechanisms and take over another device via its memory-mapped registers.

7.4.5 Tag exceptions

When a tag permission check fails, control must be transferred to the security monitor, which will either update the permission cache based on the tag of the accessed memory location, or terminate the offending protection domain. Ideally, the exception mechanism will be such that the trusted security handler can be as simple as possible, to minimize TCB size. Traditional trap and interrupt handling facilities do not conform with this, as they rely on the integrity of the MMU state, such as page tables, and privileged registers that may be modified by potentially malicious kernel code.

To address this limitation, Loki introduces a tag exception mechanism that is independent of the traditional CPU exception mechanism. On a tag exception, Loki saves exception information to a few dedicated hardware registers, disables the MMU, switches to the monitor privilege level, and jumps to the tag exception handler in the trusted monitor. The MMU must be disabled because untrusted kernel code has full control over MMU registers and page tables. For simplicity, Loki also disables external device interrupts when handling a tag exception. The predefined address for the monitor is available in a special register introduced by Loki, which can only be updated while in monitor mode, to preclude malicious code from hijacking monitor mode. As all code in the monitor is trusted, tag permission checks are disabled in monitor mode. The monitor also has direct access to a set of registers that contain information about the tag exception, such as the faulting tag.

7.5 Prototype Evaluation

One of the main goals of this chapter was to show that tagged memory support can significantly reduce the amount of trusted code in a system. To that end, this section reports on our prototype implementation of Loki hardware and the complexity and security of our LoStar software prototype. We then show that our prototype performs acceptably by evaluating its performance, and justify our hardware parameter choices by measuring the patterns and locality of tag usage.

In modifying HiStar to take advantage of Loki, we added approximately 1,300 lines of C and assembly code to the kernel, and modified another 300 lines of C code, but the resulting TCB is reduced by 6,400 lines of code—*more than a factor of two*. While Loki greatly reduces the amount of trusted code, we have no formal proof of the system’s security. Instead, our current prototype relies on manual inspection of both its design and implementation to minimize the risk of a vulnerability.

7.5.1 Loki prototype

To evaluate our design of Loki, we developed a prototype system based on the SPARC architecture. Our prototype is based on the Leon SPARC V8 processor, a 32-bit open-source synthesizable core developed by Gaisler Research [49]. We modified the pipeline to perform our security operations, and mapped the design to an FPGA board, resulting in a fully functional SPARC system that runs HiStar. This gives us the ability to run real-world applications and gauge the effectiveness of our security primitives.

Leon uses a single-issue, 7-stage pipeline. We modified its RTL code to add support for coarse and fine-grained tags, added the P-cache, introduced the security registers defined by Loki, and added the instructions that manipulate special registers and provide direct access to tags in the monitor mode. We added 6 instructions to the SPARC ISA to read/write memory tags, read/write security registers, write to the permission cache, and return from

Parameter	Specification
Pipeline depth	7 stages
Register windows	8
Instruction cache	16 KB, 2-way set-associative
Data cache	32 KB, 2-way set-associative
Instruction TLB	8 entries, fully-associative
Data TLB	8 entries, fully-associative
Memory bus width	64 bits
Prototype Board	Xilinx University Program (XUP)
FPGA device	XC2VP30
Memory	512 MB SDRAM DIMM
Network I/O	100 Mbps Ethernet MAC
Clock frequency	65 MHz

Table 7.1: The architectural and design parameters for our prototype of the Loki architecture.

a tag exception. We also added 7 security registers that store the exception PC, exception nPC, cause of exception, tag of the faulting memory location, monitor mode flag, address of the tag exception handler in the monitor, and the address of the base of the page-tag array. Figure 7.4 shows the prototype we built.

We built a permission cache using the design discussed in Section 7.4.3. This cache has 32 entries and is 2-way set-associative. During instruction fetch, the tag of the instruction’s memory word is read in along with the instruction from the I-cache. This tag is used to check the Execute permission bit. Memory operations—loads and stores—index this cache a second time, using the memory word’s tag. This is used to check the Read and Write permission bits. As a result, the permission cache is accessed at least once by every instruction, and twice by some instructions. This requires either two ports into the cache or separate execute and read/write P-caches to allow for simultaneous lookups. Figure 7.4 shows a simplified version of this design for clarity.

As mentioned in Section 7.4.1, we implement a multi-granular tag scheme with a page-tag array that stores the page-level tags for all the pages in the system. These tags are cached for performance in an 8-entry cache that resembles a TLB. Fine-grained tags can

Component	Block RAMs	4-input LUTs
Base Leon	43	14,502
Loki Logic	2	2,756
Loki Total	45	17,258
Increase over base	5%	19%

Table 7.2: Complexity of our prototype FPGA implementation of Loki in terms of FPGA block RAMs and 4-input LUTs.

be allocated on demand at word granularity. We reserve a portion of main memory for storing these tags and modified the memory controller to properly access both data and tags on cached and uncached requests. We also modified the instruction and data caches to accommodate these tag bits. We evaluate this scheme further in Section 7.5.4.

We synthesized our design on the Xilinx University Program (XUP) board which contains a Xilinx XC2VP30 FPGA. Table 7.1 summarizes the basic board and design statistics, and Table 7.2 quantifies the changes made for the Loki prototype by detailing the utilization of FPGA resources. Note that the area overhead of Loki’s logic will be lower in modern superscalar designs that are significantly more complex than the Leon. Since Leon uses a write-through, no-write-allocate data cache, we had to modify its design to perform a read-modify-write access on the tag bits in the case of a write miss. This change and its small impact on application performance would not have been necessary with a write-back cache. There was no other impact on the processor performance, as the permission table accesses and tag processing occur in parallel and are independent from data processing in all pipeline stages.

7.5.2 Trusted code base

To evaluate how well the Loki architecture allows an operating system to reduce the amount of trusted code, we compare the sizes of the original, fully trusted HiStar kernel for the Leon SPARC system, and the modified LoStar kernel that includes a security monitor, in

Lines of code	HiStar	LoStar
Kernel code	11,600 (trusted)	12,700 (untrusted)
Bootstrapping code	1,300	1,300
Security monitor code	N/A	5,200 (trusted)
TCB size: <i>trusted code</i>	11,600	5,200

Table 7.3: Complexity of the original *trusted* HiStar kernel, the *untrusted* LoStar kernel, and the *trusted* LoStar security monitor. The size of the LoStar kernel includes the security monitor, since the kernel uses some common code shared with the security monitor. The bootstrapping code, used during boot to initialize the kernel and the security monitor, is not counted as part of the TCB because it is not part of the attack surface in our threat model.

Table 7.3. To approximate the size and complexity of the trusted code base, we report the total number of lines of code. The kernel and the monitor are largely written in C, although each of them also uses a few hundred lines of assembly for handling hardware traps. LoStar reduces the amount of trusted code in comparison with HiStar by more than a factor of two. The code that LoStar removed from the TCB is evenly split between three main categories: the system call interface, page table handling, and resource management (the security monitor tags pages of memory but does not directly manage them).

7.5.3 Performance

To understand the performance characteristics of our design, we compare the relative performance of a set of applications running on unmodified HiStar on a Leon processor and on our modified LoStar system on a Leon processor with Loki support. The application binaries are the same in both cases, since the kernel interface remains the same. We also measure the performance of LoStar while using only word-granularity tags, to illustrate the need for page-level tag support in hardware.

Figure 7.5 shows the performance of a number of benchmarks. Overall, most benchmarks achieve similar performance under HiStar and LoStar (overhead for LoStar ranges from 0% to 4%), but support for page-level tags is critical for good performance, due to the

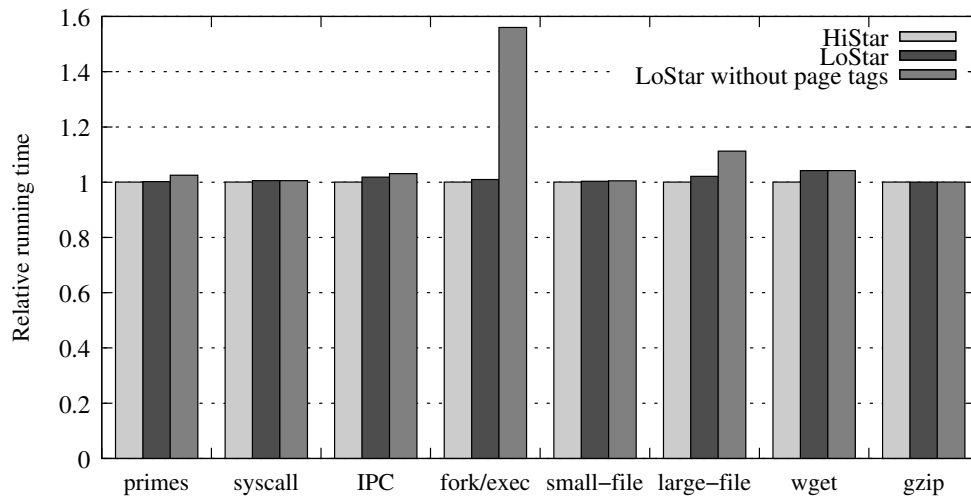


Figure 7.5: Relative running time (wall clock time) of benchmarks running on unmodified HiStar, on LoStar, and on a version of LoStar without page-level tag support, normalized to the running time on HiStar. The *primes* workload computes the prime numbers from 1 to 100,000. The *syscall* workload executes a system call that gets the ID of the current thread. The *IPC ping-pong* workload sends a short message back and forth between two processes over a pipe. The *fork/exec* workload spawns a new process using `fork` and `exec`. The *small-file* workload creates, reads, and deletes 1000 512-byte files. The *large-file* workload performs random 4KB reads and writes within a single 4MB file. The *wget* workload measures the time to download a large file from a web server over the local area network. Finally, the *gzip* workload compresses a 1MB binary file.

extensive use of page-level memory tagging. For example, the page allocator must change the tag values for all of the words in an entire page of memory in order to give a particular protection domain access to a newly-allocated page. Conversely, to revoke access to a page from a protection domain when the page is freed, the page allocator must reset all tag values back to a special tag value that no other protection domain can access. Explicitly setting tags for each of the words in a page incurs a significant performance penalty (up to 55%), and being able to adjust the tag of a page with a single memory write greatly improves performance.

Compute-intensive applications, represented by the *primes* and *gzip* workloads, achieve the same performance in both cases (0% overhead). Even system-intensive applications that do not switch protection domains, such as the system call and file system benchmarks, incur negligible overhead (0-2%), since they rarely invoke the security monitor. Applications that frequently switch between protection domains incur a slightly higher overhead, because all protection domain context switches must be done through the security monitor, as illustrated by the *IPC ping-pong* workload (2% overhead). However, LoStar achieves good network I/O performance, despite a user-level TCP/IP stack that causes significant context switching, as can be seen in the *wget* workload (4% overhead). Finally, creation of a new protection domain, illustrated by the *fork/exec* workload, involves re-labeling a large number of pages, as can be seen from the high performance overhead (55%) without page-level tags. However, the use of page-level tags reduces that overhead down to just 1%.

7.5.4 Tag usage and storage

To evaluate our hardware design parameters, we measured the tag usage patterns of the different workloads. In particular, we wanted to determine the number of pages that require fine-grained word-level tags versus the number of pages where all of the words in the page

Workload	primes	syscall	IPC	fork/exec	small files	large files	wget	gzip
Fraction of memory pages with word-granularity tags	40%	49%	54%	65%	58%	3%	18%	16%
Maximum number of concurrently accessed tags	12	11	18	24	13	13	30	12

Table 7.4: Tag usage under different workloads running on LoStar.

have the same tag value, and the working set size of tags—that is, how many different tags are used at once by different workloads. Table 7.4 summarizes our results for the workloads from the previous sub-section.

The results show that all of the different workloads under consideration make moderate use of fine-grained tags. The primary use of fine-grained tags comes from protecting the metadata of each kernel object. For example, workloads with a large number of small files, each of which corresponds to a separate kernel object, require significantly more pages with fine-grained tags compared to a workload that uses a small number of large files. Since Loki implements fine-grained tagging for a page by allocating a shadow page to store a 32-bit tag for each 32-bit word of the original page, tag storage overhead for such pages is 100%. On the other hand, pages storing user data (which includes file contents) have page-level tags, which incur a much lower tag storage overhead of $4/4096 \approx 0.1\%$. As a result, overall tag storage overhead is largely influenced by the average size of kernel objects cached in memory for a given workload. We expect that it is possible to further reduce tag storage overhead for fine-grained tags by using a more compact in-memory representation, like the one used by Mondriaan Memory Protection [90], although doing so would likely increase complexity either in hardware or software.

Finally, all workloads shown in Table 7.4 exhibit reasonable tag locality, requiring only a small number of tags at time. This supports our design decision to use a small fixed-size hardware permission cache.

7.6 Related Work

In this section, we review related hardware protection architectures. An in-depth analysis can be found in [96].

Multics [78] introduced hierarchical protection rings which were used to isolate trusted code in a coarse-grained manner. *x86* processors also have 4 privilege levels, but the page table mechanism can only distinguish between two effective levels. However, application security policies are often non-hierarchical, and Loki's 32-bit tag space provides a way of representing a large number of such policies in hardware.

The Intel i432 and Cambridge CAP systems, among others [50], augment the way applications name memory with a capability, which allows enforcing non-hierarchical security policies by controlling access to capabilities, at the cost of changing the way software uses pointers. Loki associates security policies with physical memory, instead of introducing a name translation mechanism to perform security checks. As a result, the security policy for any piece of data in Loki is always unambiguously defined, regardless of any aliasing that may be present in higher-level translation mechanisms.

The protection lookaside buffer (PLB) [44] provides a similarly non-hierarchical access control mechanism for a global address space (although only at page-level granularity). While the PLB caches permissions for virtual addresses, Loki's permissions cache stores permissions in terms of tag values, which is much more compact, as Section 7.5.4 suggests.

The IBM system i [35] associates a one-bit tag with physical memory to indicate whether the value represents a pointer or not. Similarly, the Intel i960 [38] provides a one-bit tag to protect kernel memory. Loki's tagged memory architecture is more general, providing a large number of protection domains.

Mondriaan Memory Protection (MMP) [90] provides lightweight, fine-grained (down to individual memory words) protection domains for isolating buggy code. However, MMP

was not designed to reduce the amount of trusted code in a system. Since the MMP supervisor relies on the integrity of the MMU and page tables, MMP cannot enforce security guarantees once the kernel is compromised. Loki extends the idea of lightweight protection domains to physical resources, such as physical memory, to achieve benefits similar to MMP's protection domains with stronger guarantees and a much smaller TCB. Moreover, this chapter describes how a fine-grained memory protection mechanism can be used to extend the enforcement of application security policies all the way down into hardware.

The Loki design was initially inspired by the Raksha hardware architecture [24]. However, the two systems have significant design differences. Raksha maintains four independent one-bit tag values (corresponding to four security policies) for each CPU register and each word in physical memory, and propagates tag values according to customizable tag propagation rules. Loki, on the other hand, maintains a single 32-bit tag value for each word of physical memory (allowing the security monitor to define how multiple security policies interact), does not tag CPU registers, and does not propagate tag values. Raksha's propagation of tag values was necessary for fine-grained taint tracking in unmodified applications, but it could not enforce write-protection of physical memory. Conversely, Loki's explicit specification of tag values works well for a system like HiStar, where all state in the system already has a well-defined security label that controls both read and write access.

Recent proposals in I/O virtualization have described schemes for DMA access control. AMD's Device Exclusion Vector (DEV) [1] provides a mechanism for protecting the kernel's memory from DMA requests by malicious or buggy devices and drivers. As discussed in Section 7.4.4, Loki's tagged access control mechanism could provide multiple protection domains for DMA and protect memory-mapped registers from rogue accesses, unlike DEV. IOMMU support in Intel's recent chipsets, called VT-d, can also be used to control device DMA, although properly implementing protection through translation requires avoiding peer-to-peer bus transactions and other pitfalls [76].

Hardware designs for preventing information leaks in user applications have also been

proposed [79, 87], although these designs do not attempt to reduce the TCB size. None of these designs provide a sufficiently large number of protection domains needed to capture different application security policies. Moreover, enforcement of information flow control in hardware has inherent covert channels relating to the re-labeling of physical memory locations. HiStar’s system call interface avoids this by providing a virtually unlimited space of kernel object IDs that are never re-labeled.

7.7 Summary

This chapter showed how hardware support for tagged memory can be used to enforce application security policies. We presented Loki, a hardware tagged memory architecture that provides fine-grained, software-managed access control for physical memory. We also showed how HiStar, an existing operating system, can take advantage of Loki by directly mapping application security policies to the hardware protection mechanism. This allows the amount of trusted code in the HiStar kernel to be reduced by over a factor of two. We built a full-system prototype of Loki by modifying a synthesizable SPARC core, mapping it to an FPGA board, and porting HiStar to run on it. The prototype demonstrates that our design can provide strong security guarantees while achieving good performance for a variety of workloads in a familiar Unix environment.

Chapter 8

Generalizing Tag Architectures

In this dissertation, we have addressed the development of hardware tag architectures for security, with emphasis on dynamic analysis techniques such as information flow tracking and information flow control. Hardware support for metadata is an extremely powerful abstraction that can be used by a host of other dynamic analyses. Similar to DIFT, these analyses require hardware support for tags to obtain good performance with fine-grained metadata, and to be compatible with all kinds of binaries. Extending the primitives adopted by hardware DIFT and DIFC architectures to perform other analyses amortizes the cost of the hardware changes required to the design, decreasing the risk factor for processor vendors. This allows for the construction of a *generalized tag architecture* containing primitives that can be leveraged by an expansive suite of dynamic analyses. Other analysis-specific features can be layered upon this common substrate as required. This chapter attempts to identify and codify this set of common primitives required by all analyses, and discuss the required hooks that must be provided to implement analysis-specific features.

The rest of this chapter is organized as follows. In Sections 8.1 through 8.6, we list several applications that make use of hardware tag architectures. For each of these applications, we describe the hardware and software features required by the system. As seen in Chapter 5, decoupling the analysis hardware support from the main processor helps

increase the likelihood of adoption by processor vendors. Thus, for each application, we discuss the implications of decoupling the required hardware support from the main processor. We then list the key primitives that must be exposed by any generalized tag architecture in Section 8.7, before discussing related work in Section 8.8 and concluding the chapter.

8.1 Debugging

Bugs in deployed software account for as many as 40% of computer system failures observed [29]. Software bugs crash systems, or render them unavailable, or even generate incorrect outputs or corrupt information. According to NIST [63], software bugs cost the U.S economy an estimated \$59.5 billion in 2002, or 0.6% of the GDP. Techniques for debugging software have thus become hotbeds of research in the recent past.

A popular approach to debugging memory allocation related bugs is to dynamically monitor the actual execution paths of the application. Architectures such as the x86 and SPARC provide a limited number of hardware breakpoints and watchpoints which can be used to monitor transitions of individual memory of words. More generally, systems such as iWatcher [97] use tagged memory to provide infinite hardware breakpoints and watchpoints. Every word of memory is associated with a tag. If a load or store memory operation is triggered on an address being monitored (breakpoint or watchpoint respectively), an exception is triggered. This exception invokes a software monitor responsible for logging any data and performing further analysis.

8.1.1 Tag storage and manipulation

Debugging systems associate a tag bit with every word of memory. These tags are stored in caches and main memory. Registers do not require tags. Tags are used to mark sensitive areas of memory that require monitoring. Tags are initialized and reset by a software

monitor, in accordance with the debugging policies. Thus, there is no hardware propagation. Tags must however, be checked on every memory access, since they can serve as both breakpoints and watchpoints. If a tag is used as a breakpoint, then any load of that memory address would result in an exception. If the tag is used as a watchpoint, then any store to that memory address would cause an exception. The exception then transfers control to a software monitor that logs the cause of the exception, and performs further analysis as required. Since these exceptions could be frequent events, it is important for them to be extremely light-weight.

8.1.2 Decoupling the hardware analysis

If the management and checking of tags were decoupled from the main core (for e.g. to a tag coprocessor), then the main core and the coprocessor would be required to synchronize on every instruction. This is because the hardware must raise a tag exception every time the associated data is accessed. Unlike DIFT, these exceptions must be precise, in order for the monitor to be able to log data accurately, or perform further analysis. Thus, a fully decoupled coprocessor design, such as the one described in Chapter 5 would not work well for this analysis.

8.2 Profiling

Modern systems are composed of a variety of interacting services, and run across multiple machines. Consequently, it is very difficult for developers to get a good understanding of the entire system. One of the more promising techniques for understanding system performance pathologies is Dataflow Tomography. This technique profiles the running applications using the inherent information flow in large systems to help visualize the interactions of different components of the system, across multiple layers of abstraction [60]. These systems associate tags with words of data memory, and track the propagation of tainted

data. Chow *et al.* used this idea to analyze data lifetime, and track the flow of sensitive data through the system [17]. Since the analysis requires visibility of every memory location in the system, it incurs a high performance overhead when done in a DBT.

8.2.1 Tag storage and manipulation

Profiling architectures extend all registers and memory locations to store a tag with every word. These systems use a one-bit tag per word of memory, to indicate if the associated memory has been accessed by the application. Thus, main memory, caches and the register files need to be modified to accommodate tags. Tags are initialized for all of the relevant application's memory by software.

Tags get propagated when the application in question communicates with other programs, indicating the flow of information through the system. Propagation occurs on every instruction, similar to DIFT architectures such as Raksha. Profiling systems usually perform a logical OR of the source operand tags. Profiling analyses are required to periodically log information about the state of the system. This is done by enabling tag checks at sensitive process boundaries (system calls etc.). Software is responsible for configuring the tag propagation and check policies. A software monitor similar to that used in Raksha could be used to log profile data. Since profiles are frequently generated, security exceptions should be light-weight, and have a low overhead.

8.2.2 Decoupling the hardware analysis

Similar to the DIFT coprocessor, the management, propagation and checking of tags could be done outside the main processor. Since the coprocessor merely implements a profiling analysis, the main core and coprocessor could synchronize at certain boundaries like system calls. This allows for imprecise exceptions, and for the main core to run ahead of the tag coprocessor. Decoupling the hardware analysis however, introduces (*data, metadata*)

consistency challenges similar to DIFT architectures. The consistency mechanism outlined in Chapter 6 can be used to solve this problem.

8.3 Pointer bits

As Chapter 4 discussed, many security attacks stem from incorrect handling of pointers. Thus, a number of systems have used tag bits to indicate if the associated data is a pointer [35, 38]. This information allows the system to determine if memory accesses made by a pointer value are permissible or not. Knowledge of pointer bits has also been leveraged in data forwarding [55]. This system used tags as "forwarding" bits; if the tag bit were set, accessing the associated data would trigger a fetch of the address stored in the memory word. Similar to the previously discussed analyses, performing this in software by means of binary translation would incur significant performance overheads.

8.3.1 Tag storage and manipulation

Every word of physical memory has an associated tag bit that indicates if the value represents a pointer or not. The IBM system i [35], and the Intel i960 [38] used one-bit tags as pointer bits, to protect kernel memory. The Burroughs 5500 [10] stored a three-bit tag per word of physical memory to identify the contents of the memory word as either an instruction, or data, or as control information. This served as a memory protection mechanism by preventing the execution of arbitrary data values, as instructions. The pointer tag bits are stored in main memory and the caches. Registers do not require tags.

Tag initialization involves setting tag bits for all pointers in the system. This can be done by software using compile-time information, or dynamically at run-time [25]. Pointer bits are propagated on pointer arithmetic operations, i.e. whenever new pointers are formed. The propagation rules are identical to those used by Raksha's pointer bit [25]. Tags must be checked on every memory access for potential security violations [10], or to generate

memory fetches [55]. Tag check failures cause a software exception, which should be light-weight for best performance.

8.3.2 Decoupling the hardware analysis

Since security exceptions and memory fetch operations must be triggered on access of tagged pointers, tag exceptions must be precise. This implies that data and metadata must synchronize on every instruction. Thus, a fully decoupled DIFT coprocessor design would not work well for this analysis.

8.4 Full/empty bits

Some machines such as the Cray TERA MTA supercomputer [32] provided support for *full/empty* tag bits for fine-grained producer-consumer synchronization. Every word of memory has a full/empty tag bit which is set when the word is "full" with newly produced data (i.e. on a write), and unset when the word is "empty" or consumed by another processor (i.e. on a read). Producers write to locations only if the full/empty bit is set to empty, and then leave the bit set to full. Consumers read locations only if the bit is full, and then reset it to empty. Hardware manipulates the full/empty bit to preserve the atomicity of the memory update operation [27].

8.4.1 Tag storage and manipulation

Every word of memory has an associated tag bit to maintain its full/empty status. The Cray MTA stores full/empty tags only in main memory. Memory tags are set and reset by producer and consumer processors. Thus, there is no software initialization of tags required for this analysis. Tag propagation is not relevant in the context of full/empty bits. Since tags are used to implement synchronization, the full/empty status must be checked on every

access to shared memory. Tag check failures do not raise software exceptions; instead they just reset the tag value as appropriate. This read-modify-write behavior of tags introduces additional complexity in the memory controller.

8.4.2 Decoupling the hardware analysis

Tags and data synchronize on every memory access. This is because a memory access by any processor requires for the tags to be checked and reset. Memory words can be accessed only if permitted by the tag value. Data accesses could also require a subsequent tag update. Consequently, tag and data processing must always be in lock-step. Thus, a fully decoupled DIFT coprocessor type of design would not work well for this analysis.

8.5 Fault Tolerance and Speculative Execution

As silicon integration levels increase, devices become more susceptible to soft errors. A soft error is a glitch caused in a semiconductor device by a charged particle striking the design, causing the stored information to get corrupted. While high-availability systems usually protect the processor's caches (using ECC bits), and the register file (via radiation-hardening), pipeline registers and latches are susceptible to corruption on bombardment by high energy particles. Researchers have proposed associating every instruction with a tag bit for Fault Tolerance (FT), called the π bit, that is associated with every instruction as it flows down the pipeline from decode to retirement [89]. This bit is set if the instruction is thought to be potentially incorrect. The machine checks for incorrect instructions at commit time.

A related analysis is that of Speculative Execution (SE) in a multiprocessor. Modern processors perform very aggressive speculation in order to maximize performance. The Itanium [37] associates a one-bit tag with every 64-bit register, called the NaT bit. NaT stands for "Not a Thing" and is used by SE to indicate that the register values are undefined.

Speculative loads, for example, do not produce exceptions, but set the NaT bit instead. A subsequent check instruction will jump to fix-up code if the NaT bit is set.

8.5.1 Tag storage and manipulation

Both FT and SE require that every register in the processor's pipeline have an associated tag bit. Neither application requires for tags to be stored in the caches or main memory. Tags are set and reset by checking hardware inside the pipeline of the processor, and are propagated across registers within the pipeline during instruction execution. Data that derives from speculative or potentially incorrect values must be marked so. Tag checks are performed at instruction commit time to prevent a speculative or incorrect value from being written to memory.

8.5.2 Decoupling the hardware analysis

The management and checking of tags used for SE and FT must be done within the main processor. Since tags are associated with pipeline registers, they have to be operated upon in parallel with the data. Thus, tag management cannot be decoupled from the main processor.

8.6 Transactional Memory and Cache QoS

Transactional Memory (TM) is a popular concurrency control mechanism that allows a group of memory instructions to execute in an atomic way. Hardware support for TM helps reduce the runtime overheads of implementing TM. Efficient implementation of TM requires the caches to be modified to maintain tags with every line. These tags are logically associated with data coherence, and are used by systems to maintain speculative state [34], or serve as mark bits [77].

The quality of service (QoS) offered by today's platforms is very non-deterministic

when multiple virtual machines or applications are run simultaneously. This is because different workloads place very different constraints on the system's resources. Recent studies on cache QoS have shown that proper management of cache resources can provide service differentiation and deterministic performance when running disparate workloads [43]. Cache QoS schemes maintain a tag for every cache line, to associate space consumed with IDs of executing applications, and enforce distribution of resources. This scheme has also been applied on TLBs to ensure deterministic performance [86].

8.6.1 Tag storage and manipulation

Both TM and QoS require the caches (or TLBs) to contain tags. Every cache line has an associated one-bit tag. Registers and main memory do not require the addition of tags. Tags are initialized by the hardware to either indicate what transaction the line belongs to (in the case of TM), or what thread the cache line belongs to (in the case of cache QoS). Software is responsible for configuring the QoS policies for the system, which in turn dictate the cache eviction policies. The tags are thus used to ensure equitable distribution of resources. Tag values do not propagate through the system, and are not written back to memory on cache line eviction. Since tags are used for resource management, they must be checked and potentially updated on every access to the cache line.

8.6.2 Decoupling the hardware analysis

In the case of TM and QoS, the tags are tied to the cache lines. Every physical access to a cache line requires a lookup of the tag. Thus, tags cannot be decoupled from the main processor's caches.

Requirement	DIFT	IFC	Debug bits	Profiling	Pointer bits	Full/empty bits	FT/SE	TM/Cache QoS
Fine-grained hardware metadata	Y	Y	Y	Y	Y	Y	Y	Y
Hardware tag checks	Y	Y	Y	Y	Y	Y	Y	Y
Software management of tag policies	Y	Y	Y	Y	Y	Y	N	Y
Low-overhead tag exceptions	Y	Y	Y	Y	Y	Y	N	Y
Hardware propagation	Y	N	N	Y	Y	N	N	N
Support imprecise tag exceptions	Y	N	N	Y	N	N	N	N

Table 8.1: Comparison of different tag analyses.

8.7 Generalizing Architectures for Hardware Tags

All the above described systems make use of hardware tags for dynamic analysis. The common features of these applications include association of metadata with data at a fine granularity, and hardware maintenance and checking of metadata. Additionally, the analyses that interact with software require both software management of policies governing the metadata, and a low-overhead mechanism for invoking a software handler for further analysis. Specifically, all these systems require that hardware maintain the metadata in order to have low performance overheads, and perform periodic checks on the metadata at certain boundaries (defined by the system). When the analysis interacts with software, the system must maintain a software handler that both manages the policies in order to ensure flexibility and configurability, and perform a further analysis in the case of a tag exception.

As Table 8.1 illustrates, the previously mentioned systems have two fundamental differences. First, not all systems require propagation of tags. While every analysis requires some kind of support for tag checks, only information flow analyses such as DIFT and profiling require support for propagation of tags. The second difference is the decoupling allowed between data and metadata. Some analyses such as DIFT do not require precise tag exceptions, allowing for the use of a coprocessor such as the one described in Chapter 5 to minimize changes required to be made to the main processor core.

A general architecture for tags must thus have the following features:

- **Ability to associate metadata with every word of data in the system.** Hardware should provide a fine-grained tag management scheme, allowing the analysis to be able to specify policies at the granularity of words, or even bytes, of memory. In addition, many analyses have shown that metadata exhibits significant spatial locality. Thus, the architecture must also have the ability to specify metadata at coarser granularities, such as at the granularity of a page of data. The system must also provide support for a multi-granular tag management scheme to account for the spatial locality that tags tend to exhibit [24, 96]. This in turn begets the need for a flexible scheme for maintaining and caching tags. This scheme would provide correct tag management in the caches, when configured with the desired length of tags.
- **Hardware to perform low-level operations on the metadata.** The hardware should store the metadata, and perform tag checks. In order for the architecture to be compliant with existing DRAM memory formats, it is necessary to maintain metadata on a separate page. This requires that the operating system be made aware of metadata in order to perform memory allocation and schedule memory swapping accordingly. Tag propagation and decoupling tag analyses onto a dedicated coprocessor are related issues that are not central to all analyses. The techniques described in Chapter 5 are applicable to any analysis that requires information flow propagation. Other analyses

that do not fit the information flow paradigm could use a more generalized propagation mechanism such as that implemented in FlexiTaint [88], where software is responsible for setting the propagation policies on a per-instruction basis. While many analyses such as those using pointer bits, or full-empty bits require tight coupling between data and tags, analyses such as DIFT allow for the decoupling of metadata processing. These analyses differ in the granularity of synchronization required between data and tags. Analyses that do not require synchronization on every instruction can be decoupled to a coprocessor. Analyses such as information flow control require support for precise exceptions. Decoupling such analyses would require that instruction commit be delayed until the metadata is processed and checked by the coprocessor. This is similar to the DIVA architecture for reliability, which shows that the performance overheads of such a scheme, while higher than that of the DIFT coprocessor described in Chapter 5, are acceptable under certain scenarios [3].

- **Software management of metadata policies.** As argued in Chapter 3, hardcoding policies in hardware restricts the adaptability and malleability of the analysis system. As illustrated by Table 8.1, many analysis systems require the ability to specify and configure the analysis policies in a software handler. Software policies can be encoded in hardware registers which in turn define the check (and if required, propagation) policies. In order to be able to apply an analysis routine on the operating system, the software handler must run in a special operating mode outside supervisor mode.
- **Low-overhead hardware exceptions.** Many analysis architectures require the ability to invoke the software handler to run further analysis, log data, or terminate the application as the case may be. The frequency of invocation of this handler is dependent upon the analysis chosen. In order to reduce the overhead of the software

analysis routine, hardware must provide a low-overhead exception mechanism. Traditional exception mechanisms require context switch operations which are very expensive operations. Running the software handler in the same address space as the application allows for an inexpensive transition to the analysis routine when a hardware check fails. This provides the system with the ability to run more complex analyses in software as required, extending its capabilities significantly.

As mentioned earlier, features such as propagation of tags are not central to all analysis systems. The ability to incorporate such features is thus, best provided by means of a decoupled coprocessor. This minimizes the changes to the main core, and allows for the ability to update the coprocessor easily depending upon the choice of analysis.

8.8 Related Work

While there has been significant work on adding analysis-specific microarchitectural features to systems [32, 35, 81], very few systems have focused on adding a configurable set of features that can be programmed to serve different needs. Consequently, chip designers are often loathe to adding such analysis-specific features to their designs, since they cannot be reused for other purposes. The log-based architecture [12, 13] is one such design that attempts to provide a set of hardware primitives that can be used to perform a variety of dynamic analyses. As explained in Chapter 5, this architecture offloads the functionality of the analysis to another core in a multi-core chip. The analysis is performed in a software dynamic binary translation environment. The core running the application generates a trace of executing instructions which is used by the analysis core. While this approach provides the flexibility to implement arbitrarily complex analyses in software, the hardware changes are invasive, and have a high area and performance overhead, as explained in Chapter 5.

Smart Memories [31, 56] is an architecture that provides configurability in memory controllers, and breaks down the on-chip memory system's functionality into a set of basic

operations. The system also provides the necessary means for combining and sequencing these operations. This configurability allows the system to dynamically change the data communication protocol implemented by its memory controller. In order to provide this configurability, there are six metadata bits associated with every data word of memory whose functionality can be extensively programmed. The memory controller also has the ability to update these bits on a hardware access, and accesses them concurrently with data. Smart Memories used these bits to implement a variety of memory models by configuring them to implement cache line states, transaction read/write sets, or even fine-grained locks [56]. The system provides both the ability to associate metadata with every word of memory, and the support to maintain and manage this metadata. Combined with a software monitor for managing the metadata policies and a low-overhead hardware exception mechanism, it could potentially serve as a generalized architecture for metadata analysis.

8.9 Summary

Architectural support for dynamic analysis has been a fertile area of research. There have been many architectures proposed that make use of tags for dynamic analyses. For an architectural change to be practically viable to processor vendors, it must be applicable to a suite of applications, thus allowing for the cost of implementation to be amortized. Since most of the applications require a certain common subset of features to be implemented by the analysis system, it is possible to build a general tag architecture framework that can be used by a whole suite of analyses.

In this chapter, we surveyed some of the more common tag architectures, and codified the common primitives exposed by these systems, in order to obtain a blueprint of a generalized tag architecture. Such an architecture would maintain and manage tags in hardware, and manage policies in software, with a low-overhead tag exception mechanism. Other application-specific features such as propagation of tags could be optionally implemented

in an offcore coprocessor similar to the one proposed in Chapter 5. This allows hardware vendors to amortize the cost and design complexity of tags over multiple processor designs, and use them for multiple analyses and applications, thereby decreasing the risk of implementation.

Chapter 9

Conclusions

Dynamic Information Flow Tracking, or DIFT, is a powerful and flexible security technique that provides comprehensive protection against a variety of critical software threats. This dissertation demonstrated that a well-designed hardware DIFT system can protect unmodified applications, and even the operating system, from a wide range of vulnerabilities, with little or no performance, area, and cost penalties.

We developed Raksha, a flexible hardware DIFT platform that allows specification of DIFT security policies using software managed tag policy registers. Raksha provides comprehensive protection against low-level memory corruption exploits such as buffer overflows and high-level semantic attacks such as SQL injections on unmodified applications, and even the operating system kernel. We built a full-system prototype of Raksha using a synthesizable SPARC V8 processor and an FPGA board, and demonstrated that the area and performance overheads of the Raksha architecture are minimal.

We developed a coprocessor based DIFT architecture to address the practicality issue of implementing DIFT in the real world. Using a coprocessor that encapsulates all DIFT functionality greatly reduces the design and validation overheads of implementing DIFT in the main processor pipeline, and allows for easy reuse across different designs. We prototyped this architecture on a synthesizable SPARC V8 core on an FPGA board. This

decoupled design had low performance overheads, and did not compromise the security of the DIFT approach.

We provided a practical and fast hardware solution to the problem of inconsistency between data and metadata in multiprocessor systems when DIFT functionality is decoupled from the main core. This solution leverages cache coherence mechanisms to record interleaving of memory operations from application threads and replays the same order on metadata processors to maintain consistency, thereby allowing correct execution of dynamic analysis on multithreaded programs.

We also explored using tagged memory architectures to solve security problems other than DIFT. We showed that HiStar, an existing operating system, could take advantage of a tagged memory architecture to enforce its information flow control policies directly in hardware, and thereby reduce the amount of trusted code in its kernel by over a factor of two. Using a full-system prototype built with a synthesizable SPARC core and an FPGA board, we showed that the overheads of such an architecture are minimal.

9.1 Future Work

While there has been significant interest in DIFT in academia, there remain several challenges to the widespread adoption of DIFT in the real world. More study is required to determine what security policies scale to enterprise environments, and what the necessary configurations are. There has also been very little work in exposing APIs to allow for system administrators to easily express their security policies in terms of DIFT mechanisms. Additionally, some web based vulnerabilities will benefit greatly from DIFT support in the language. Very little is known about the implications of adding DIFT support to an existing language [22].

There also remains a lot of work to be done towards building a unified architecture for

tags. While Chapter 8 identified some critical features required by different dynamic analyses, no current architecture is flexible enough to accommodate all the different requirements of these applications. This would require a flexible software interface, and APIs to allow system administrators and even application developers to specify their policies that would be directly enforced by the hardware. Such a design would also require the ability to run multiple orthogonal analyses simultaneously with minimal performance and power penalties. Multiplexing different policies on the same tag bits would reduce the storage overhead required, but would impose other correctness and performance challenges on the system. Progress in these areas would be an excellent first step in promoting industry-wide adoption of DIFT and hardware analysis techniques.

Bibliography

- [1] AMD. AMD I/O Virtualization Technology Specification, 2007.
- [2] AMD. AMD Lightweight Profiling Proposal. <http://developer.amd.com/assets/HardwareExtensionsforLightweightProfilingPublic20070720.pdf>, 2007.
- [3] Todd Austin. DIVA: A Reliable Substrate for Deep Submicron Microarchitecture Design. In *the Proc. of the 32nd International Symposium on Microarchitecture (MICRO)*, Haifa, Israel, November 1999.
- [4] David E. Bell and Leonard LaPadula. Secure computer system: Unified exposition and Multics interpretation. Technical Report MTR-2997, Rev. 1, MITRE Corp., Bedford, MA, March 1976.
- [5] Fabrice Bellard. QEMU, a fast and portable dynamic translator. In *Proc. of the 2005 USENIX, Freenix track*, Anaheim, CA, April 2005.
- [6] Kenneth J. Biba. Integrity considerations for secure computer systems. Technical Report TR-3153, MITRE Corp., Bedford, MA, April 1977.
- [7] Christian Bienia, Sanjeev Kumar, and Kai Li. PARSEC vs. SPLASH-2: A Quantitative Comparison of Two Multithreaded Benchmark Suites on Chip-Multiprocessors.

- In *the Proc. of the 2008 International Symposium on Workload Characterization (IISWC)*, Seattle, WA, 2008.
- [8] Christian Bienia, Sanjeev Kumar, Jaswinder Pal Singh, and Kai Li. The PARSEC Benchmark Suite: Characterization and Architectural Implications. In *the Proc. of the 17th International Conference on Parallel Architectures and Compilation Techniques (PACT)*, Toronto, Canada, October 2008.
- [9] Edson Borin, Cheng Wang, Youfeng Wu, and Guido Araujo. Software-based Transparent and Comprehensive Control-flow Error Detection. In *the Proc. of the 4th Intl. Symp. Code Generation and Optimization (CGO)*, New York, NY, March 2006.
- [10] The Burroughs 5500 computer architecture.
- [11] CERT Coordination Center. Overview of attack trends. http://www.cert.org/archive/pdf/attack_trends.pdf, 2002.
- [12] Shimin Chen, Babak Falsafi, et al. Logs and Lifeguards: Accelerating Dynamic Program Monitoring. Technical Report IRP-TR-06-05, Intel Research, Pittsburgh, PA, 2006.
- [13] Shimin Chen, Michael Kozuch, Theodoros Strigkos, Babak Falsafi, Phillip B. Gibbons, Todd C. Mowry, Vijaya Ramachandran, Olatunji Ruwase, Michael Ryan, and Evangelos Vlachos. Flexible Hardware Acceleration for Instruction-Grain Program Monitoring. In *the Proc. of the 35th International Symposium on Computer Architecture (ISCA)*, Beijing, China, June 2008.
- [14] Shuo Chen, Jun Xu, Nithin Nakka, Zbigniew Kalbarczyk, and Ravishankar Iyer. Defeating Memory Corruption Attacks via Pointer Taintedness Detection. In *the Proc. of the 35th International Conference on Dependable Systems and Networks (DSN)*, Yokohama, Japan, June 2005.

- [15] Shuo Chen, Jun Xu, Emre C. Sezer, Prachi Gauriar, and Ravishankar K. Iyer. Non-Control-Data Attacks Are Realistic Threats. In *the Proc. of the 14th USENIX Security Symposium*, Baltimore, MD, August 2005.
- [16] Andy Chou, Junfeng Yang, Benjamin Chelf, and Dawson Engler. An empirical study of operating system errors. In *the Proc. of the 18th ACM Symposium on Operating Systems Principles (SOSP)*, 2001.
- [17] Jim Chow, Ben Pfaff, Tal Garfinkel, Kevin Christopher, and Mendel Rosenblum. Understanding Data Lifetime via Whole system Simulation. In *the Proc. of the 13th USENIX Security Conference*, August 2004.
- [18] JaeWoong Chung, Michael Dalton, Hari Kannan, and Christos Kozyrakis. Thread-Safe Dynamic Binary Translation using Transactional Memory. In *the Proc. of the 14th International Conference on High-Performance Computer Architecture (HPCA)*, Salt Lake City, UT, February 2008.
- [19] M. Costa, J. Crowcroft, M. Castro, A. Rowstron, L. Zhou, L. Zhang, and P. Barham. Vigilante: End-to-end containment of internet worms. In *the Proc. of the 20th ACM Symposium on Operating Systems Principles (SOSP)*, Brighton, UK, October 2005.
- [20] Jedidiah R. Crandall and Frederic T. Chong. MINOS: Control Data Attack Prevention Orthogonal to Memory Model. In *the Proc. of the 37th International Symposium on Microarchitecture (MICRO)*, Portland, OR, December 2004.
- [21] Cross-Compiled Linux From Scratch. <http://cross-lfs.org>.
- [22] Michael Dalton. *The Design and Implementation of Dynamic Information Flow Tracking Systems For Software Security*. PhD thesis, Stanford University, December 2009.

- [23] Michael Dalton, Hari Kannan, and Christos Kozyrakis. Deconstructing Hardware Architectures for Security. In *the 5th Annual Workshop on Duplicating, Deconstructing, and Debunking (WDDD)*, Boston, MA, June 2006.
- [24] Michael Dalton, Hari Kannan, and Christos Kozyrakis. Raksha: A Flexible Information Flow Architecture for Software Security. In *the Proc. of the 34th International Symposium on Computer Architecture (ISCA)*, San Diego, CA, June 2007.
- [25] Michael Dalton, Hari Kannan, and Christos Kozyrakis. Real-World Buffer Overflow Protection for Userspace and Kernel-space. In *the Proc. of the 17th Usenix Security Symposium*, San Jose, CA, July 2008.
- [26] Michael Dalton, Christos Kozyrakis, and Nickolai Zeldovich. Nemesis: Preventing Authentication and Access Control Vulnerabilities in Web Applications. In *the Proc. of the 18th Usenix Security Symposium*, Montreal, QC, August 2009.
- [27] David Culler, Jaswinder Pal Singh, Anoop Gupta. Parallel Computer Architecture: A Hardware/Software Approach. Morgan Kaufmann, 1998.
- [28] Dorothy E. Denning and Peter J. Denning. Certification of programs for secure information flow. *ACM Communications*, 20(7), 1977.
- [29] E. Marcus and H. Stern. Blueprints for High Availability. John Willey and Sons, 2000.
- [30] Petros Efstathopoulos, Maxwell Krohn, Steve VanDeBogart, Cliff Frey, David Ziegler, Eddie Kohler, David Mazières, Frans Kaashoek, and Robert Morris. Labels and event processes in the Asbestos operating system. In *the Proc. of the 20th ACM Symposium on Operating Systems Principles (SOSP)*, Brighton, UK, October 2005.

- [31] Amin Firoozshahian, Alex Solomatnikov, Ofer Shacham, Zain Asgar, Stephen Richardson, Christos Kozyrakis, and Mark Horowitz. A Memory System Design Framework: Creating Smart Memories. In *the Proc. of the 36th International Symposium on Computer Architecture (ISCA)*, Austin, TX, June 2009.
- [32] George Davison, Constantine Pavlakos, Claudio Silva. Final Report for the Tera Computer TTI CRADA. Sandia National Labs Report SAND97-0134, January 1997.
- [33] Vivek Haldar, Deepak Chandra, and Michael Franz. Dynamic taint propagation for java. *Computer Security Applications Conference, Annual*, 0:303–311, 2005.
- [34] Lance Hammond, Vicky Wong, Mike Chen, Brian D. Carlstrom, John D. Davis, Ben Hertzberg, Manohar K. Prabhu, Honggo Wijaya, Christos Kozyrakis, and Kunle Olukotun. Transactional memory coherence and consistency. In *the Proc. of the 31st International Symposium on Computer Architecture (ISCA)*. Munchen, Germany, Jun 2004.
- [35] IBM Corporation. IBM system i. <http://www-03.ibm.com/systems/i>.
- [36] Imperva Inc., How Safe is it Out There: Zeroing in on the vulnerabilities of application security. <http://www.imperva.com/company/news/2004-feb-02.html>, 2004.
- [37] Intel. Intel Itanium Architecture Software Developer’s Manual.
- [38] Intel Corporation. Intel i960 processors. <http://developer.intel.com/design/i960/>.
- [39] Intel Virtualization Technology (Intel VTx). <http://www.intel.com/technology/virtualization>.

- [40] Hari Kannan. Ordering Decoupled Metadata Accesses in Multiprocessors. In *the Proc. of the 42nd International Conference on Microarchitecture (MICRO)*, New York City, NY, December 2009.
- [41] Hari Kannan, Michael Dalton, and Christos Kozyrakis. Raksha: A Flexible Architecture for Software Security. In *the Technical Record of the 19th Hot Chips Symposium*, Stanford, CA, August 2007.
- [42] Hari Kannan, Michael Dalton, and Christos Kozyrakis. Decoupling Dynamic Information Flow Tracking with a Dedicated Coprocessor. In *the Proc. of the 39th International Conference on Dependable Systems and Networks (DSN)*, Estoril, Portugal, July 2009.
- [43] Hari Kannan, Fei Guo, Li Zhao, Ramesh Illikkal, Ravi Iyer, Don Newell, Yan Solihin, and Christos Kozyrakis. From Chaos to QoS: Case Studies in CMP Resource Management. In *the 2nd Workshop on Design, Architecture, and Simulation of Chip-Multiprocessors (dasCMP)*, Orlando, FL, December 2006.
- [44] Eric Koldinger, Jeff Chase, and Susan Eggers. Architectural support for single address space operating systems. Technical Report 92-03-10, University of Washington, Department of Computer Science and Engineering, March 1992.
- [45] Maxwell Krohn. Building secure high-performance web services with OKWS. In *Proc. of the 2004 USENIX*, June–July 2004.
- [46] Maxwell Krohn, Alexander Yip, Micah Brodsky, Natan Cliffer, M. Frans Kaashoek, Eddie Kohler, and Robert Morris. Information flow control for standard OS abstractions. In *the Proc. of the 21st ACM Symposium on Operating Systems Principles (SOSP)*, Stevenson, WA, October 2007.

- [47] Ian Kuon and Jonathan Rose. Measuring the Gap Between FPGAs and ASICs. In *the Proceedings of the 14th International Symposium on Field-Programmable Gate Arrays*, Monterey, CA, February 2006.
- [48] Butler Lampson, Martín Abadi, Michael Burrows, and Edward P. Wobber. Authentication in distributed systems: Theory and practice. *ACM TOCS*, 10(4):265–310, 1992.
- [49] LEON3 SPARC Processor. <http://www.gaisler.com>.
- [50] Henry M. Levy. *Capability-Based Computer Systems*. Digital Press, 1984.
- [51] Benjamin Livshits and Monica S. Lam. Finding security errors in Java programs with static analysis. In *Proc. of the 14th USENIX Security Symposium*, August 2005.
- [52] Benjamin Livshits, Michael Martin, and Monica S. Lam. SecuriFly: Runtime Protection and Recovery from Web Application Vulnerabilities. Technical report, Stanford University, September 2006.
- [53] Shih-Lien Lu, Peter Yiannacouras, Rolf Kassa, Michael Konow, and Taeweon Suh. An FPGA-Based Pentium in a Complete Desktop System. In *the Proc. of the 15th International Symposium on Field-Programmable Gate Arrays (FPGA)*, Monterey, CA, February 2007.
- [54] Chi-Keung Luk, Robert Cohn, Robert Muth, Harish Patil, Artur Klauser, Geoff Lowney, Steven Wallace, Vijay Janapa Reddi, and Kim Hazelwood. Pin: Building Customized Program Analysis Tools with Dynamic Instrumentation. In *the Proc. of the Conf. on Programming Language Design and Implementation (PLDI)*, Chicago, IL, June 2005.

- [55] Chi-Keung Luk and Todd Mowry. Memory Forwarding: Enabling Aggressive Layout Optimizations by Guaranteeing the Safety of Data Relocation. In *the Proc. of the 26th International Symposium on Computer Architecture (ISCA)*, Atlanta, GA, May 1999.
- [56] K. Mai, T. Paaske, N. Jayasena, R. Ho, W.J. Dally, and M. Horowitz. Smart Memories: A Modular Reconfigurable Architecture. In *the Proc. of the 27th International Symposium on Computer Architecture (ISCA)*, Vancouver, BC, June 2000.
- [57] Mark Dowd. Application-specific attacks: Leveraging the actionscript virtual machine. In *IBM Global Technology Services Whitepaper*, 2008. <http://documents.iss.net/whitepapers/IBM.X-Force.WP.Final.pdf>.
- [58] M. M. Martin, D. J. Sorin, et al. Multifacet's general execution-driven multiprocessor simulator (GEMS) toolset. In *Computer Architecture News (CAN)*, September 2005.
- [59] P. McKenney and J. Walpole. Introducing technology into the Linux kernel: a case study. *ACM SIGOPS Operating Systems Review*, 42(5), 2008.
- [60] Shashidhar Mysore, Bitan Mazloom, Banit Agrawal, and Timothy Sherwood. Understanding and Visualizing Full Systems with Data Flow Tomography . In *the Proc. of the 13th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, Seattle, WA, March 2008.
- [61] Vijay Nagarajan and Rajiv Gupta. Architectural Support for Shadow Memory in Multiprocessors. In *the Proc. of the 5th Conference on Virtual Execution Environments (VEE)*, Washington D.C., March 2009.
- [62] Vijay Nagarajan, Ho-Seop Kim, Youfeng Wu, and Rajiv Gupta. Dynamic Information Tracking on Multicores. In *the Proc. of the 12th Workshop on the Interaction between Compilers and Computer Architecture (INTERACT)*, Salt Lake City, UT, February 2008.

- [63] National Institute of Science and Technology (NIST), Department of Commerce. Software Errors cost the U.S economy \$59.5 billion annually. NIST News Release 2002-10, June 2002.
- [64] Nergal. The advanced return-into-lib(c) exploits: PaX case study. In *Phrack Magazine*, 2001. Issue 58, Article 4.
- [65] Nicholas Nethercote. *Dynamic Binary Analysis and Instrumentation*. PhD thesis, University of Cambridge, November 2004.
- [66] James Newsome and Dawn Xiaodong Song. Dynamic Taint Analysis for Automatic Detection, Analysis, and Signature Generation of Exploits on Commodity Software. In *the Proc. of the 12th NDSS*, San Diego, CA, February 2005.
- [67] A. Nguyen-Tuong, S. Guarnieri, D. Greene, J. Shirley, and D. Evans. Automatically Hardening Web Applications using Precise Tainting. In *Proc. of the 20th IFIP Intl. Information Security Conference*, Chiba, Japan, May 2005.
- [68] V. Orgovan and M. Tricker. An introduction to driver quality, Aug 2003.
- [69] The Pentium Datasheet, Intel, 1997. <http://www.intel.com>.
- [70] Perl taint mode. <http://www.perl.com>.
- [71] Tadeusz Pietraszek and Chris Vanden Berghe. Defending against Injection Attacks through Context-Sensitive String Evaluation. In *the Proc. of the Recent Advances in Intrusion Detection Symposium*, Seattle, WA, September 2005.
- [72] President's Information Technology Advisory Committee (PITAC). CyberSecurity: A Crisis of Prioritization. http://www.nitrd.gov/pitac/reports/20050301_cybersecurity/cybersecurity.pdf, February 2005.

- [73] Feng Qin, Cheng Wang, Zhenmin Li, Ho-Seop Kim, Yuanyuan Zhou, and Youfeng Wu. LIFT: A Low-Overhead Practical Information Flow Tracking System for Detecting Security Attacks. In *the Proc. of the 39th International Symposium on Microarchitecture (MICRO)*, Orlando, FL, December 2006.
- [74] Mohan Rajagopalan, Matti Hiltunen, Trevor Jim, and Richard Schlichting. Authenticated System Calls. In *the Proc. of the 35th International Conference on Dependable Systems and Networks (DSN)*, Yokohama, Japan, June 2005.
- [75] Mohan Rajagopalan, Matti Hiltunen, Trevor Jim, and Richard Schlichting. System call monitoring using authenticated system calls. *IEEE Trans. on Dependable and Secure Computing*, 3(3):216–229, 2006.
- [76] Joanna Rutkowska and Rafal Wojtczuk. Preventing and detecting Xen hypervisor subversions. <http://invisiblethingslab.com/bh08/part2-full.pdf>, August 2008.
- [77] Bratin Saha, Ali-Reza Adl-Tabatabai, and Quinn Jacobson. Architectural Support for Software Transactional Memory. In *the Proc. of the 39th International Symposium on Microarchitecture (MICRO)*, Orlando, FL, December 2006.
- [78] Michael D. Schroeder and Jerome H. Saltzer. A hardware architecture for implementing protection rings. *Commun. ACM*, 15(3):157–170, 1972.
- [79] Weidong Shi, Joshua Fryman, Hsein-Hsin Lee, Youtao Zhang, and Jun Yang. InfoShield: A Security Architecture for Protecting Information Usage in Memory. In *the Proc. of the 12th International Conference on High-Performance Computer Architecture (HPCA)*, Austin, TX, 2006.
- [80] Personal communication with Shih-Lien Lu, Senior Principal Researcher, Intel Microprocessor Technology Labs, Hillsboro, OR.

- [81] G. Edward Suh, Jaewook Lee, and Srinivas Devadas. Secure Program Execution via Dynamic Information Flow Tracking. In *the Proc. of the 11th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, Boston, MA, October 2004.
- [82] Taeweon Suh, Douglas Blough, and Hsein-Hsin Lee. Supporting Cache Coherence in Heterogeneous Multiprocessor Systems. In *the Proc. of the Symposium on Design, Automation and Test in Europe (DATE)*, Paris, France, February 2004.
- [83] Symantec Internet Security Threat Report, Volume X: Trends for January 06 - June 06, September 2006.
- [84] David Thomas and Andrew Hunt. *Programming Ruby: the pragmatic programmers guide*, August 2005.
- [85] Shyamkumar Thoziyoor, Naveen Muralimanohar, Jung Ho Ahn, and Norman Jouppi. Cacti 5.1, 2008. HPL Technical Report HPL-2008-20.
- [86] Omesh Tickoo, Hari Kannan, Vineet Chadha, Ramesh Illikkal, Ravi Iyer, and Donald Newell. qTLB: Looking inside the Look-aside buffer. In *the 14th International Conference on High Performance Computing (HiPC)*, Goa, India, December 2007.
- [87] Neil Vachharajani, Matthew J. Bridges, Jonathan Chang, Ram Rangan, Guilherme Ottoni, Jason Blome, George Reis, Manish Vachharajani, and David August. RIFLE: An Architectural Framework for User-Centric Information-Flow Security. In *the Proc. of the 37th International Symposium on Microarchitecture (MICRO)*, Portland, OR, December 2004.
- [88] Guru Venkataramani, Ioannis Doudalis, Yan Solihin, and Milos Prvulovic. FlexiTaint: A Programmable Accelerator for Dynamic Taint Propagation. In *the Proc. of the 14th*

International Conference on High-Performance Computer Architecture (HPCA), Salt Lake City, UT, February 2008.

- [89] Christopher Weaver, Joel Emer, Shubu Mukherjee, and Steve Reinhardt. Techniques to Reduce the Soft Error Rate of a High-Performance Microprocessor. In *the Proc. of the 31st International Symposium on Computer Architecture (ISCA)*, Munchen, Germany, June 2004.
- [90] Emmett Witchel, Josh Cates, and Krste Asanovic. Mondrian memory protection. In *Proc. of the 10th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, San Jose, CA, October 2002.
- [91] Steven Cameron Woo, Moriyoshi Ohara, Evan Torrie, Jaswinder Pal Singh, and Anoop Gupta. The SPLASH2 Programs: Characterization and Methodological Considerations. In *the Proceedings of the 22nd International Symposium on Computer Architecture (ISCA)*, Santa Margherita, Italy, June 1995.
- [92] Min Xu, Ras Bodik, and Mark Hill. A Regulated Transitive Reduction (RTR) for Longer Memory Race Recording. In *the Proc. of the 12th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, San Jose, CA, October 2006.
- [93] Wei Xu, Sandeep Bhatkar, and R. Sekar. Taint-enhanced policy enforcement: A practical approach to defeat a wide range of attacks. In *the Proc. of the 15th USENIX Security Symp.*, Vancouver, Canada, August 2006.
- [94] Nickolai Zeldovich, Silas Boyd-Wickizer, Eddie Kohler, and David Mazières. Making information flow explicit in HiStar. In *Proc. of the 7th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, Seattle, WA, November 2006.

- [95] Nickolai Zeldovich, Silas Boyd-Wickizer, and David Mazières. Securing distributed systems with information flow control. In *Proc. of the 5th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, San Francisco, CA, April 2008.
- [96] Nickolai Zeldovich, Hari Kannan, Michael Dalton, and Christos Kozyrakis. Hardware Enforcement of Application Security Policies using Tagged Memory. In *the Proc. of the 8th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, San Diego, CA, December 2008.
- [97] Pin Zhou, Feng Qin, Wei Liu, Yuanyuan Zhou, and Josep Torrellas. iWatcher: Efficient architectural support for software debugging. In *the Proc. of the 31st International Symposium on Computer Architecture (ISCA)*, June 2004.